RV1126B Linux IPC SDK 快速开发指南

文档标识：RK-JC-YF-A44

发布版本：V1.6.0

日期：2025-09-30

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文主要描述了Linux IPC SDK的基本使用方法，旨在帮助开发者快速了解并使用SDK开发包。

产品版本

芯片名称	内核版本
RV1126B/RV1126B-P	Linux 6.1

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	CWW	2025-03-28	初始版本
V1.1.0	CWW	2025-04-17	增加如何下载NPU模型转换工具以及runtime库
V1.2.0	CWW	2025-06-12	1. 增加如何查看多媒体驱动部分内存使用情况 2. 更新安全启动相关代码以及文档说明 3. 添加运动DV 介绍
V1.2.1	CWW	2025-06-16	1. 更新BoardConfig.mk的配置项说明
V1.3.0	SYF	2025-06-19	增加快启流程基本框架
V1.4.0	CWW、LLDM	2025-08-29	1. 增加获取eMMC支持列表 2. 增加获取DDR支持列表 3. 更新BoardConfig.mk的配置项说明 4. 增加EIS防抖说明 5. 增加AVS说明 6. 增加AI-ISP说明 7. 增加AI-Remosaic说明 8. 增加FEC说明 9. 增加如何读取CPU序列号 10. 增加如何开启 DDR 变频 11. 更新如何使用DM-Verity和DM-Crypt功能 12. 更新如何使用Recovery 13. 更新安全启动相关代码以及文档说明
V1.5.0	SYF	2025-09-28	1. 增加快启各个阶段耗时统计介绍 2. 增加MCU快启驱动开发说明
V1.6.0	CWW	2025-09-30	增加如何测试大语言模型（LLM）和视觉语言模型（VLM）

目录

RV1126B Linux IPC SDK 快速开发指南

1. RV1126B和RV1126B-P差异说明
    1.1 规格差异
2. SDK参考硬件EVB板使用说明
    2.1 RV1126B_EVB1_V10接口说明
    2.2 RV1126B-P_EVB1_V10接口说明
        2.2.1 上电说明
        2.2.2 固件烧录说明
        2.2.3 按键说明
        2.2.4 视频预览
        2.2.5 音频（MIC）测试
        2.2.6 Web直播（RTMP）
        2.2.7 Wi-Fi
        2.2.8 RTC
        2.2.9 TF卡
3. 服务器环境搭建
4. RV1126B Linux IPC SDK软件包获取方法
    4.1 下载授权
    4.2 通过代码服务器下载
    4.3 通过本地压缩包解压获取
    4.4 更新SDK代码
    4.5 交叉工具链下载以及安装
5. SDK 使用说明
    5.1 BoardConfig.mk的配置项说明
    5.2 查看SDK版本以及编译配置
    5.3 一键自动编译
    5.4 编译U-Boot
    5.5 编译kernel
    5.6 编译rootfs
    5.7 编译media
    5.8 编译参考应用
    5.9 编译内核驱动
    5.10 打包env.img
    5.11 固件打包
    5.12 SDK目录结构说明
        5.12.1 Sysdrv目录说明
        5.12.2 Media目录说明
        5.12.3 App目录说明
    5.13 镜像存放目录说明
    5.14 调试工具
        5.14.1 通过网络tftp传输文件
        5.14.2 通过网络ADB调试
        5.14.3 通过Telnet调试
        5.14.4 通过串口调试
6. 文档说明
7. 工具说明
    7.1 驱动安装工具
    7.2 固件烧录
    7.3 update.img相关工具
        7.3.1 打包

7.3.2 解包
7.3.3 update.img烧写
7.4 SD升级启动制作工具
7.5 工厂固件说明
7.6 量产升级工具
8. CVR 框架介绍
8.1 功能简介
8.2 软件框架
8.3 APP介绍
8.4 功能模块说明
8.5 LVGL介绍
8.5.1 LVGL配置文件
8.6 rkadk介绍
8.6.1 rkadk 功能列表
8.6.2 rkadk API说明
9. 运动DV 介绍
9.1 功能简介
9.2 使用说明
9.2.1 硬件需求
9.2.2 编译配置
9.2.3 应用功能
9.3 代码仓库
9.4 基本数据流
10. EIS防抖说明
10.1 算法配置参数
10.2 驱动调试说明
11. 电池快启IPC介绍
11.1 快启流程说明
11.1.1 快启流程基本框架
11.1.2 快启固件配置及编译
11.1.3 快启应用说明
11.1.3.1 双摄快启demo说明
11.1.3.2 单摄快启demo说明
11.1.3.3 预览画面
11.1.4 快速启动日志说明
11.1.5 快启各个阶段耗时统计介绍
11.2 光敏使用说明
11.2.1 光敏类型
11.2.2 光敏参数
11.2.3 模拟光敏标定方法
11.2.4 软光敏快启流程说明
11.3 meta分区说明
11.3.1 meta信息头说明
11.3.2 Sensor初始化参数说明
11.3.3 Camera Sensor IQ 参数说明
11.3.4 AE/AWB参数说明
11.3.5 应用参数说明
11.3.6 内核CMDLINE参数说明
11.3.7 生成meta.img
11.3.8 单板端meta分区操作
11.3.8.1 查看meta分区数据
11.3.8.2 更新meta分区数据
11.3.8.3 更新cmdline，使能内核打印开机log

11.3.8.4 更新cmdline，使能摄像头AIQ的调试log
11.3.8.5 更新Sensor初始化
11.3.8.6 更新Sensor IQ文件
11.3.8.7 其他参数说明
11.4 修改系统内存布局
11.4.1 修改内核DTS的内存布局
11.4.2 修改FIT打包脚本
11.4.3 重新打包编译
11.5 MCU快启驱动开发说明
12. AVS说明
13. AI-ISP说明
14. AI-Remosaic说明
15. FEC说明
16. IPC Linux SDK Q&A
16.1 如何修改分区表和增加自定义分区以及分区可读写说明
16.1.1 存储介质以及文件系统类型说明
16.1.2 分区表说明
16.1.3 增加自定义分区
16.1.4 oem分区挂载说明
16.2 如何在U-Boot终端下使用tftp进行分区升级
16.3 如何在U-Boot终端下使用SD卡进行分区升级
16.4 如何在U-Boot阶段通过按键触发SD卡升级功能
16.5 插入SD卡检测不到设备
16.6 如何添加第三方库到sysdrv目录编译
16.7 如何在project/app里增加新应用程序
16.8 如何查看多媒体驱动部分内存使用情况
16.9 如何使用coredump功能
16.10 内核驱动insmod说明
16.11 RV1126B平台相关库文件和驱动文件的信息
16.12 如何使用NFS文件系统
16.13 如何增加新用户并设置登陆密码
16.14 如何使用OTA升级
16.14.1 如何使用Recovery
16.15 A/B系统使用方法
16.15.1 启动方案介绍
16.15.2 配置
16.15.3 OTA升级工具
16.15.4 A/B系统切换
16.15.5 A/B系统升级
16.16 获取摄像头支持列表
16.17 获取Flash支持列表
16.18 获取eMMC支持列表
16.19 获取DDR支持列表
16.20 压力测试使用方法
16.20.1 memtester test
16.20.2 stressapptest
16.20.3 cpufreq test
16.20.4 flash stress test
16.20.5 reboot test
16.21 如何使用rndis功能
16.22 如何增加非root用户登陆
16.23 如何添加新的Camera sensor配置
16.24 如何添加reboot命令进U-Boot终端

16.25 如何通过指令修改GPIO寄存器配置
16.25.1 U-Boot终端
16.25.2 根文件系统
16.26 如何修改串口波特率
16.26.1 ddr bin文件
16.26.2 内核dts
16.27 启用 sysstat 测试套件
16.28 如何使用DM-Verity和DM-Crypt功能
16.29 如何使用UVC预览
16.30 安全启动相关代码以及文档说明
16.30.1 U-Boot配置
16.30.2 rkbin配置
16.30.3 BoardConfig配置
16.30.4 生成Key
16.30.5 固件签名
16.30.6 应用场景
16.31 如何下载NPU模型转换工具以及runtime库
16.32 如何测试大语言模型（LLM）和视觉语言模型（VLM）
16.33 如何读取CPU序列号
16.34 如何开启 DDR 变频
17. 注意事项

1. RV1126B和RV1126B-P差异说明

RV1126B是全功能版本，包含RV1126B完整功能，比如：USB3.0、Audio Codec、以太网PHY等。RV1126B-P是Pin2Pin RV1126的版本，客户之前使用RV1126开发的产品可以直接用RV1126B-P替代，不用重新设计硬件。

1.1 规格差异

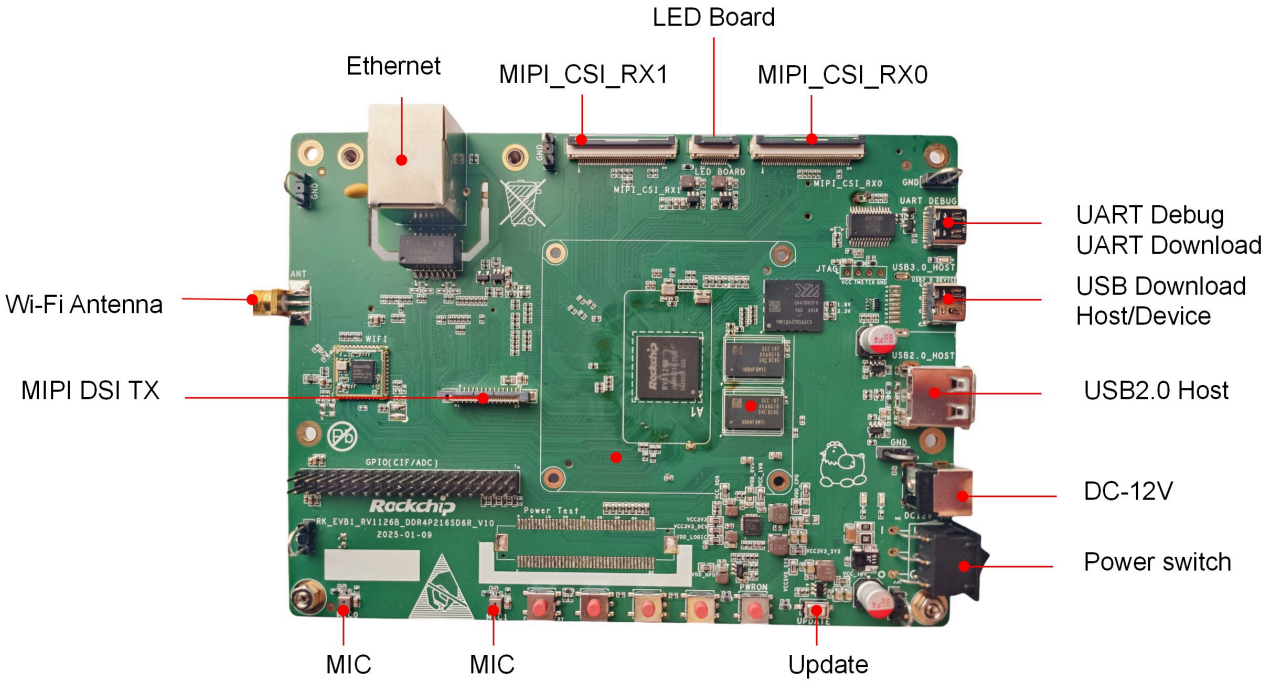
规格	RV1126	RV1126B-P（pin to pin）	RV1126B
CPU	4*A7	4*A53	4*A53
Audio	SAI(I2S/PCM/TDM)+PDM+ 2*DSM(PWM Audio)	SAI(I2S/PCM/TDM)+PDM+ 2*DSM(PWM Audio)	SAI(I2S/PCM/TDM)+PDM+ 2*DSM(PWM Audio)+2*Audio ADC
SARADC	6	6	24
PWM	16	28	28
MIPI CSI	2 x 4lane	2 x 4lane	2 x 4lane/4 x 2lane
USB	USB2.0 DRD+USB2.0 HOST	USB2.0 DRD+USB2.0 HOST	USB2.0/3.0 DRD+USB2.0 HOST
RTC	N/A	N/A	Built-in
Ethernet	GMAC	GMAC	GMAC+FEPHY

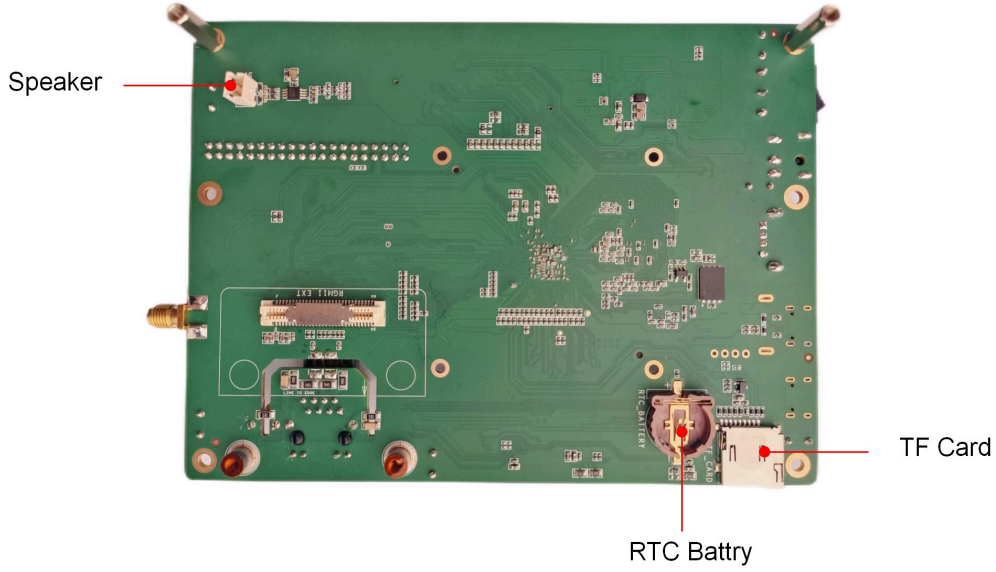
详细的差异请参考硬件文档《RV1126&RV1126B-P_Differences_Description_and_Compatibility_Design_Notes_V1.26_20250604_CN&EN.pdf》。

2. SDK参考硬件EVB板使用说明

2.1 RV1126B_EVB1_V10接口说明

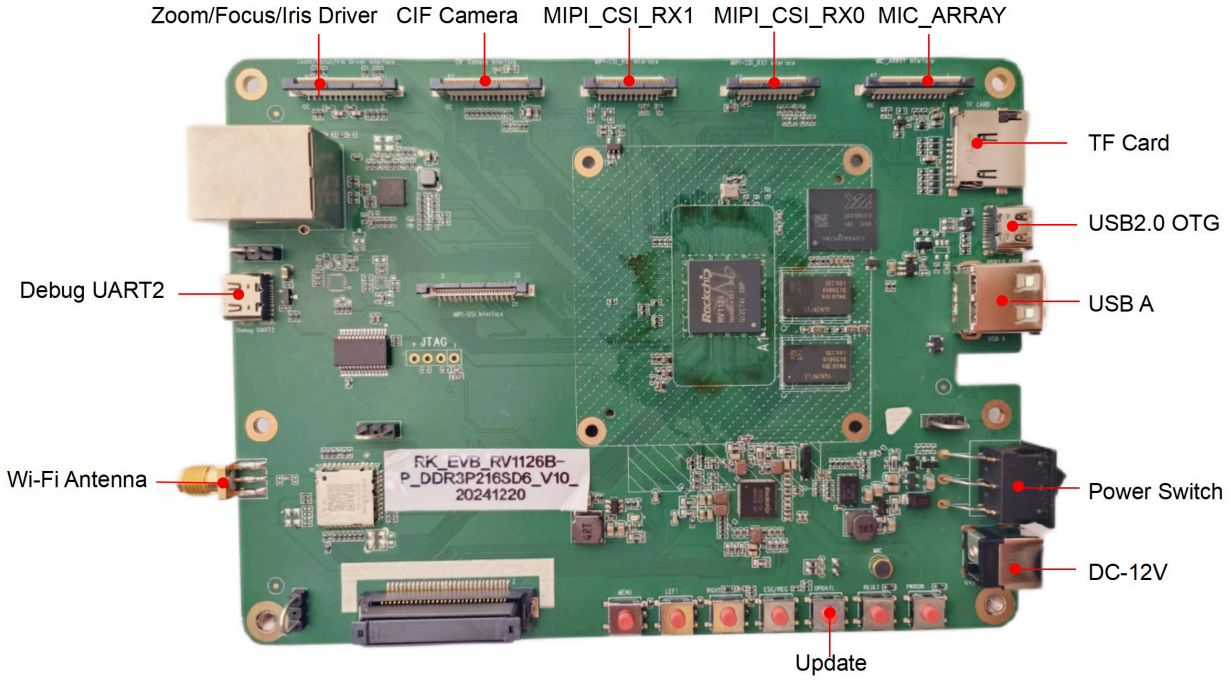
以下是RV1126B EVB1 V10的硬件布局。

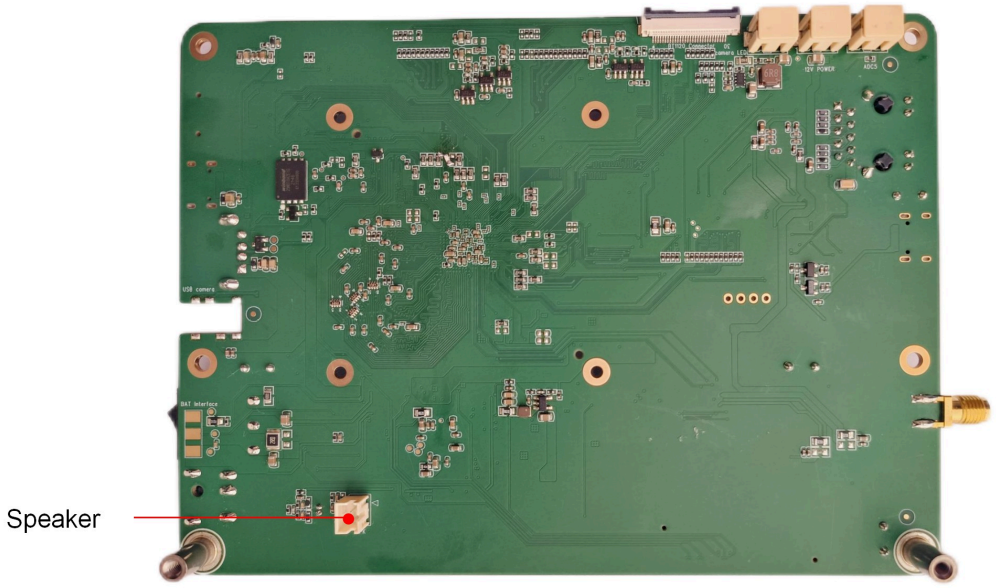




2.2 RV1126B-P_EVB1_V10接口说明

以下是RV1126B-P EVB1 V10的硬件布局。





2.2.1 上电说明

12V直流电源。

2.2.2 固件烧录说明

请参考固件烧录。

2.2.3 按键说明

按键	用途
Update	设备烧录与升级
MENU/LEFT/RIGHT/PWREON/RECOVERY	预留

2.2.4 视频预览

设备支持在同一个局域网中预览，在设备联网后，使用PC端的RTSP软件（如VLC）打开网络串流，输入以下地址：

```
rtsp://（你的设备的IP地址）/live/0
```



即可预览摄像头的画面：

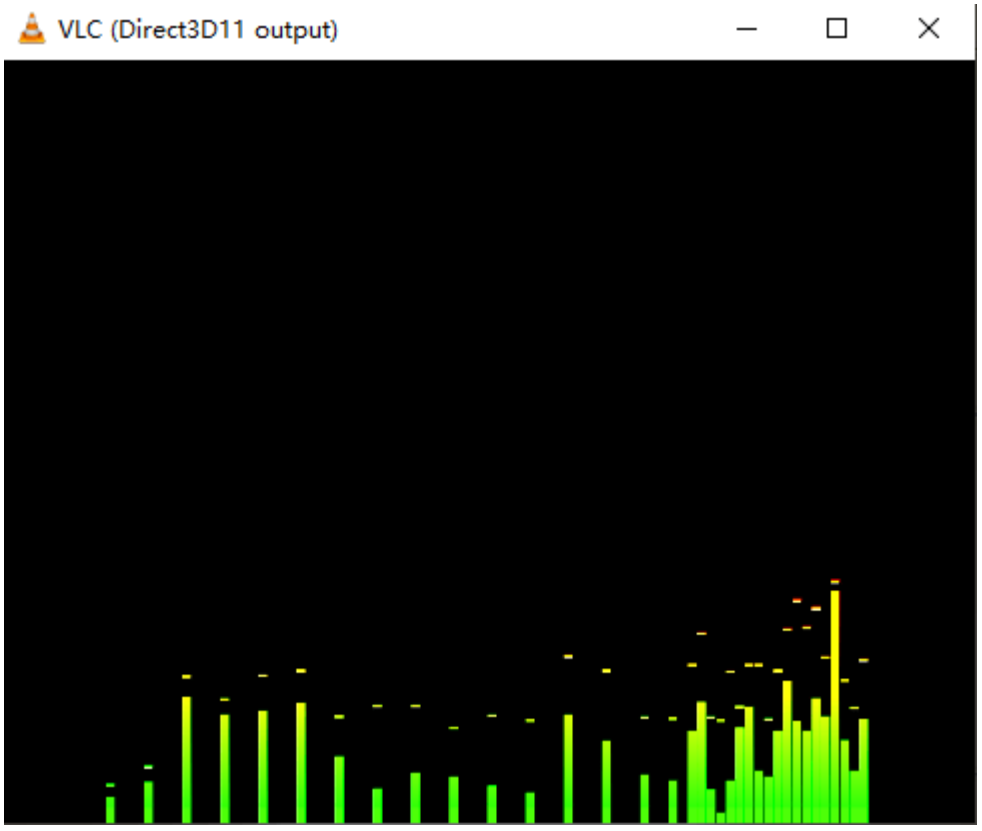


2.2.5 音频（MIC）测试

通过PC端访问RTSP码流预览，可同时获取到音频。音频可直接扬声器外放，或使用软件自带的可视化音频进行检测，音频频谱查看方法如下：



频谱效果如下：

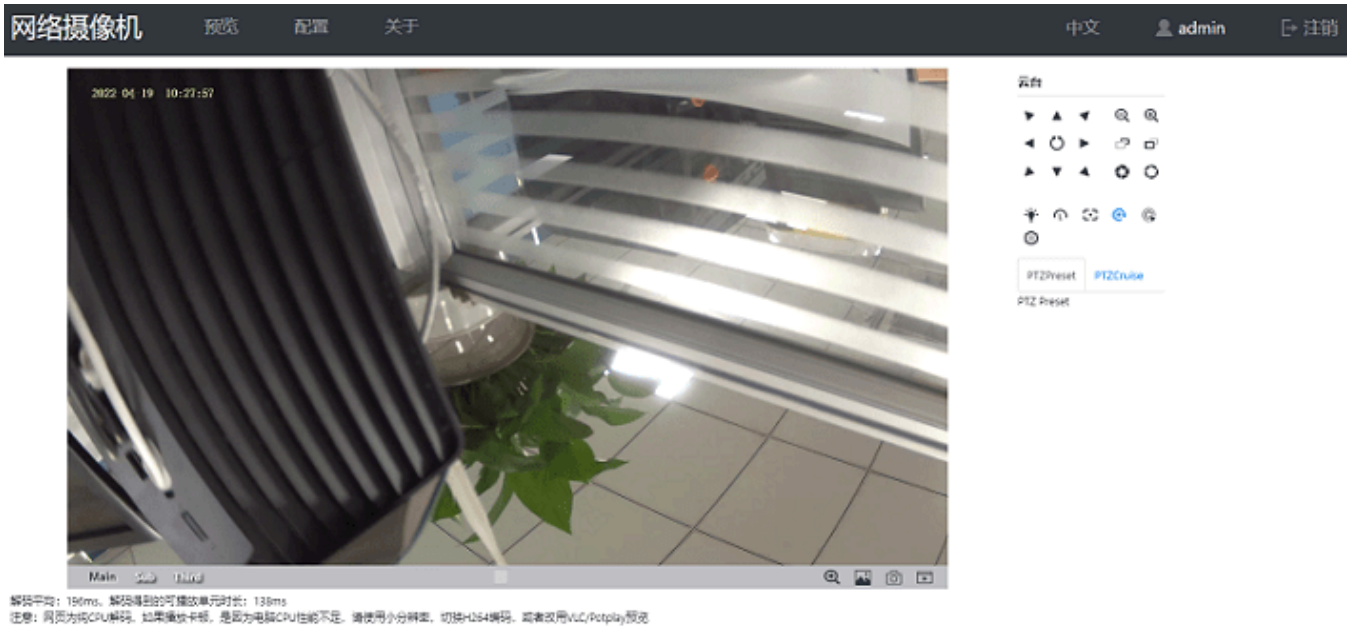


2.2.6 Web直播（RTMP）

Web端支持主码流、子码流，H264/H265直播。通过浏览器访问设备IP地址，输入用户名和密码（默认均为admin），进入直播预览系统。



预览效果如下：



2.2.7 Wi-Fi

Wi-Fi功能默认未开启，使用时需要进行如下配置：

```
cd /oem/usr/ko/
./insmod_wifi.sh
```

运行成功后将在`ifconfig`中得到wlan0：

```
# ifconfig
eth0      Link encap:Ethernet  HWaddr DA:54:55:F7:10:18
          inet addr:192.168.1.169  Bcast:255.255.255.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:58 errors:0 dropped:0 overruns:0 frame:0
          TX packets:6 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6852 (6.6 KiB)  TX bytes:2052 (2.0 KiB)
          Interrupt:45

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:356100 errors:0 dropped:0 overruns:0 frame:0
          TX packets:356100 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:106376267 (101.4 MiB)  TX bytes:106376267 (101.4 MiB)

wlan0     Link encap:Ethernet  HWaddr F0:B0:40:69:53:17
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:375 errors:0 dropped:325 overruns:0 frame:0
          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:107823 (105.2 KiB)  TX bytes:288 (288.0 B)
```

最终得到网络信息：

```
rkwifi_server connect <SSID> <password>
```

```
wlan0     Link encap:Ethernet  HWaddr F0:B0:40:69:53:17
          inet addr:192.168.1.155  Bcast:255.255.255.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:506 errors:0 dropped:419 overruns:0 frame:0
          TX packets:4 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:141895 (138.5 KiB)  TX bytes:1028 (1.0 KiB)
```

测试：

```
# ping www.baidu.com
PING www.baidu.com (163.177.151.109): 56 data bytes
64 bytes from 163.177.151.109: seq=0 ttl=54 time=25.981 ms
64 bytes from 163.177.151.109: seq=1 ttl=54 time=26.812 ms
64 bytes from 163.177.151.109: seq=2 ttl=54 time=50.451 ms
64 bytes from 163.177.151.109: seq=3 ttl=54 time=56.411 ms
^C
--- www.baidu.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 25.981/39.913/56.411 ms
```

2.2.8 RTC

RTC需要装上电池（电池型号如CR1220），才能在断电上电后保持时钟不变。

下面是RTC的部分使用方法：

查看RTC信息：

```
hwclock
```

```
# hwclock
Fri Jan  1 12:52:21 2021  0.000000 seconds
```

使用RTC同步系统时钟：

```
hwclock -s
```

```
# date
Fri Jan  1 00:00:45 UTC 2021
# hwclock -r
Tue Apr 19 15:16:14 2022  0.000000 seconds
# hwclock -s
# date
Tue Apr 19 15:16:30 UTC 2022
# hwclock -r
Tue Apr 19 15:16:35 2022  0.000000 seconds
```

使用系统时钟同步RTC：

```
hwclock -w
```

```
# date
Tue Apr 19 15:20:04 UTC 2022
# hwclock -r
Fri Jan  1 00:00:40 2021  0.000000 seconds
# hwclock -w
# date
Tue Apr 19 15:20:14 UTC 2022
# hwclock -r
Tue Apr 19 15:20:17 2022  0.000000 seconds
#
```

2.2.9 TF卡

插入TF卡后开机，TF卡将会自动挂载至`/mnt/sdcard`下。

```
/dev/mmcblk1 on /mnt/sdcard type vfat (rw,noatime,uid=1000,
```

如需将录像保存至TF卡中，需要修改应用配置文件`rkipc.ini`，使能存储模块和配置挂载路径。

```
vi /data/rkipc.ini
```

以`/mnt/sdcard`挂载路径为例，修改如下图所示：

```
[storage]
mount_path                      = /mnt/sdcard
free_size_del_min               = 500
free_size_del_max               = 1000


[storage.0]
enable                          = 1
folder_name                     = video0
file_format                     = mp4
file_duration                   = 60
video_quota                     = 30
```

重启设备，录像将写入`/mnt/sdcard/video0`中。

3. 服务器环境搭建

本 SDK 开发环境是在 Ubuntu 系统上开发测试。其他的 Linux 版本可能需要对软件包做相应调整。除了系统要求外，还有其他软硬件方面的要求。

硬件要求：64 位系统，硬盘空间大于 20G。如果您进行多个构建，将需要更大的硬盘空间。

软件要求：Ubuntu 18.04 系统

编译 SDK 环境搭建所依赖的软件包安装命令如下：

```
sudo apt-get install repo git ssh make gcc \
gcc-multilib g++-multilib module-assistant \
expect g++ gawk texinfo libssl-dev \
bison flex fakeroot cmake unzip gperf autoconf \
pkg-config lz4 gettext libtool automake autoconf-archive \
python2 python3 scons \
device-tree-compiler libncurses5-dev
```

建议使用 Ubuntu18.04 系统或更高版本开发，若编译遇到报错，可以视报错信息，安装对应的软件包。

4. RV1126B Linux IPC SDK软件包获取方法

4.1 下载授权

如您已经完成了账号开通，在下载代码前需要生成自己的 HTTPS 密码，请下载仓库`https://gerrit.rock-chips.com:8443/repo-release/tools/script`，通过仓库下的`script/Gernerate-Credential.sh`下载授权，如下所示：

```
mkdir ~/rk_gerrit_password
cd ~/rk_gerrit_password
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/script
cd ./script
./Generate-Credential.x
```

运行授权脚本的日志如下：

```
Adding configuration for Gerrit server to /home/Test123/rk_gerrit_password
********************************************************************************
********************************************************************************
**************************************************
 Copyright Statement

 Copyright (C) 2024 Rockchip Electronics Co., Ltd. All rights reserved.

 BY OPENING OR USING THIS FILE, RECEIVER HEREBY ACKNOWLEDGES AND AGREES THAT THE
SOFTWARE/FIRMWARE AND ITS DOCUMENTATIONS ("ROCKCHIP SOFTWARE") RECEIVED FROM ROCKCHIP ON AN
"AS-IS" BASIS ONLY WITHOUT ANY AND ALL WARRANTIES,
 EITHER EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, ANY WARRANTY OR
CONDITION WITH RESPECT TO TITLE, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE, OR
NON-INFRINGEMENT.
 NEITHER DOES ROCKCHIP PROVIDE ANY WARRANTY WHATSOEVER WITH RESPECT TO ANY OPEN SOURCE
TECHNOLOGIES, THIRD-PARTY TECHNOLOGIES OR ANY STANDARD TECHNOLOGIES WHICH MAY BE SUPPORTED
BY, INCORPORATED IN, OR SUPPLIED WITH THE ROCKCHIP SOFTWARE.
```

```
 RECEIVER EXPRESSLY ACKNOWLEDGES THAT IT IS RECEIVER'S SOLE RESPONSIBILITY TO OBTAIN AND
MAINTAIN ALL NECESSARY LICENSES AND RIGHTS FROM THEIR RESPECTIVE OWNERS TO USE ANY SUCH
THIRD-PARTY TECHNOLOGIES OR ANY STANDARD TECHNOLOGIES.
 RECEIVER'S SOLE AND EXCLUSIVE REMEDY AND ROCKCHIP'S ENTIRE AND CUMULATIVE LIABILITY WITH
RESPECT TO THE ROCKCHIP SOFTWARE RELEASED HEREUNDER WILL BE, AT ROCKCHIP 'S OPTION, TO
REVISE OR REPLACE THE ROCKCHIP SOFTWARE AT ISSUE,
 OR REFUND ANY FEES OR CHARGE PAID BY RECEIVER TO ROCKCHIP FOR SUCH ROCKCHIP SOFTWARE AT
ISSUE.
**********************************************************************************************
**********************************************************************************************
**************************************************
Do you agree to the above terms? (yes/no) yes
Please input your username (make sure to use the username exactly the same as the one in RK
email!!): Test123
Please input your email (make sure to use the company email address!!): Test123@rock-
chips.com
Please input your SSH private key filename (e.g. id_rsa):
Your Credential check has passed
New git credential will be added into your history!!
You are All Set!!
```

4.2 通过代码服务器下载

获取 SDK 软件包，需要有一个帐户访问 Rockchip 提供的源代码仓库。客户向瑞芯微技术窗口申请 SDK，同步提供SSH公钥进行服务器认证授权，获得授权后即可同步代码。关于瑞芯微代码服务器 SSH公钥授权，请参考SSH 公钥操作说明章节。

repo 是 google 用 Python 脚本写的调用 git 的一个脚本，主要是用来下载、管理项目的软件仓库，其下载地址如下：

```
mkdir -p $HOME/repo-tool
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/repo $HOME/repo-tool

export PATH="$HOME/repo-tool:$PATH"
# 测试命令
repo version
```

SDK 下载命令如下：

```
# 创建SDK目录
mkdir RV1126B_Linux_IPC_SDK
cd RV1126B_Linux_IPC_SDK

# 配置下载地址
repo init --repo-url \
   https://gerrit.rock-chips.com:8443/repo-release/tools/repo \
   -u https://gerrit.rock-chips.com:8443/linux/rockchip/ipc/manifests \
   -b rv1126b -m rv1126b_linux_ipc_release.xml

# 下载并同步代码
.repo/repo/repo sync --no-clone-bundle -c --no-tags -j4
```

4.3 通过本地压缩包解压获取

为方便客户快速获取 SDK 源码，瑞芯微技术窗口通常会提供对应版本的 SDK 初始压缩包，开发者可以通过这种方式，获得 SDK 代码的初始压缩包，该压缩包解压得到的源码，进行同步后与通过 repo 下载的源码是一致的。
以 RV1126B_Linux_IPC_SDK_VX.X.X_XXX.tar.bz2 为例，拷贝到该初始化包后，通过如下命令可检出源码：

```
mkdir RV1126B_Linux_IPC_SDK
tar xvf RV1126B_Linux_IPC_SDK_VX.X.X_XXX.tar.bz2 -C RV1126B_Linux_IPC_SDK
cd RV1126B_Linux_IPC_SDK
.repo/repo/repo sync -l
.repo/repo/repo sync --no-clone-bundle -c --no-tags -j4
```

后续开发者可根据 FAE 窗口定期发布的更新说明，通过 `.repo/repo/repo sync --no-clone-bundle -c --no-tags -j4` 命令同步更新。

4.4 更新SDK代码

更新SDK代码前，需要备份本地修改。

```
# 更新SDK代码命令
.repo/repo/repo sync -c --no-tags
# 如果有仓库下载不了，可以加上强制更新的参数 --force-sync
.repo/repo/repo sync -c --no-tags --force-sync
# 更新SDK代码后，需要进行clean操作
./build.sh clean
```

4.5 交叉工具链下载以及安装

交叉工具链可以从SDK目录下tools/linux/toolchain/获取。

芯片名称	交叉工具链	描述
RV1126B	aarch64-rockchip1240-linux-gnu-gcc	用于编译U-Boot
RV1126B	arm-rockchip1240-linux-gnueabihf-gcc	用于编译内核、应用

```
cd tools/linux/toolchain/arm-rockchip1240-linux-gnueabihf
source env_install_toolchain.sh
# or install toolchain to the dirname
# source env_install_toolchain.sh dirname
cd ../aarch64-rockchip1240-linux-gnu
source env_install_toolchain.sh
```

5. SDK 使用说明

5.1 BoardConfig.mk的配置项说明

- 板级配置文件BoardConfig.mk说明

SDK的板级配置在`project/cfg/`目录下，BoardConfig.mk文件是SDK编译的重要文件。
`project/cfg-all-items-introduction.txt`会记录最新配置项说明。

配置项	说明
RK_ARCH	arm或arm64 定义编译32位或64位程序
RK_CHIP	不可修改 不同的芯片对应不同的SDK
RK_TOOLCHAIN_CROSS	不可修改 定义交叉工具链
RK_BOOT_MEDIUM	emmc或spi_nor或spi_nand 定义板子存储类型
RK_UBOOT_DEFCONFIG	U-Boot defconfig文件名 文件目录sysdrv/source/uboot/u-boot/configs
RK_UBOOT_DEFCONFIG_FRAGMENT	U-Boot config文件名（可选） 文件目录sysdrv/source/uboot/u-boot/configs 对RK_UBOOT_DEFCONFIG定义的defconfig进行覆盖
RK_KERNEL_DEFCONFIG	内核defconfig文件名 文件目录sysdrv/source/kernel/arch/$RK_ARCH/configs
RK_KERNEL_DEFCONFIG_FRAGMENT	内核defconfig文件名（可选） 文件目录sysdrv/source/kernel/arch/$RK_ARCH/configs 对RK_KERNEL_DEFCONFIG定义的defconfig进行覆盖
RK_KERNEL_DTS	内核dts文件名 RK_ARCH=arm目录： sysdrv/source/kernel/arch/arm/boot/dts RK_ARCH=arm64目录： sysdrv/source/kernel/arch/arm64/boot/dts/rockchip
RK_MISC	如果打开recovery功能，系统启动时读取标志选择进recovery系统或应用系统（没有recovery时，可以去掉）
RK_CAMERA_SENSOR_IQFILES	Camera Sensor的IQ配置文件 文件目录media/isp/camera_engine_rkaiq/iqfiles或 media/isp/camera_engine_rkaiq/rkaiq/iqfiles 多个IQ文件用空格隔开，例如RK_CAMERA_SENSOR_IQFILES="iqfile_1 iqfile_2"
RK_PARTITION_CMD_IN_ENV	配置分区表（重要） 分区表格式：<partdef>[,<partdef>] <partdef>格式：<size>[@<offset>](part-name) 详细配置参考分区表说明章节
RK_PARTITION_FS_TYPE_CFG	配置分区文件系统类型以及挂载点（重要） 格式说明： 分区名称@分区挂载点@分区文件系统类型 注：根文件系统的分区挂载点默认值是IGNORE（不可修改）
RK_SQUASHFS_COMP	配置squashfs镜像压缩算法（可选） 支持：lz4/lzo/lzma/xz/gzip (default xz)
RK_UBIFS_COMP	配置ubifs镜像压缩算法（可选） 支持：lzo/zlib (default lzo)
RK_APP_TYPE	配置编译参考的应用（可选） 运行./build.sh info 可以查看支持的参考应用
RK_APP_IPCWEB_BACKEND	配置是否编译web应用（可选） y:使能
RK_BUILD_APP_TO_OEM_PARTITION	配置是否将应用安装到oem分区（可选） y:使能

配置项	说明
RK_ENABLE_RECOVERY	配置是否使能recovery功能（可选） y:使能 n:关闭
RK_ENABLE_FASTBOOT	配置是否快速启动功能（可选） y:使能 需要配合U-Boot和内核修改，可以参考SDK提供的BoardConfig-*-TB.mk
RK_ENABLE_GDB	配置是否编译gdb（可选） y:使能 n:关闭
RK_ENABLE_ADBD	配置是否支持adb功能（可选） y:使能 n:关闭 注：需要内核打开对应USB配置
RK_BOOTARGS_CMA_SIZE	配置内核CMA大小（可选）
RK_POST_BUILD_SCRIPT	配置的脚本将会在打包rootfs.img前执行（脚本放在BoardConfig对应目录下）（可选）
RK_PRE_BUILD_OEM_SCRIPT	配置的脚本将会在打包oem.img前执行（脚本放在BoardConfig对应目录下）（可选）
RK_BUILD_APP_TO_OEM_PARTITION	配置是否将应用安装到oem分区（可选）y:使能
RK_ENABLE_RNDIS	配置是否打开rndis功能（可选）y:使能 n:关闭
RK_META_PARAM	配置meta分区参数（可选，用于电池IPC类产品）
RK_AIISP_MODEL	配置AI-ISP算法模型文件名 *.aiisp
RK_AOV_APP_BUILD_STATIC	配置AOV应用使用静态编译 y:使能 n:关闭
RK_AOV_DEFAULT_APP	配置AOV默认的参考应用
RK_AUDIO_MODEL	配置音频算法模型 rkaudio*.rknn
RK_AVS_CALIB	配置AVS的校准文件 output/out/media_out/avs_calib
RK_CAMERA_SENSOR_CAC_BIN	配置摄像头的去紫边映射文件 output/out/media_out/isp_iqfiles/CAC*
RK_ENABLE_AOV	配置使能AOV功能 y:使能 n:关闭
RK_ENABLE_EUDEV	配置udev y:使能 n:关闭
RK_ENABLE_FIO	配置FIO工具 y:使能 n:关闭
RK_ENABLE_I2C_TOOLS	配置I2C工具 y:使能 n:关闭

配置项	说明
RK_ENABLE_LIBDRM	配置libdrm y:使能 n:关闭
RK_ENABLE_LVGL	配置LVGL y:使能 n:关闭
RK_ENABLE_ROCKCHIP_TEST	配置rockchip test y:使能 n:关闭
RK_ENABLE_SYSSTAT	配置sysstat y:使能 n:关闭
RK_ENABLE_WIFI	配置WiFi y:使能 n:关闭
RK_ENABLE_WIFI_CHIP	配置Wifi芯片驱动 sysdrv/drv_ko/wifi/Makefile
RK_META_SIZE	配置meta分区大小
RK_NAND_BLOCK_SIZE	配置Nand Flash的block大小
RK_NAND_PAGE_SIZE	配置Nand Flash的page大小
RK_NPU_MODEL	配置NPU模型文件名
RK_RECOVERY_KERNEL_DEFCONFIG_FRAGMENT	配置Recovery内核defconfig文件
RK_ROOTFS_JFFS2_ERASEBLOCK_4KB	配置SPI NOR是否按4KB擦除制作镜像
RK_UBOOT_RKBIN_MCU_CFG	配置快启MCU固件 ./build.sh mcu info
RK_UBOOT_RKBIN_MINIALL_INI_FILE	配置idblock.img打包的配置文件
RK_ENABLE_RAMDISK_PARTITION	配置ramdisk是否单独一个分区 y:使能 n:关闭 注：RK_PARTITION_CMD_IN_ENV需要增加rootfs分区， RK_PARTITION_FS_TYPE_CFG需要配置rootfs@IGNORE@erofs

- 选择BoardConfig的命令

`./build.sh lunch`

```
You're building on Linux
Lunch menu...pick a combo:

BoardConfig-*.mk naming rules:
BoardConfig-"启动介质"-"电源方案"-"硬件版本"-"应用场景".mk
BoardConfig-"boot medium"-"power solution"-"hardware version"-"application".mk

----------------------------------------------------------------
0. BoardConfig_IPC/BoardConfig-EMMC-RK801-RV1126B_EVB1_V10_V11-IPC.mk
                            boot medium(启动介质): EMMC
                          power solution(电源方案): RK801
```

```
                        hardware version(硬件版本): RV1126B_EVB1_V10_V11
                             application(应用场景): IPC
-----------------------------------------------------------------

-----------------------------------------------------------------
1. BoardConfig_IPC/BoardConfig-EMMC-RK809-RV1126BP_EVB1_V10_V11-IPC.mk
                             boot medium(启动介质): EMMC
                           power solution(电源方案): RK809
                        hardware version(硬件版本): RV1126BP_EVB1_V10_V11
                             application(应用场景): IPC
-----------------------------------------------------------------

-----------------------------------------------------------------
2. BoardConfig_IPC/BoardConfig-SPI_NAND-RK801-RV1126B_EVB1_V10_V11-IPC.mk
                             boot medium(启动介质): SPI_NAND
                           power solution(电源方案): RK801
                        hardware version(硬件版本): RV1126B_EVB1_V10_V11
                             application(应用场景): IPC
-----------------------------------------------------------------

-----------------------------------------------------------------
3. BoardConfig_IPC/BoardConfig-SPI_NOR-RK801-RV1126B_EVB1_V10_V11-IPC.mk
                             boot medium(启动介质): SPI_NOR
                           power solution(电源方案): RK801
                        hardware version(硬件版本): RV1126B_EVB1_V10_V11
                             application(应用场景): IPC
-----------------------------------------------------------------

Which would you like? [0]:
```

输入对应的序号选择对应的参考板级。

BoardConfig: BoardConfig_IPC	硬件说明	备注
BoardConfig-EMMC-RK801-RV1126B_EVB1_V10_V11-IPC.mk	用于RV1126B EVB1板子V10/V11 ,摄像头: SC450AI/SC850SL	支持AI-ISP
BoardConfig-EMMC-RK801-RV1126B_EVB1_V10_V11-IPC_DUAL_CAMERA.mk	用于RV1126B EVB1板子V10/V11 ,摄像头: SC450AI/SC850SL	双摄融合拼接
BoardConfig-EMMC-RK801-RV1126B_EVB1_V10_V11-IPC_FASTBOOT.mk	用于RV1126B EVB1板子V10/V11 ,摄像头: SC450AI/SC850SL	支持快速启动
BoardConfig-EMMC-RK801-RV1126B_EVB1_V10_V11-UVC.mk	用于RV1126B EVB1板子V10/V11 ,摄像头: SC450AI/SC850SL	支持UVC
BoardConfig-EMMC-RK809-RV1126BP_EVB1_V12-IPC.mk	用于RV1126B-P EVB1板子V12 ,摄像头: SC450AI/SC850SL	支持AI-ISP
BoardConfig-EMMC-RK801-RV1126B_EVB1_V10_V11-IPC_64BIT.mk	用于RV1126B EVB1板子V10/V11 ,摄像头: SC450AI/SC850SL	支持AI-ISP
BoardConfig-EMMC-RK809-RV1126BP_EVB1_V12-IPC_64BIT.mk	用于RV1126B-P EVB1板子V12 ,摄像头: SC450AI/SC850SL	支持AI-ISP
BoardConfig-SPI_NAND-RK801-RV1126B_EVB1_V10_V11-IPC.mk	用于RV1126B EVB1板子V10/V11 ,摄像头: SC450AI/SC850SL	
BoardConfig-SPI_NAND-RK801-RV1126B_EVB1_V10_V11-IPC_FASTBOOT.mk	用于RV1126B EVB1板子V10/V11 ,摄像头: SC450AI/SC850SL	支持快速启动
BoardConfig-SPI_NAND-RK809-RV1126BP_EVB1_V12-IPC.mk	用于RV1126B-P EVB1板子V12 ,摄像头: SC450AI/SC850SL	
BoardConfig-SPI_NOR-RK801-RV1126B_EVB1_V10_V11-IPC.mk	用于RV1126B EVB1板子V10/V11 ,摄像头: SC450AI/SC850SL	
BoardConfig-SPI_NOR-RK801-RV1126B_EVB1_V10_V11-IPC_FASTBOOT.mk	用于RV1126B EVB1板子V10/V11 ,摄像头: SC450AI/SC850SL	支持快速启动
BoardConfig-EMMC-RK801-RV1126B_EVB1_V10_V11-LLM_VLM_64BIT.mk	用于RV1126B EVB1板子V10/V11 ,摄像头: SC450AI/SC850SL	支持LLM/VLM测试

5.2 查看SDK版本以及编译配置

```
./build.sh info
```

板端查看SDK版本命令`sdkinfo`

```
# sdkinfo
```

5.3 一键自动编译

```
./build.sh lunch   # 选择参考板级
./build.sh         # 一键自动编译
```

5.4 编译U-Boot

```
./build.sh clean uboot
./build.sh uboot

# ./build.sh info 可以查看uboot详细的编译命令格式
```

生成镜像文件：output/image/download.bin、output/image/idblock.img 和 output/image/uboot.img

5.5 编译kernel

```
./build.sh clean kernel
./build.sh kernel

# ./build.sh info 可以查看kernel详细的编译命令格式
# 注：如果BoardConfig配置RK_ENABLE_FASTBOOT=y，还需要运行`./build.sh firmware`，才会生成boot.img。
```

生成镜像文件：output/image/boot.img

5.6 编译rootfs

```
./build.sh clean rootfs
./build.sh rootfs
```

编译后使用`./build.sh firmware`命令打包成rootfs.img
生成镜像文件：output/image/rootfs.img

5.7 编译media

```
./build.sh clean media
./build.sh media
```

生成文件的存放目录：output/out/media_out

5.8 编译参考应用

```
./build.sh clean app
./build.sh app
```

生成文件的存放目录：output/out/app_out
注：app依赖media

5.9 编译内核驱动

```
./build.sh clean driver
./build.sh driver
```

生成文件的存放目录：output/out/sysdrv_out/kernel_drv_ko/

5.10 打包env.img

```
./build.sh env
```

env.img是用uboot的mkenvimage工具进行打包。

env.img打包命令格式：`mkenvimage -s $env_partition_size -p 0x0 -o env.img env.txt`
注意：不同的存储介质，**$env_partition_size** 不一样，具体查看分区表说明

查看env.img内容：`strings env.img`

```
# 例如eMMC的env.txt内容
blkdevparts=mmcblk0:32K(env),512K@32K(idblock),256K(uboot),32M(boot),2G(rootfs),1G(oem),2G(u
serdata),-(media)
```

注意：不同的存储介质，env.img内容会不一样，可以用`strings env.img`查看。

blkdevparts会被uboot传递给内核，并覆盖内核对应bootargs的参数。

5.11 固件打包

```
./build.sh firmware
```

生成文件的存放目录： output/image

5.12 SDK目录结构说明

Directory Path	Introduction
build.sh	SDK编译脚本 软链接到project/build.sh
media	多媒体编解码、ISP等算法相关
sysdrv	U-Boot、kernel、rootfs目录
project	参考应用、编译配置以及脚本目录
docs	SDK文档目录
tools	烧录镜像打包工具以及烧录工具
output	SDK编译后镜像文件存放目录
output/image	烧录镜像输出目录
output/out	编译生成的文件
output/out/app_out	参考应用编译后的文件
output/out/media_out	media相关编译后的文件
output/out/sysdrv_out	sysdrv编译后的文件
output/out/sysdrv_out/kernel_drv_ko	外设和多媒体的ko文件
output/out/rootfs_xxx	文件系统打包目录
output/out/S20linkmount	分区挂载脚本
output/out/userdata	userdata

注：media和sysdrv可以独立SDK编译。

5.12.1 Sysdrv目录说明

sysdrv配置cfg/cfg.mk后，可以独立于SDK进行编译，包含U-Boot、kernel、rootfs以及一些镜像打包工具。

编译命令：

```
# 默认全部编译
make all

# 编译U-Boot
make uboot_clean
make uboot

# 编译内核
```

```
make kernel_clean
make kernel

# 编译rootfs
make rootfs_clean
make rootfs

# 清除编译
make clean

# 清除编译并删掉out目录
make distclean

# 查看编译配置，比如uboot、kernel详细的编译命令
make info
```

sysdrv子目录	说明
cfg	内核和U-Boot编译相关的配置
out	sysdrv编译输出目录
out/bin/board_*	运行在板端的程序
out/bin/pc	运行在PC端的程序
out/bin/image_*	生成的烧录镜像输出目录
out/bin/rootfs_*	根文件系统目录
source/busybox	busybox编译目录，源码在sysdrv/tools/board/busybox
source/busybox/objs_config_normal	busybox编译生成的文件
source/busybox/objs_config_tiny_arm	Recovery固件里busybox编译生成的文件
source/kernel	内核源码目录
source/objs_kernel	内核编译生成的文件
source/uboot	U-Boot源码目录以及rkbin（ddr初始化预编译镜像）
tools/board	板端程序源码
tools/pc	PC端打包镜像的工具

5.12.2 Media目录说明

media配置cfg/cfg.mk后，可以独立于SDK进行编译，包含多媒体编解码、ISP等算法相关。

编译命令：

```
# 默认全编译
make

# 清除编译文件
make clean

# 查看编译配置
make info
```

media子目录	说明
cfg	配置模块是否编译
alsa-lib	Advanced Linux Sound Architecture (ALSA) library
common_algorithm	音频3A算法、移动检测、遮挡检测
isp	isp图像处理算法
iva	智能视频分析算法
libdrm	Direct Rendering Manager
libv4l	video4linux2设备用户层接口
mpp	编解码接口，给rkmedia和rockit调用，不建议直接调用mpp
rga	RGA是一个独立的2D硬件加速器
rockit	多媒体接口（推荐）
sysutils	外设参考接口（ADC/GPIO/TIME/WATCHDOG）
samples	测试例程
out	media 编译输出目录

5.12.3 App目录说明

App目录project/app

project/app子目录	说明
component/rkadk	rkadk封装基础通用组件，如录像、拍照、播放、预览等，简化了应用开发难度
component/rkfsmk	优化存储相关的库（包含FAT32格式化，FAT32文件系统修复，MP4文件修复接口）
component/lvgl	一个开源的轻量图形库
rkipc	IPC参考应用
wifi_app	Wi-Fi参考应用
uvc_app_tiny	UVC参考应用
ipcweb	网页应用

5.13 镜像存放目录说明

编译后的烧录镜像在output/image目录下

固件镜像名称	说明
download.bin	烧录工具升级通讯的设备端程序，只会下载到板子内存
env.img	包含分区表和启动参数（SDK默认的env分区放在0地址）
idblock.img	loader镜像（包含DDR初始化），负责加载U-Boot
uboot.img	uboot镜像
boot.img	Linux内核镜像
rootfs.img	根文件系统镜像
oem.img	oem镜像（可选）
userdata.img	userdata镜像（可选）

5.14 调试工具

SDK支持adb和tftp工具用于PC和单板端文件传输。

5.14.1 通过网络tftp传输文件

```
### PC端的IP地址192.168.1.159
### 从PC端tftp服务器下载文件到单板
cd /tmp
tftp 192.168.1.159 -g -r test-file

### 从单板上传文件到PC端tftp服务器
tftp 192.168.1.159 -p -l test-file
```

注：tftp服务器配置请参考如何在U-Boot终端下使用tftp进行分区升级

5.14.2 通过网络ADB调试

```
### 获取EVB板的IP地址192.168.1.159
adb connect 192.168.1.159

adb devices
List of devices attached
192.168.1.159:5555      device

### adb登陆EVB板子调试
adb -s 192.168.1.159:5555 shell

### 从PC端上传文件test-file到EVB板的目录/userdata
adb -s 192.168.1.159:5555 push test-file /userdata/

### 下载EVB板上的文件/userdata/test-file到PC端
adb -s 192.168.1.159:5555 pull /userdata/test-file test-file
```

5.14.3 通过Telnet调试

```
### 设置IP地址
udhcpc -i eth0

### 在板端运行telnetd
telnetd
```

```
### 获取EVB板的IP地址192.168.1.159
### 在PC端运行telnet
telnet 192.168.1.159
### username: root
### password: rockchip
```

5.14.4 通过串口调试

芯片	PC端串口配置说明
RV1126B	波特率：1500000， 数据位：8，奇偶校验：无，停止位：1，流控制：无

6. 文档说明

```
docs/
├── zh ---------------------------- SDK的中文文档
│   ├── bsp
│   ├── isp
│   ├── npu_iva
│   ├── media
│   ├── security
│   └── ipc/Rockchip_RV1126B_Quick_Start_Linux_IPC_SDK_CN.pdf --- SDK中文Quick Start文档
│
└── en ---------------------------- SDK的英文文档
    ├── bsp
    ├── isp
    ├── npu_iva
    ├── media
    ├── security
    └── ipc/Rockchip_RV1126B_Quick_Start_Linux_IPC_SDK_EN.pdf --- SDK英文Quick Start文档
```

7. 工具说明

随 Rockchip Linux IPC SDK 发布的工具，用于开发调试阶段及量产阶段。工具版本会随SDK更新不断更新，如有工具上的疑问及需求，请联系我们的 FAE 窗口fae@rock-chips.com。
Rockchip Linux IPC SDK 中在 tools 目录下附带了linux（Linux操作系统环境下使用工具）、windows（Windows操作系统环境下使用工具）2个版本。

- Windows工具

工具说明文档：tools/windows/ToolsRelease.txt

工具名称	工具用途
SocToolKit	固件升级工具
DriverAssitant	驱动安装工具

- Linux工具

工具说明文档：tools/linux/ToolsRelease.txt

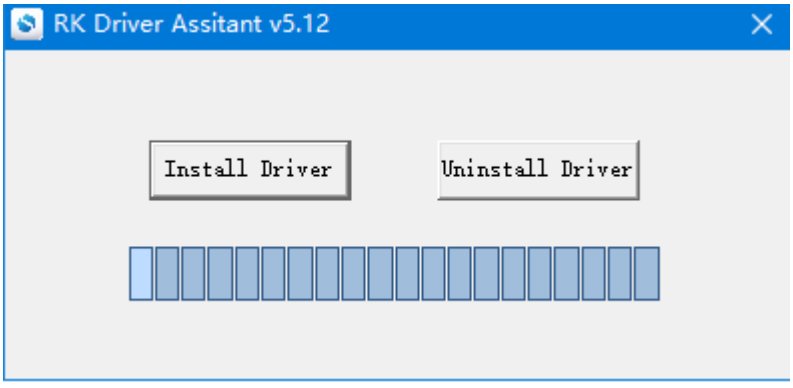
工具名称	工具用途
SocToolKit	固件升级工具
Linux_Upgrade_Tool	命令行烧录工具（只支持USB）

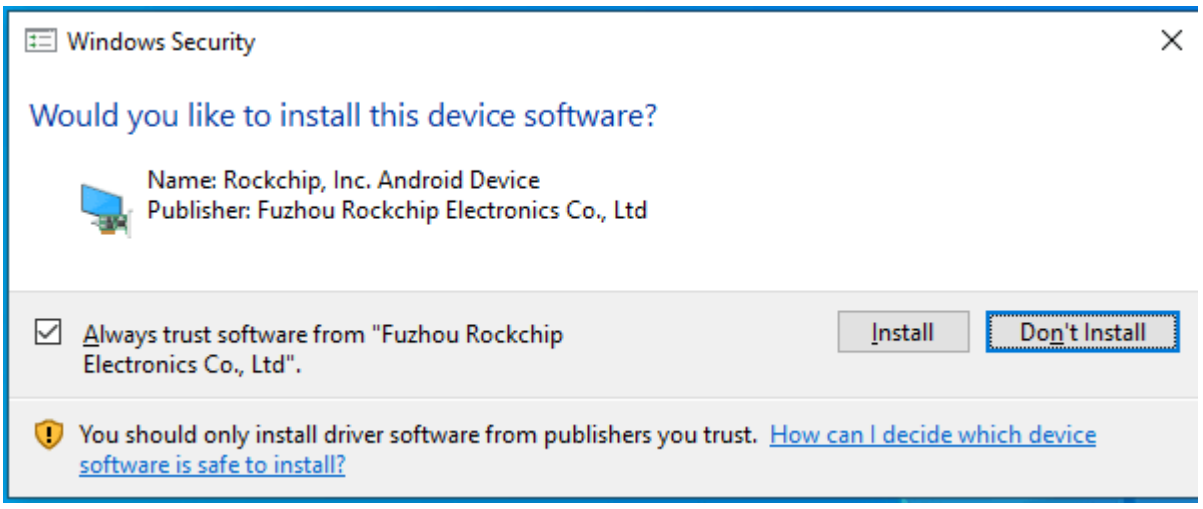
7.1 驱动安装工具

Rockchip USB 驱动安装助手存放在 `<SDK>/tools/windows/DriverAssitant_<版本>.zip`。支持win7_64, win10_64等操作系统。

安装步骤如下：







7.2 固件烧录

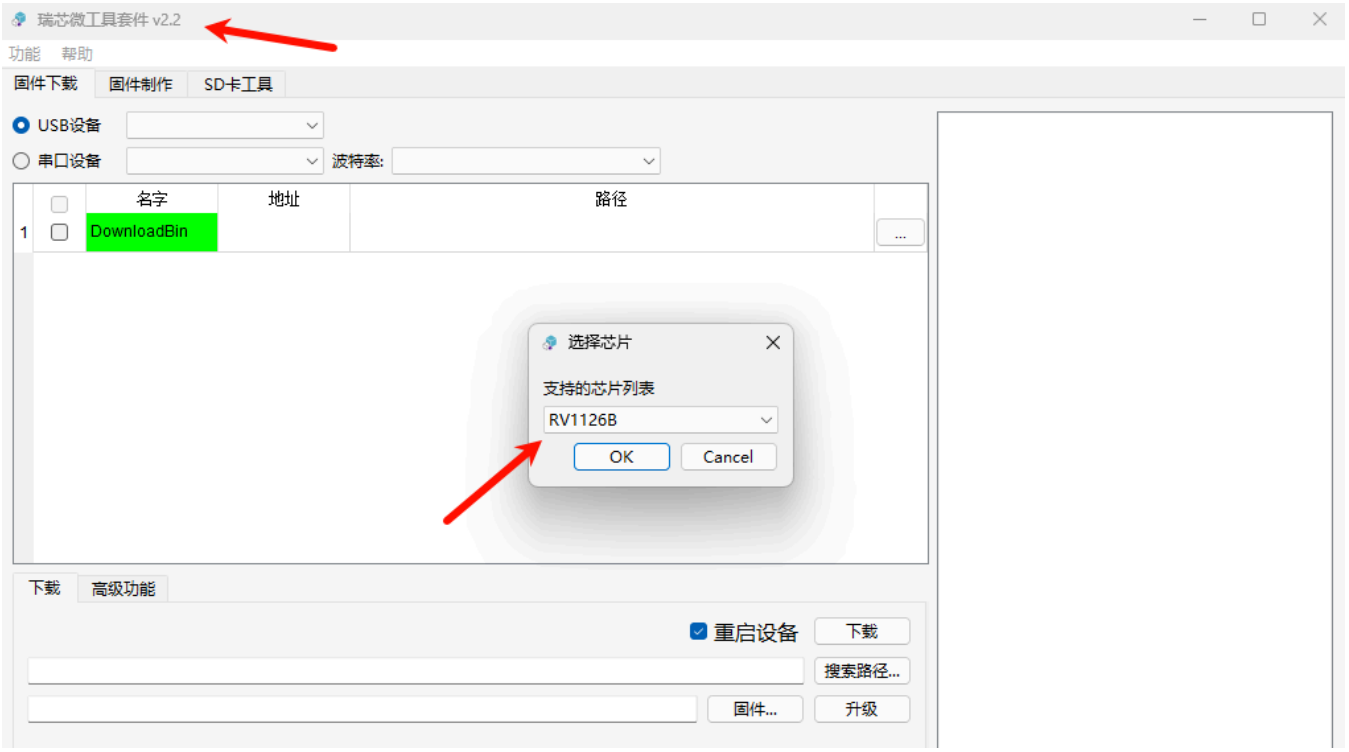
- 切换烧录模式的方法

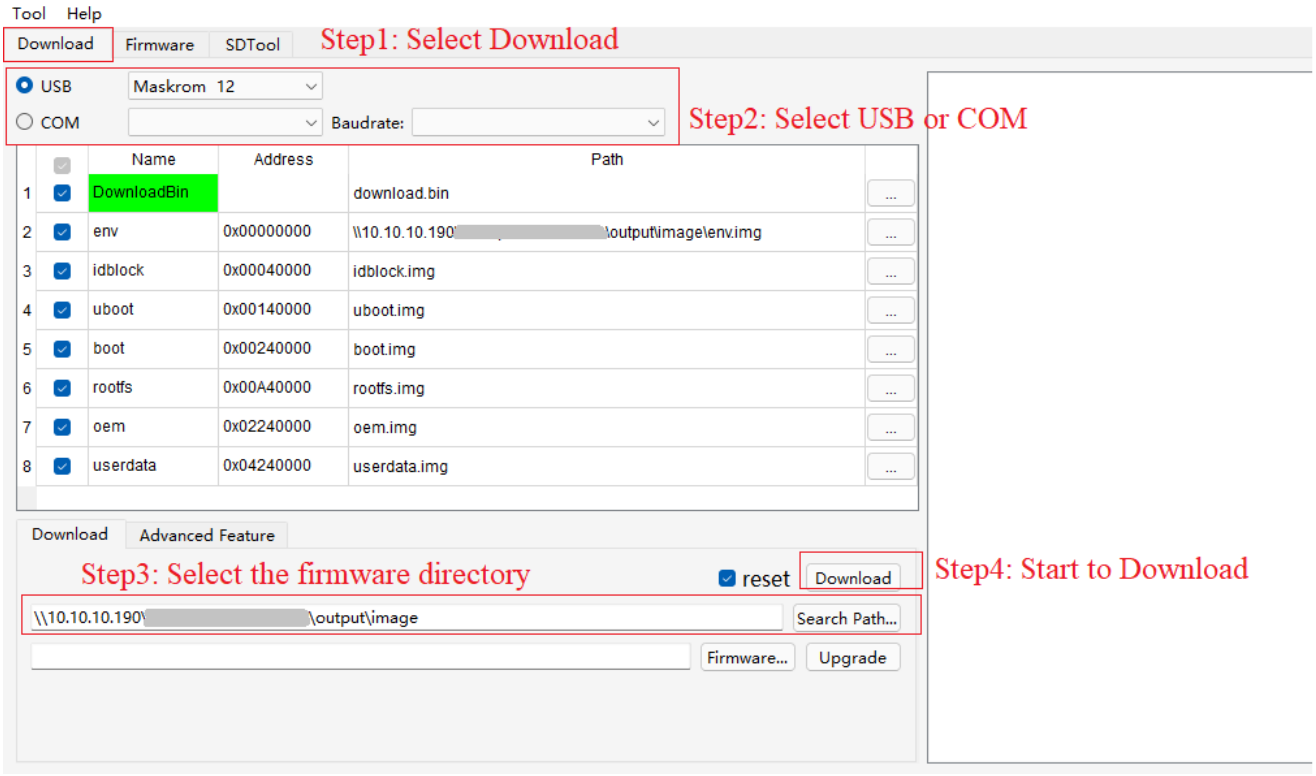
按住按键“Update”不放并按下复位键“RESET”后松手，就能进入Maskrom模式。

- SDK 提供 Windows 烧写工具，工具位于工程根目录。

  如果单板已烧录过固件，可以进U-Boot进行升级固件。详细操作请参考以下章节：
  如何在U-Boot终端下使用tftp进行分区升级
  如何在U-Boot终端下使用SD卡进行分区升级

```
<SDK>/Tools/windows/SocToolKit/SocToolKit.exe
```





7.3 update.img相关工具

7.3.1 打包

SDK在一键自动编译（`./build.sh`）时，会自动将需要烧录的固件打包成update.img，存放在`<SDK>/output/image`目录。同时，也可以运行以下命令，手动打包上述目录中的固件：

```
./build.sh updateimg
```

如需自定义固件目录等，可以手动运行打包脚本，查看帮助（-h），或输入相应选项：

```
<SDK>/tools/linux/Linux_Pack_Firmware/mk-update_pack.sh -id <RK_CHIP> -i <IMAGE_DIR>
```

7.3.2 解包

该功能需手动运行，可将`<SDK>/output/image/update.img`解包为分立固件，存放在`<SDK>/output/image/unpack`目录下。解包命令如下：

```
./build.sh unpackimg
```

如需自定义固件路径等，可以手动运行解包脚本，查看帮助（-h），或输入相应选项：

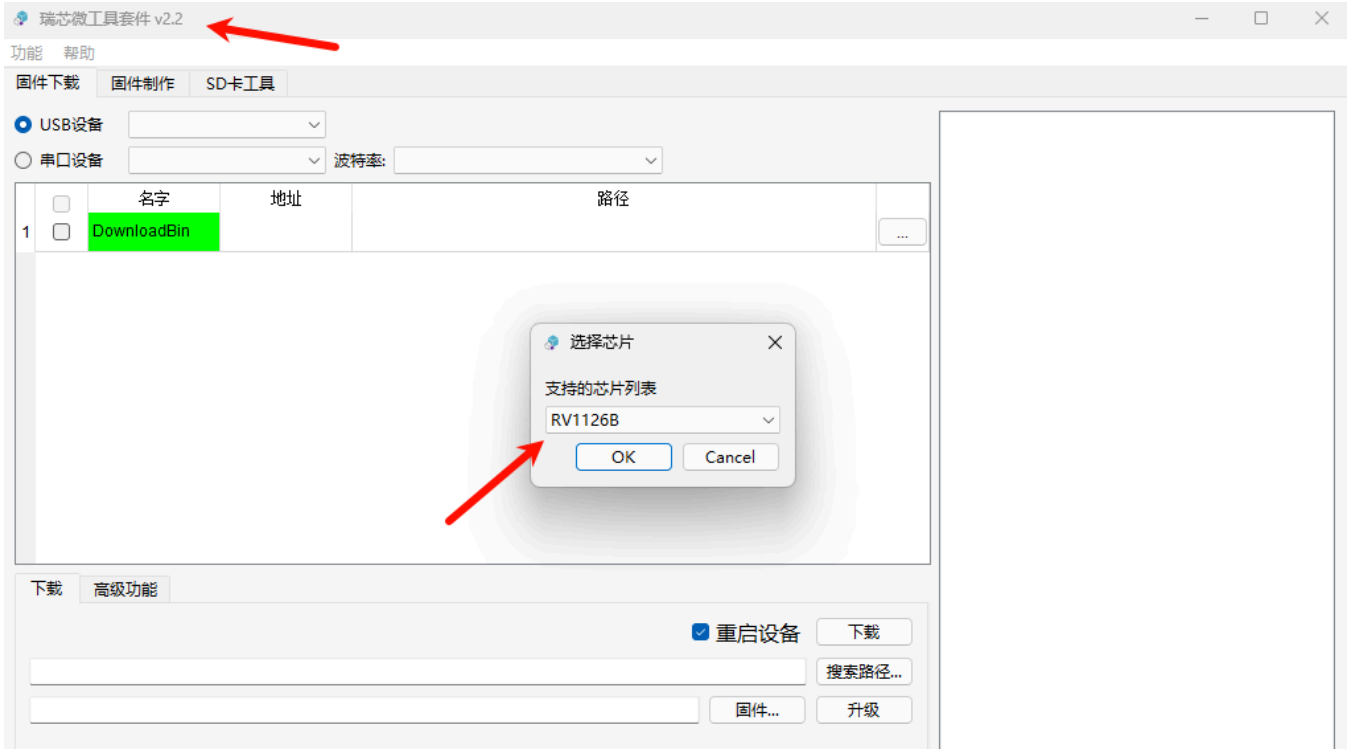
```
<SDK>/tools/linux/Linux_Pack_Firmware/mk-update_unpack.sh -i <IMAGE_PATH> -o <UNPACK_DIR>
```

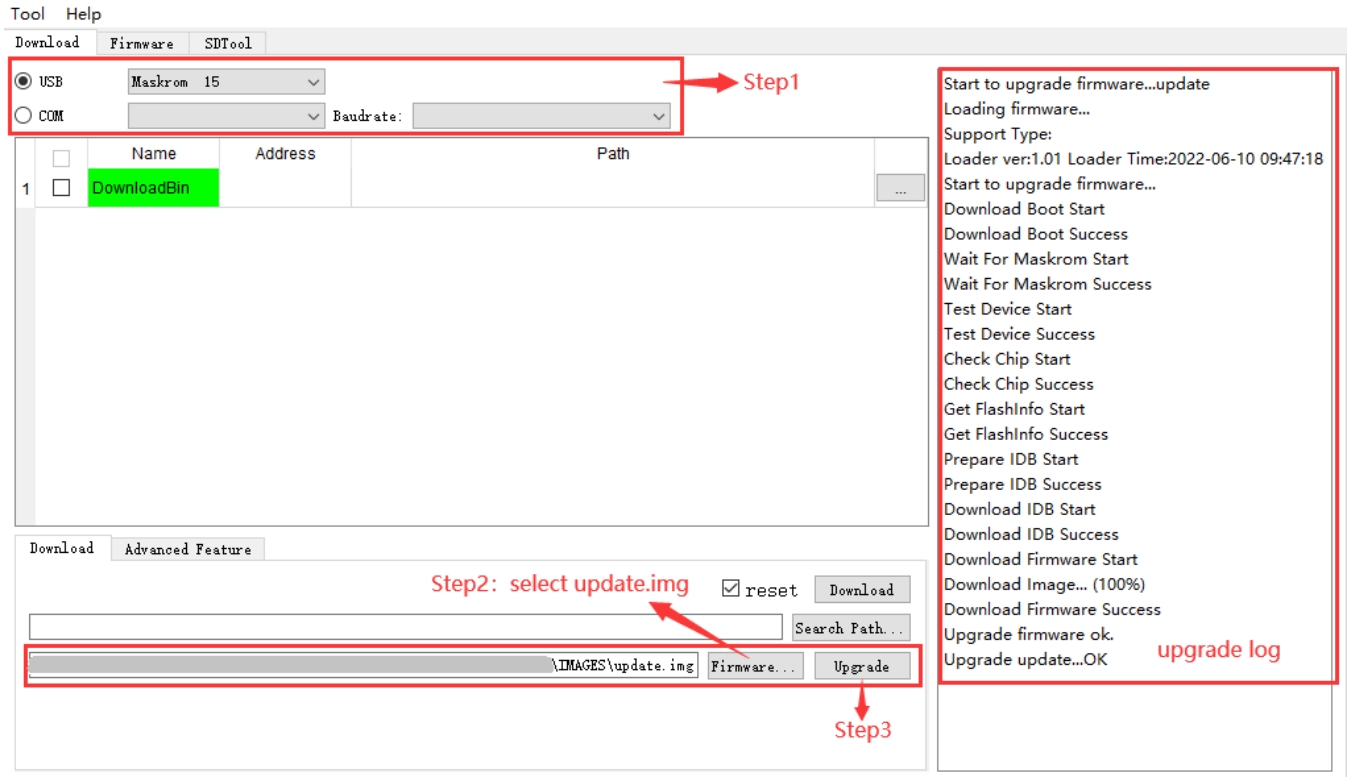
7.3.3 update.img烧写

- SDK 提供 Windows 烧写工具，工具位于工程根目录。

```
<SDK>/Tools/windows/SocToolKit/SocToolKit.exe
```

注：需要V2.2或更高版本的SocToolKit工具才能支持rv1126b的update.img烧写功能。



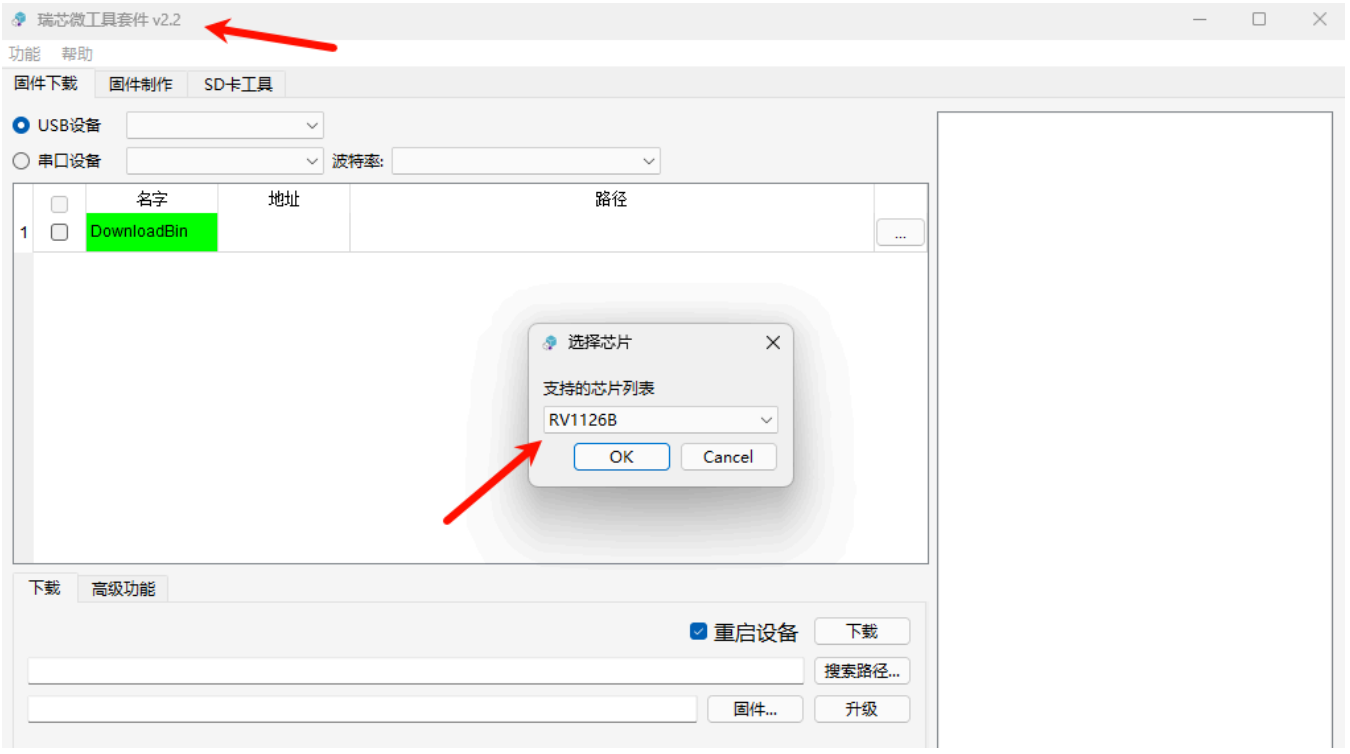


7.4 SD升级启动制作工具

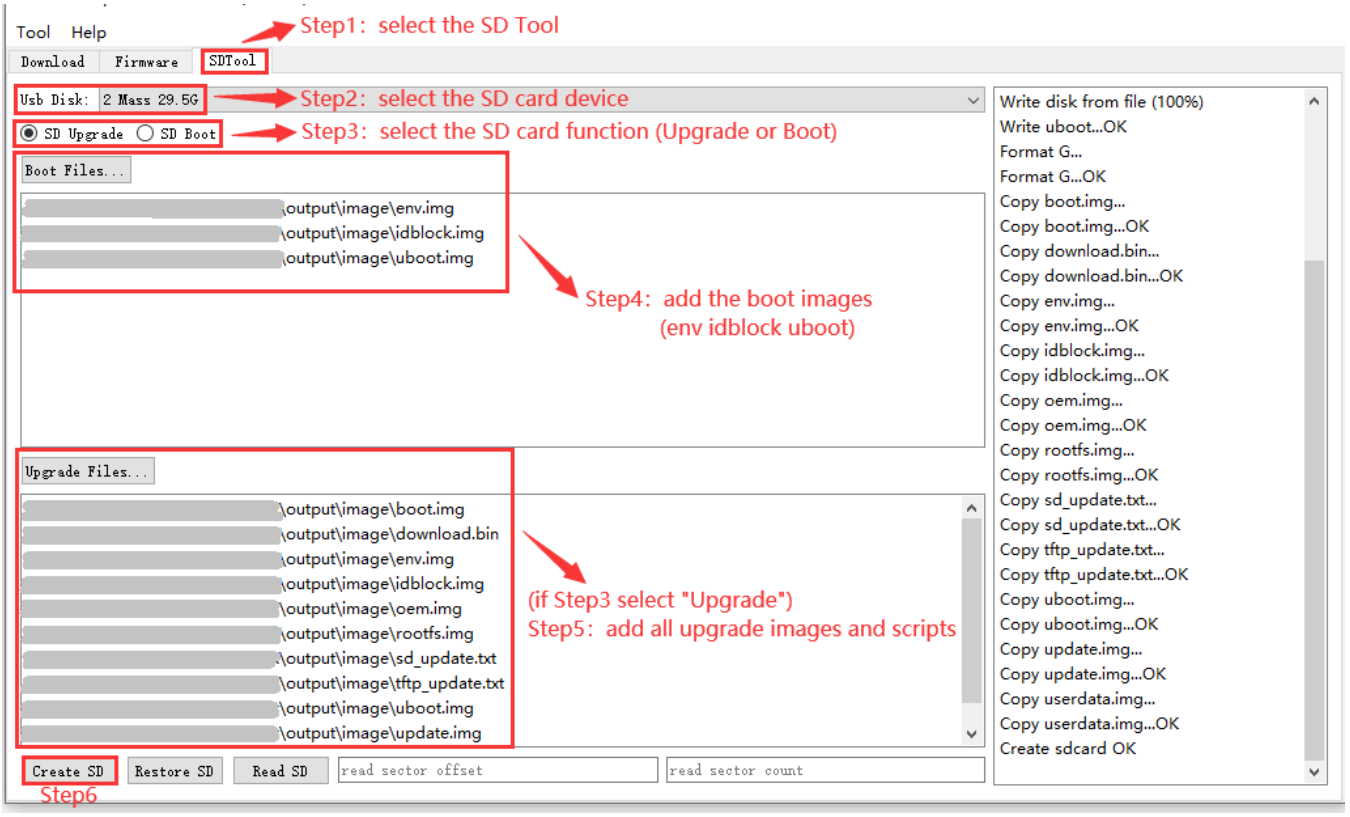
- SDK 提供 Windows SD卡升级启动制作工具，工具位于工程根目录。

注：**SD启动卡**只支持用eMMC固件。

```
<SDK>/Tools/windows/SocToolKit/SocToolKit.exe
```



注2：此功能需要用户以管理员身份运行SocToolKit.exe（开启工具时会默认询问）。



- 将制作成功的SD卡插入设备后重启，设备将优先进入SD卡中的U-Boot终端。
- 如果SD卡做为升级卡，开机会自动升级设备。（注：Uboot要使能CONFIG_ROCKCHIP_CMD="sd_update -" 和 CONFIG_CMD_SCRIPT_UPDATE=y）
- 升级完成后，需要拔掉SD卡，再重启设备，方能进入设备系统中。

7.5 工厂固件说明

制作工厂固件需要使用programmer_image_tool工具，该工具位于`<SDK>/tools/linux/SocToolKit/bin/linux`目录。输入的镜像为update.img，详细内容见update.img相关工具。

programmer_image_tool的文档在`<SDK>/docs/zh/bsp/Rockchip_Developer_Guide_Linux_Flash_Open_Source_Solution_CN.pdf`。

SDK支持编译部分flash类型的工厂固件，生成的固件存放在`<SDK>/output/image/factory`目录下。**其他flash类型的固件可以参考文档Rockchip_Developer_Guide_Linux_Nand_Flash_Open_Source_Solution_CN.pdf，使用工具programmer_image_tool自行生成**。

- 对于SPI NAND，需要配置block size和page size：

在对应的BoardConfig文件里配置如下：

```
export RK_NAND_BLOCK_SIZE=0x20000
export RK_NAND_PAGE_SIZE=2048
```

- SDK编译命令：

```
# 先编译代码
./build.sh all

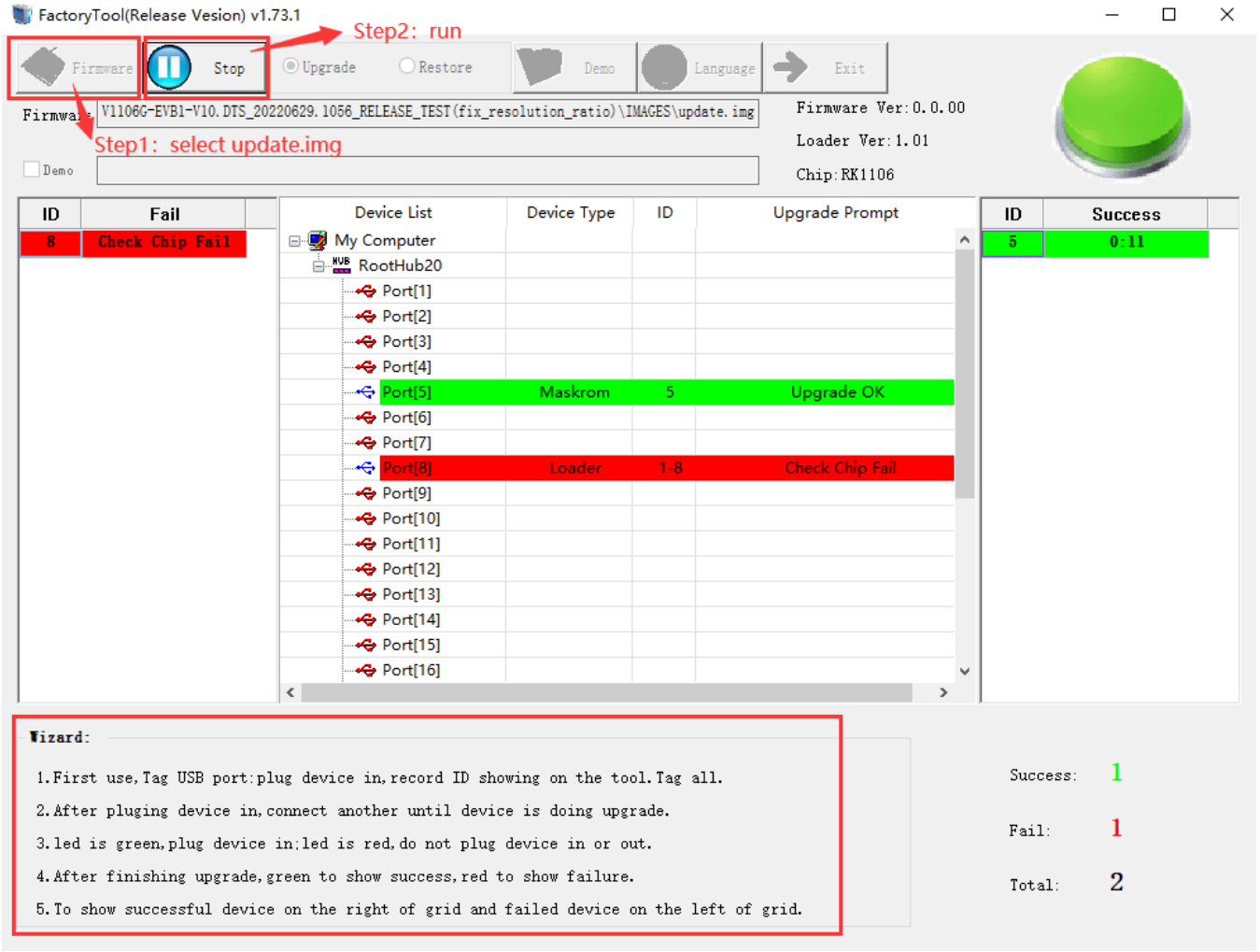
# 打包 udpate.img
./build.sh updateimg

# 生成工厂固件: output/image/factory
./build.sh factory
```

更多存储相关问题调试及处理请参考：https://redmine.rock-chips.com/documents/108

7.6 量产升级工具

量产升级需要使用FactoryTool工具，该工具位于`<SDK>/tools/windows`目录。输入的镜像为update.img，详细内容见update.img相关工具。



开始运行升级后，只要电脑和设备通过USB连接，并且设备进入Maskrom或Loader模式，就能自动开始升级，不需要其他操作。

其他内容可见工具下方的友情提醒。

8. CVR 框架介绍

8.1 功能简介

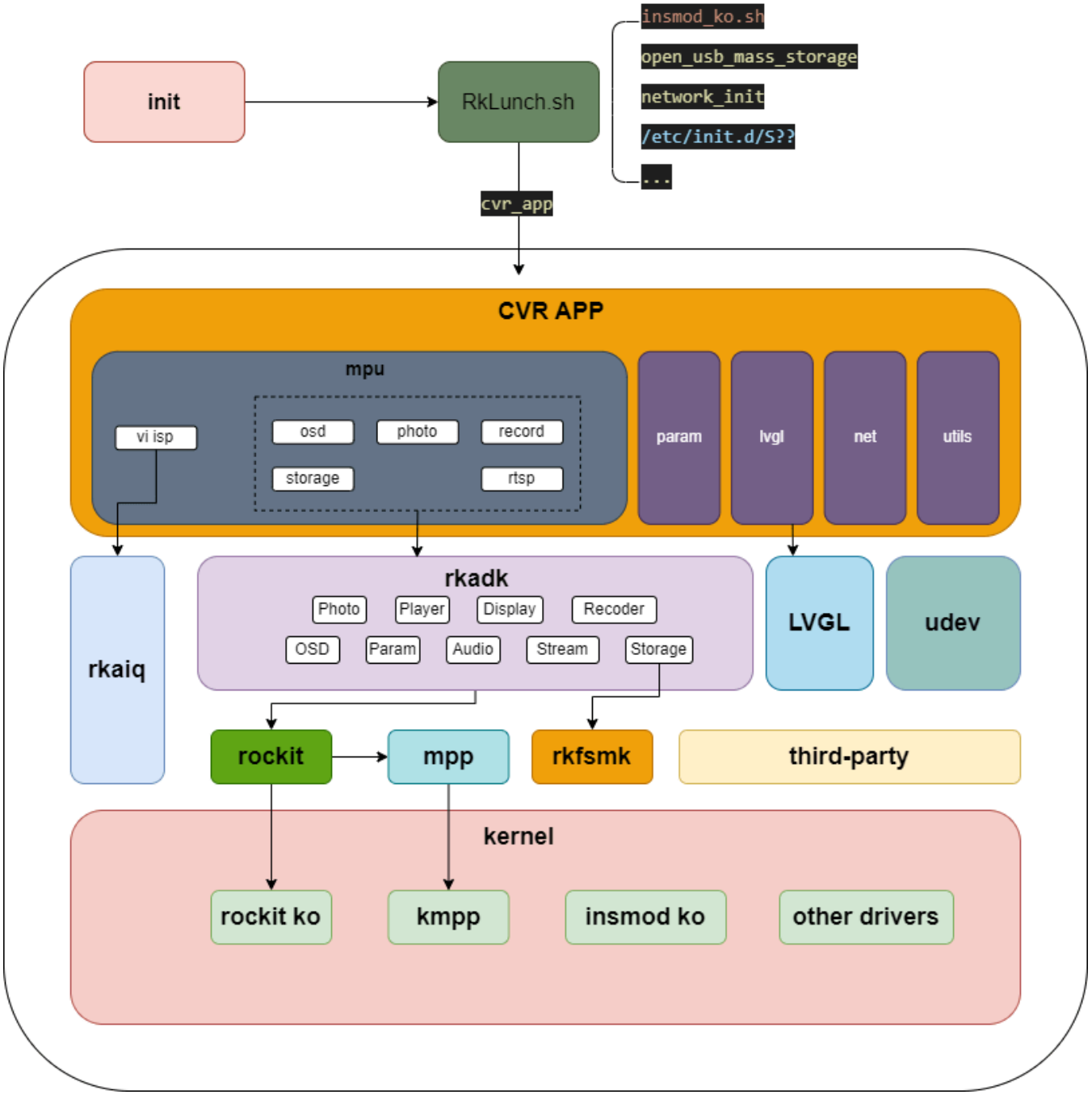
瑞芯微CVR SDK开发了以下基础功能，客户基于该SDK可以快速开发出CVR相关的产品：

- **录像相关**：循环录像、文件分割、预录像、录像回放、缩时录像、多路录像；
- **照片相关**：拍照、缩略图、照片回放；
- **播放器**：录像和音频文件播放，支持播控操作：播放、暂停、 Seek；
- **直播**：提供标准RTSP直播流的基本服务；提供RTMP直播服务。RTSP和RTMP不支持同时启动；
- **存储相关**：存储预分配、写卡优化、文件修复、格式化、媒体文件管理；
- **预览**：实时视频预览，为预览提供获取Video和Audio信息，启停VENC，启停AENC ，注册处理音视频帧数据函数的回调接口；

- **OSD**：提供基本的水印功能；
- **快速启动**：快速启动、快速抓拍、快速出流；(后续版本支持)
- **UI相关**：LVGL(轻量级和通用图形库)是一个免费和开源的图形库;
- **图像效果**：针对记录仪场景优化的图像效果；
- **算法**：支持ADAS、DMS、BSD等车载常用算法；（SDK未包含，需要第三方算法适配）
- **音频**：支持音频降噪、回声消除；

8.2 软件框架

如图所示，系统由kernel进入user层后，启动init进程。init进程会执行RkLunch.sh（位于 project/app/cvr/cvr/ 目录下），该脚本用于配置cvr_app运行环境。随后，init进程启动cvr_app，该应用程序封装了对rkaiq、rkadk、lvgl、udev等的调用，包含数据流处理和CVR业务逻辑所需的各种接口。



模块名称	功能说明
rkadk	核心框架，为APP提供功能API,文档路径： `<SDK>/project/app/component/rkadk/Rockchip_Developer_Guide_Linux_RKADK_CN.pdf`
rockit	多媒体框架API
rkaiq	ISP图像处理API
rkfsmk	存储卡格式化、文件修复
LVGL	UI框架
cvr_app	Demo APP

8.3 APP介绍

`cvr_app` 是CVR SDK自带的演示程序，专为行车记录仪产品设计，实现了常规功能。

8.4 功能模块说明

```
.
└── cvr
    ├── res
    │   ├── conf                # APP配置文件
    │   ├── font                # 字体资源
    │   ├── sound
    │   ├── string              # 多语种文本
    │   └── ui                  # icon
    ├── src
    │   ├── algorithm           # 集成算法
    │   ├── app                 # 应用处理模块：包括按键、热拔插、自动化测试处理
    │   ├── conf                # 用户参数管理
    │   ├── core                # 系统管理
    │   ├── lvgl                # UI框架：LVGL所需显示以及输入设备的驱动
    │   ├── mpu                 # 多媒体模块，基于rkadk：包含ISP 、OSD 、拍照、录像、存储、RTSP相关
应用
    │   ├── net
    │   ├── ui                  # UI源码
    │   └── utils               # 代码常用组件
    └── tools
        ├── ntp                 # NTP对时工具
        ├── remote_config       # 自动化测试工具
        ├── script_job
        └── sound</left>
```

8.5 LVGL介绍

LVGL是最流行的免费、开源的嵌入式图形库，可用于为任何MCU、MPU和显示类型相关产品创建漂亮的用户界面。`cvr_app` 的UI使用LVGL构建，SDK中提供了 LVGL源码。有关详细使用接口的文档，请参考官网 https://lvgl.io/。

8.5.1 LVGL配置文件

根据需求对LVGL进行配置，以有效减少资源占用。请复制LVGL目录旁边的 `/project/app/component/lvgl/lvgl/lv_conf_template.h`，并将其重命名为 `lv_conf_xxx.h`。打开文件并将开头的 `#if 0` 更改为 `#if 1` 以启用其内容。配置文件中的注释提供了选项的含义说明。配置完成后，请将 `project/app/component/lvgl/Makefile` 中的 `LVGL_SRC_CONF` 设置为 `lv_conf_xxx.h` 的存放路径。

```
# project/app/component/lvgl reference code
...
PKG_NAME := lvgl
PKG_BIN ?= out
PKG_BUILD ?= build
PKG_CFG := rockchip-conf  #配置文件存放目录

LVGL_DEF_CONF := $(CURRENT_DIR)/lv_conf_default.h  #默认配置文件路径
LVGL_DST_CONF := $(CURRENT_DIR)/$(PKG_NAME)/lv_conf.h

# 根据选中板级设置不同的配置文件，更改配置文件路径修改LVGL_SRC_CONF
ifneq (, $(filter RKCVR_RV1106, $(RK_APP_TYPE)))
LVGL_SRC_CONF := $(CURRENT_DIR)/$(PKG_CFG)/lv_conf_rv1106_cvr.h
else ifneq (, $(filter RKCVR_RV1103B, $(RK_APP_TYPE)))
LVGL_SRC_CONF := $(CURRENT_DIR)/$(PKG_CFG)/lv_conf_rv1103b_cvr.h
else ifneq (, $(filter FACIAL_GATE_RV1106_NORMAL, $(RK_APP_TYPE)))
LVGL_SRC_CONF := $(CURRENT_DIR)/$(PKG_CFG)/lv_conf_facialgate.h
else ifneq (, $(filter RKCVR_RV1126, $(RK_APP_TYPE)))
LVGL_SRC_CONF := $(CURRENT_DIR)/$(PKG_CFG)/lv_conf_rv1126_cvr.h
else ifneq (, $(filter RKCVR_RV1126B, $(RK_APP_TYPE)))
LVGL_SRC_CONF := $(CURRENT_DIR)/$(PKG_CFG)/lv_conf_rv1126b_cvr.h
else ifneq (, $(filter RKIPC_RV1126B_DV, $(RK_APP_TYPE)))
LVGL_SRC_CONF := $(CURRENT_DIR)/$(PKG_CFG)/lv_conf_rv1126b_dv.h
else
LVGL_SRC_CONF := $(LVGL_DEF_CONF)
endif

...
```

8.6 rkadk介绍

基于rockit的进一步封装，rkadk提供了音视频录制应用所需的基础通用组件，包括录像、拍照、播放、预览、媒体文件管理等。这一封装简化了应用开发难度，支持应用软件的快速开发。通过rkadk模块的接口，客户能够迅速开发具备录像、拍照等功能的产品。

值得注意的是，该组件仅支持单进程功能实现，除非特别说明，不支持多进程同时使用。

8.6.1 rkadk 功能列表

功能名称	功能说明
录像	1、录像任务的创建与销毁； 2、录像任务的启动与停止； 3、手动切分录像文件； 4、预录像； 5、缩时录像； 6、多路录像； 7、MP4 文件缩略图
拍照	1、单拍模式； 2、连拍模式； 3、照片缩略图；
媒体流	1、RTSP； 2、RTMP；
播放器	1、播放视频； 2、播放图片； 3、播放音频； 4、支持播放、暂停、Seek；
多媒体文件管理	1、多媒体文件扫描； 2、多媒体文件修复； 3、多媒体文件列表获取； 4、存储设备格式化，设备容量和状态查询；
水印叠加	1、录像主、子码流叠加水印； 2、拍照叠加水印； 3、远程、本地预览叠加水印； 4、RTSP、RTMP 叠加水印；
参数管理	1、设置和获取模块参数； 2、保存指定参数； 3、恢复默认参数；
本地预览	1、提供本地预览功能；

8.6.2 rkadk API说明

有关rkadk模块API接口的详细说明，请参考相关文档：

```
<SDK>/project/app/component/rkadk/Rockchip_Developer_Guide_Linux_RKADK_CN.pdf
```

9. 运动DV 介绍

9.1 功能简介

瑞芯微运动DV SDK开发了以下基础功能，客户可以基于该SDK进行运动DV相关的产品开发。

- **视频相关**：普通录像、慢动作摄影、延时摄影、快速连拍
- **播放器**：录像和音频文件播放，支持播控操作：播放、暂停、 Seek
- **直播**：提供标准RTSP直播流的基本服务
- **存储相关**：存储预分配、写卡优化、文件修复、格式化、媒体文件管理
- **预览**：实时视频预览
- **快速启动**：支持基于休眠唤醒的快速应用启动
- **UI相关**：LVGL(轻量级和通用图形库)是一个免费和开源的图形库
- **图像效果**：支持普通防抖、地平线防抖、畸变矫正、AMTD(Auto Motion Trajectory Detection)等图像算法
- **音频**：支持AI音频降噪

9.2 使用说明

9.2.1 硬件需求

- 开发板：RV1126B EVB1 V10
- IMU SENSOR：ICM42670
- CAMERA：IMX586，GC8613，SC850SL

9.2.2 编译配置

在SDK根目录输入命令：

```
./build.sh lunch
```

选择配置`BoardConfig-EMMC-RK801-RV1126B_EVB1_V10-DV.mk`，再输入命令一键编译：

```
./build.sh
```

9.2.3 应用功能

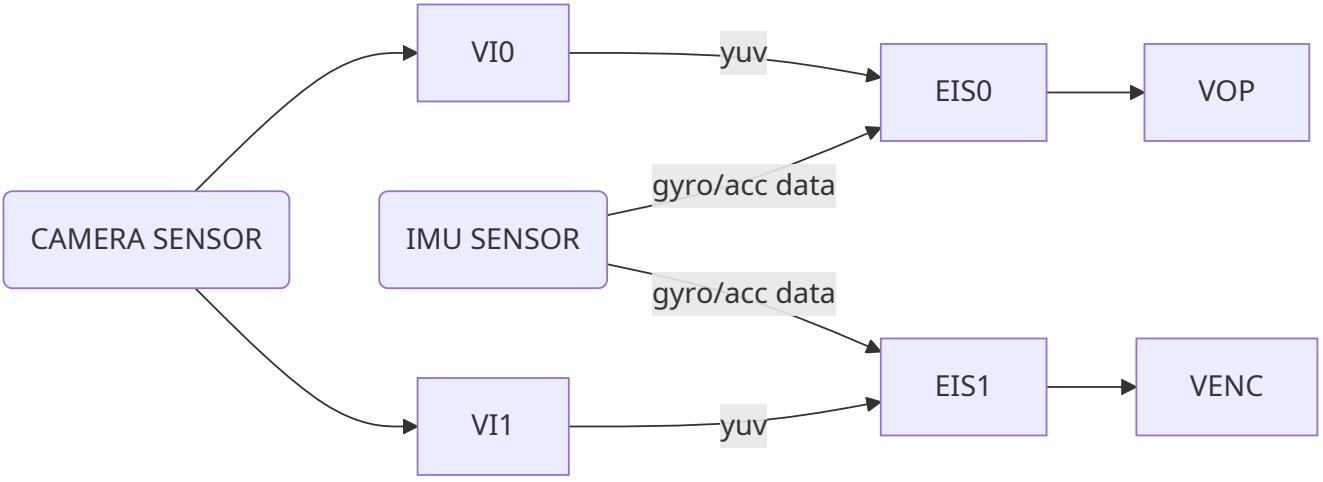
系统启动后，会运行脚本`/oem/usr/bin/RkLunch.sh`来进行必要的环境配置，然后会自动运行`rkipc`应用。应用的基本功能如下：

- 视频录制/拍照：点击屏幕正下方的拍照键，进行录制、拍照
- 模式切换：点击屏幕右下方的设置按键进入设置界面，再点击模式切换选择对应模式
- 防抖模式设置：点击屏幕右下方的设置按键进入设置界面，再点击视频设置->EIS模式来切换防抖设置
- 录像/照片回放：点击屏幕左下角进入相册，可选择视频或者图片进行查看
- 格式化：点击屏幕右下方的设置按键进入设置界面，再点击格式化按键进行格式化操作

9.3 代码仓库

运动DV的参考应用代码位于`project/app/rkipc/rkipc/src/rv1126b_dv`，具体信息请参考rkipc目录下的文档

9.4 基本数据流



运动DV应用使用两条数据流，数据流0为预览流，EIS算法不使用缓存帧来保证实时性。数据流1为录像流，EIS算法会使用缓存帧来实现更好的防抖效果。客户可参考该数据流定制自己需要的应用。

10. EIS防抖说明

电子防抖，又称电子稳像（EIS，Electronic Image Stabilization），其本质是根据运动偏移量对图像进行透视变换。

RKEIS的基本原理是计算出当前图像的实际运动偏移量，然后经过平滑处理得到期望的运动偏移量，补偿两者的差异，从而得到防抖的效果，在补偿的过程中需要同时考虑到卷帘快门效应的影响。

运动偏移量可以是图像平移，也可以是图像旋转。

函数接口请参考Rockchip_Developer_Guide_MPI_C_CN.pdf里的GDC部分。

EIS陀螺仪防抖的基本原理、数据要求以及标定调试时的各种注意事项，请参考Rockchip_Developer_Guide_EIS_CN.pdf。

10.1 算法配置参数

以下为EIS算法配置参数的基本说明，更多信息请参考Rockchip_Developer_Guide_EIS_CN.pdf。

```
{
    "cfgVersion": "1.0.4",
    "initParams": {
        "still_crop": 0, \\ 0: 使用EIS算法; 1: 不使用EIS算法，保持裁剪放大;  2: 不使用EIS算法，原图输
出
        "src_type": 4, \\ 输入相机类型
        "dst_type": 4, \\ 输出相机类型
        "src_width": 3840, \\ 输入图像宽度
        "src_height": 2160, \\ 输入图像高度
        "src_stride": 3842, \\ 输入图像行距
        "src_height_stride": 2160, \\ 输入图像列距
        "src_format": 1, \\ 输入图像压缩格式，0表示非压缩格式YUV420，1表示TILE0，2表示RKFBC
        "dst_width": 3840, \\ 输出图像宽度
        "dst_height": 2160, \\ 输出图像高度
        "dst_stride": 3840, \\ 输出图像行距
        "dst_height_stride": 2160, \\ 输出图像列距
        "dst_format": 0, \\ 输出图像压缩格式，0表示非压缩格式YUV420，1表示TILE0，2表示RKFBC
        "clip_width": 3072, \\ 输入图像裁剪宽度
        "clip_height": 1728, \\ 输入图像裁剪高度
        "auto_scaling": 1,
        "delay_num": 7, \\ 后向帧帧数
        "ois_mode": 0,
        "dvs_mode": 1, \\ 防抖模式，0: 不使用后向帧; 1: 使用后向帧
        "horizon_mode": 0, \\ 地平线防抖模式开关，0表示不启用，1表示启用
        "use_async": 1,
        "view_angle": 0.0,
        "frame_rate": 30.0, \\ 图像帧率
        "sensor_axes_type": 6, \\ 标定参数，请参考调试文档
        "motion_detect_mode": 0,
        "num_cores": 1,
        "affinity_mask": 6, \\ 算法线程的CPU亲和度
        "imu_rate": 800.0, \\ IMU帧率
        "remap_hw": 1, \\ 重映射硬件模式，0表示使用GPU,1表示使用FEC
        "x_grid_step": 32, \\ 水平步长
        "y_grid_step": 16, \\ 垂直步长
        "follow_mode_params" :{
            "th1": 3.5,
            "th2": 5.5,
            "th3": 8.0
        },
        "opencl_lib_path": "/usr/lib/libOpenCL.so", // opencl库地址，仅在remap_hw=0时有效
        "opencl_program_path": "/data/rkEIS_CLProgram_Bifrost-G52-g13p0-01eac0.bin" //
opencl程序地址，仅在remap_hw=0时有效
    },
    "estParams": {
        "view_angle": 0.0,
        "src_focal_length_x": 2.11070312e+03, \\ 标定参数，请参考调试文档
        "src_focal_length_y": 2.10308130e+03, \\ 标定参数，请参考调试文档
```

```
        "src_center_x": 1.88957458e+03, \\ 标定参数，请参考调试文档
        "src_center_y": 1.14424390e+03, \\ 标定参数，请参考调试文档
        "dst_focal_length_x": 2.11070312e+03, \\ 标定参数，请参考调试文档
        "dst_focal_length_y": 2.10308130e+03, \\ 标定参数，请参考调试文档
        "dst_center_x": 1.88957458e+03, \\ 标定参数，请参考调试文档
        "dst_center_y": 1.14424390e+03, \\ 标定参数，请参考调试文档
        "f_len_unit": 1.0,
        "force_use_json_rs": 0, \\ 是否强制使用外部提供的rs_skew,0表示不使用，1表示使用
        "rs_skew": 32472000, \\ 外部提供的卷帘快门时间，仅在force_use_json_rs=1时生效
        "start_of_scanline": 0,
        "image_timestamp_offset": 0, \\ 图像时间偏移量，单位：ns
        "gyro_timestamp_offset": -32083000, \\ imu时间偏移量量，单位：ns
        "hall_timestamp_offset": 0,
        "src_ldc": {
            "k1": -5.94682135e-02, \\ 标定参数，请参考调试文档
            "k2": -2.41034608e-02, \\ 标定参数，请参考调试文档
            "p1": 1.47078466e-02, \\ 标定参数，请参考调试文档
            "p2": -6.27873745e-03, \\ 标定参数，请参考调试文档
            "k3": 0.0, \\ 标定参数，请参考调试文档
            "xi": 0.0 \\ 标定参数，请参考调试文档
        },
        "dst_ldc": {
            "k1": -5.94682135e-02, \\ 标定参数，请参考调试文档
            "k2": -2.41034608e-02, \\ 标定参数，请参考调试文档
            "p1": 1.47078466e-02, \\ 标定参数，请参考调试文档
            "p2": -6.27873745e-03, \\ 标定参数，请参考调试文档
            "k3": 0.0, \\ 标定参数，请参考调试文档
            "xi": 0.0 \\ 标定参数，请参考调试文档
        },
        "gyro_bias_x": 0.0, \\预设陀螺仪x轴偏置
        "gyro_bias_y": 0.0, \\预设陀螺仪y轴偏置
        "gyro_bias_z": 0.0, \\预设陀螺仪z轴偏置
        "acce_bias_x": 0.0, \\预设加速度计x轴偏置
        "acce_bias_y": 0.0, \\预设加速度计y轴偏置
        "acce_bias_z": 0.0  \\预设加速度计z轴偏置
    },
    "rtParams": {
        "frame_rate": 0.0,
        "zoom_ratio": 0.0,
        "bypass_zoom_thd": 0.0,
        "fix_level": 0,
        "moving_subject_thd": 0,
        "xno_movement_thdi": 0
    },
    "calibParams": {
        "calib_mode": 3, \\ 温漂标定模式
        "imu_rate": 800.0, \\ imu帧率
        "affinity_mask": 6,
        "gyro_static_calib_value_path": "/data/IMUCalibratorFile/gyro_static_calib_val.txt",
\\ 陀螺仪温漂静态标定值路径
        "accel_calib_value_path": "/data/IMUCalibratorFile/accel_calib_val.txt", \\ 加速度计温
漂标定值路径
```

```
        "gyro_dynamic_calib_value_path":
"/data/IMUCalibratorFile/gyro_dynamic_calib_val.txt", \\ 陀螺仪温漂动态标定值路径
        "use_init_temp_coef": 1, \\ 是否使用初始化温漂标定参数
        "init_temp_coef": {
            "x_a0": 0.0,
            "x_a1": 0.0,
            "x_a2": 0.0,
            "y_a0": 0.0,
            "y_a1": 0.0,
            "y_a2": 0.0,
            "z_a0": 0.0,
            "z_a1": 0.0,
            "z_a2": 0.0
        }
    }
}
```

10.2 驱动调试说明

EIS算法对于IMU数据的时间戳、YUV帧的时间戳稳定性有较高要求。基于Linux系统，RK提供了`VIRTUAL GPIO`来减少中断例程的处理延迟。同时通过`cpu isolate`等方式来减少调度导致的中断时间戳不稳定等问题。参考DTS配置：

```
/ {
    model = "Rockchip RV1126B EVB1 V10 DV Board";
    compatible = "rockchip,rv1126b-evb1-v10-dv", "rockchip,rv1126b";
    chosen {
        bootargs = "earlycon=uart8250,mmio32,0x20810000 console=ttyFIQ0 rw
root=PARTUUID=614e0000-0000 rootfstype=ext4 rootwait
snd_soc_core.prealloc_buffer_size_kbytes=16 coherent_pool=32K isolcpus=3 nohz_full=3";
    };
};

// Bind the interrupts of the IMU to CPU3, and set isolcpus=3 at the same time to isolate
CPU3 for use by the IMU to avoid scheduling issues
&gpio4 {
    interrupt-affinity = <&cpu0>, <&cpu0>, <&cpu0>, <&cpu3>;
    interrupt-pins = <0>,
             <0>,
             <0>,
             <RK_PIN_TO_BIT(RK_PA3)>;
};

&spi0 {
    status = "okay";
    max-freq = <20000000>;
    pinctrl-names = "default";
    pinctrl-0 = <&spi0m2_clk_pins &spi0m2_csn0_pins &inv_int1>;
    icm42670: icm42670@0 {
        compatible = "invensense,icm42670";
        reg = <0x0>;
        spi-max-frequency = <20000000>;
```

```
        spi-cpha;
        spi-cpol;
        //vdd-supply = <&vcc_3v3_s0>;
        int1-gpio = <&gpio4 RK_PA3 GPIO_ACTIVE_HIGH>;
        interrupt-parent = <&gpio4>;
        interrupts = <3 IRQ_TYPE_EDGE_FALLING>;
        status = "okay";
    };
};
```

应用启动时，需要做对应的配置，将IMU、VICAP的中断处理绑定到CPU3，可参考`RkLunch.sh`脚本，关键代码如下：

```
# Define a function to get the interrupt number and bind it to CPU3
bind_interrupt_to_cpu3() {
    local interrupt_line=$1
    local interrupt_number=$(echo $interrupt_line | awk '{print $1}' | sed 's/:$//')
    local cpu_mask=$((8)) # CPU3 bitmask, assuming CPU numbers start from 0

    # Check if the interrupt number is valid
    if [[ -n $interrupt_number ]]; then
        # Check if the interrupt binding file exists
        if [ -f "/proc/irq/$interrupt_number/smp_affinity" ]; then
            # Write the CPU3 bitmask to the interrupt binding file
            echo $cpu_mask > "/proc/irq/$interrupt_number/smp_affinity"
            echo "Interrupt number $interrupt_number has been bound to CPU3"
        else
            echo "Interrupt number $interrupt_number binding file does not exist"
        fi
    else
        echo "Invalid interrupt number: $interrupt_number"
    fi
}

killall irqbalance
# Get interrupt numbers and bind them to CPU3, Cannot bind ISP interrupt, otherwise imu pts
will fluctuate greatly
interrupt_lines=$(cat /proc/interrupts | grep -E 'rkcifhw|spi|inv_mpu')
echo "$interrupt_lines" | while read -r line; do
    bind_interrupt_to_cpu3 "$line"
done
# disable cpu idle
echo 1 > /sys/devices/system/cpu/cpu3/cpuidle/state1/disable
```

客户自行调试IMU驱动时，请注意在IMU的中断上半部**先获取时间戳**，避免其他行为带来的时间戳抖动。此外需要通过`gpiod_to_irq`来获取中断号，这样在休眠唤醒后，virtual gpio的配置可以恢复正常。算法对于IMU的频率和量程有一定要求，一般频率建议不小于500hz，量程则是越大越好。参考脚本：

```
echo 800 > /sys/bus/iio/devices/iio:device1/in_anglvel_sampling_frequency
echo 800 > /sys/bus/iio/devices/iio:device1/in_accel_sampling_frequency
echo 0.004785 > /sys/bus/iio/devices/iio:device1/in_accel_scale
echo 0.001064225 > /sys/bus/iio/devices/iio:device1/in_anglvel_scale
echo 121 > /sys/bus/iio/devices/iio:device1/in_accel_lpf_bw
echo 0 > /sys/bus/iio/devices/iio:device1/in_anglvel_lpf_bw
```

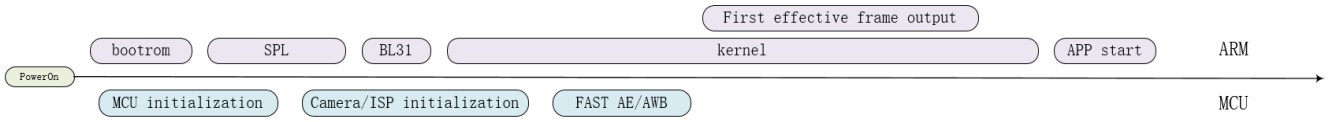
11. 电池快启IPC介绍

11.1 快启流程说明

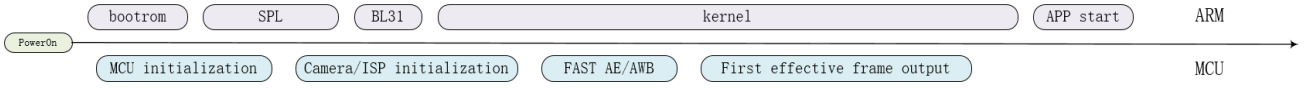
11.1.1 快启流程基本框架

芯片内置一个高性能的MCU，在启动阶段高性能MCU可以协助ARM初始化Camera和ISP，进而达到快速抓取有效第一帧的目的。

快启基本流程如图所示：



或



快速启动的过程中，Uboot会被跳过，直接由SPL加载内核，如果客户打开Uboot功能，会影响启动速度。

11.1.2 快启固件配置及编译

快启相关BoardConfig目录为：project/cfg/BoardConfig_BatteryIPC/，自定义配置固件可参考此目录。这里以BoardConfig-SPI_NOR-NONE-RV1126B_EVB2_V12-BAT_IPC.mk为例，介绍RV1126B+SC450AI使能快启相关配置。

1. 指定RTT MCU固件

```
# Uboot Loader ini overlay
# build.sh mcu info --> rv1126b_evb2-SC450AI-ADC
export RK_UBOOT_RKBIN_MCU_CFG=rv1126b_evb2-SC450AI-ADC
```

2. 指定sensor iqbin文件

```
# Config sensor IQ files
# RK_CAMERA_SENSOR_IQFILES format:
#     "iqfile1 iqfile2 iqfile3 ..."
# ./build.sh media and copy <SDK root
dir>/output/out/media_out/isp_iqfiles/$RK_CAMERA_SENSOR_IQFILES
export RK_CAMERA_SENSOR_IQFILES="common/sc450ai_default_default.bin"
```

3. 指定快启应用

```
# app config
export RK_APP_TYPE="RK_FASTBOOT_SINGLE_PROCESS"
#export RK_APP_TYPE="RK_FASTBOOT_DUAL_PROCESS"
```

4. 快启使能

```
# enable fastboot
export RK_ENABLE_FASTBOOT=y
```

5. meta参数配置，用于配置rtt及内核状态下camera工作参数

```
export RK_META_PARAM=" \
 --meta_load_addr=0x41240000 \
 --rk_cam_mirror_flip=0 \
 --rk_cam_fps=15 \
 --rk_cam1_max_fps=60 \
 --rk_cam_hdr=0 \
 --rk_led_value=20 \
 --rk_night_mode=2 \
 --rk_rtt_stream_on_threshold=15 \
 --rk_rtt_stream_buf_cnt=2 \
"
```

11.1.3 快启应用说明

11.1.3.1 双摄快启demo说明

双摄快启是单进程。

代码路径：project/app/fastboot_client/fastboot_dual_camera_demo.c

安装脚本：project/cfg/BoardConfig_BatteryIPC/tb-dual-cam-spi_nor-post.sh

应用启动放在/etc/init.d/rcS

11.1.3.2 单摄快启demo说明

单摄快启支持单进程或双进程应用。客户APP进程文件大小比较大的情况下，建议双进程，fastboot_server可以协助快速初始化VI和VENC通道，进而快速出流。

单进程代码路径：project/app/fastboot_client/fastboot_demo.c

双进程server端代码路径：project/app/component/fastboot_server/fastboot_server.c
双进程client端代码路径：project/app/fastboot_client/fastboot_client.c

在BoradConfig通过RK_APP_TYPE指定应用类型，RK_FASTBOOT_SINGLE_PROCESS为单进程，RK_FASTBOOT_DUAL_PROCESS为双进程。

应用启动同样放在/etc/init.d/rcS

11.1.3.3 预览画面

1. 使用拉流软件预览码流画面

```
# rndis.sh
serialnumber is  0000000000000000
config usb0 IP..

# ifconfig
usb0      Link encap:Ethernet  HWaddr 4A:FC:84:D6:A4:1A
          inet addr:192.168.1.100  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:80 errors:0 dropped:63 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:12637 (12.3 KiB)  TX bytes:0 (0.0 B)
```

拉流软件配置地址
rtsp://192.168.1.100/live/0

2. 查看前30帧码流

sdk自带快启应用默认保存前30帧码流数据，存放在/tmp/venc0.bin下，可通过tftp或是sz拿到码流数据

```
tftp -pl /tmp/venc0.bin 192.168.1.101
```

或者

```
sz /tmp/venc0.bin
```

3. 使用MIPI屏幕预览画面

11.1.4 快速启动日志说明

快启MCU的日志需要开机运行`make_meta --rtt-log`命令进行查看。以下是机器开机过程部分log说明

```
########################## DDR 版本信息 ##########################
DDR c4d8224017 wesley.yao 25/06/13-15:11:04,fwver: v1.03
...

########################## MCU 版本信息 ##########################
version info: 2025-06-19 14:33:08.702659 df3b8c1eb7(rtt) 356c4481(hal)
...

########################## idblock （即SPL）的版本信息 ##########################
U-Boot 2017.09-gd7e4fd2296a-250612 #syf (Jun 19 2025 - 14:13:26 +0800)
...

########################## idblock 总共耗时时间 ##########################
Total: 173.190/217.223 ms
...

########################## MCU fastAE 日志 ##########################
[STREAM]: L 0, 1, 0, 0x869000, 783360, 0x22e, 0x1ac40, tick:37
[AELIB]:
1,luma=245.60,setP=28.00,curE=79.90051,newE=14.47106,T=0.00827,G=1713.000,ispG=1.021,regT=55
8,regG=109632,dcg=0
[STREAM]: L 1, 2, 0, 0x929000, 783360, 0x22e, 0x1ac40, tick:45
[AELIB]:
2,luma=245.60,setP=28.00,curE=79.90051,newE=2.27621,T=0.00827,G=275.228,ispG=1.000,regT=558,
regG=17615,dcg=0
[STREAM]: L 2, 3, 0, 0x9e9000, 783360, 0x22e, 0x1ac40, tick:53
[AELIB]:
3,luma=239.00,setP=28.00,curE=56.62541,newE=4.42213,T=0.00827,G=534.702,ispG=1.000,regT=558,
regG=34221,dcg=0

########################## MCU fastAE 收敛日志 ##########################
===========fastae is match===========
...

########################## MCU 系统退出的日志，如下log说明在244ms退出 ##########################
[STREAM]: will go to wfi, tick:244!
```

11.1.5 快启各个阶段耗时统计介绍

快启各个阶段耗时保存于/tmp/thunderboot_time.txt文件, 以下具体说明。

注意：下文 HPMCU RTT log中的tick代表的时间和ARM不是同一个时间轴，RTT的tick需要加上硬件上电时间，而ARM不需要。

1. RTT 应用下序列及camera stream on时间统计(并非SOC收到数据流的时间)。

```
[thunderboot_time] rtt app enter tick: 1(ms)
[thunderboot_time] set camera initial regs enter tick: 1(ms)
[thunderboot_time] set camera initial regs exit tick: 12(ms)
[thunderboot_time] camera stream on enter tick: 30(ms)
[thunderboot_time] camera stream on exit tick: 31(ms)
```

2. RTT AE收敛时间统计。

```
[thunderboot_time] fastae 12 is match tick: 131(ms)
```

3. RTT 切到大图序列时间。

```
[thunderboot_time] camera switch resolution enter tick: 131(ms)
[thunderboot_time] camera switch resolution exit tick: 145(ms)
```

4. init进程开始启动。

```
[    0.290646] Run /sbin/init as init process
```

5. app首行。

```
[    0.322967] [app]: [thunderboot_time] fastboot_demo enter
```

6. 编码器开始编码。

```
[    0.477346] [thunderboot_time] venc run task begin.
```

7. 编码器完成编码。

```
[    0.499243] [thunderboot_time] venc done.
```

8. 应用取到所有离线帧编码数据。

```
[    0.598923] [app]: [thunderboot_time] get venc all reserved frames
```

11.2 光敏使用说明

11.2.1 光敏类型

光敏方案	说明
软光敏	即无光敏方案，通过获取 ISP 亮度使用软件计算。
模拟光敏	SDK默认方案，不同摄像头Sensor需要配置不同的MCU固件。 具体参考如何修改固件支持的摄像头

光敏方案	说明
数字光敏	需要重新编译MCU固件，SDK调试过LTR308

光敏类型包括有软光敏和硬光敏，其中硬光敏又分为模拟光敏和数字光敏。软光敏是通过获取ISP的亮度统计值，根据AE标定参数用软件计算出环境亮度。模拟光敏是通过读取硬件adc值，根据标定好的adc值计算出环境亮度，测量速度较快。数字光敏等效于一个sensor，无需标定可直接输出环境亮度值，但需要一定的曝光积分时间，测量的值域范围及精度与曝光增益和积分时间有关，数字光敏的测量值域范围和精度高于模拟光敏。

11.2.2 光敏参数

详细的meta分区说明请参考meta分区说明。

meta分区中与光敏相关的调试参数主要有光敏类型als_type，日夜切换阈值als_value，以及标定参数ae_init_cfg和awb_init_cfg，具体的配置可参考SDK里`project/make_meta/sensor_ae_awb_table/XX_2a.c`。

根据光敏类型，可将meta分区中XXsensor_init.c的als_type配置成ALS_TYPE_NONE（0：软光敏）、ALS_TYPE_ANALOG（1：模拟光敏）、ALS_TYPE_DIGITAL（2：数字光敏）。als_value表示环境照度阈值（单位：lux），环境照度低于该值时进入夜视模式。值得注意的是，软光敏中als_value不再是原来的环境照度阈值，而且软光敏根据物体反光来进行环境亮度判断的阈值，默认值为0。光敏参数配置完成后，在RTT阶段会打印`>>> als type: 0, als vlaue: xx, night mode: xx`的log作为提示，其中als type（0：去光敏，1：模拟光敏，2：数字光敏）。

下面针对光敏的标定参数作详细的介绍，ae_init_cfg和awb_init_cfg的结构体定义参见SDK里`project/make_meta/include/sensor_2a_common.h`。

```
#define ADC_CALIB_NO          4
#define ADC_CALIB_TBL_LEN     15
#define AE_START_EXP_NO       3
#define AE_RESERVE_NO         4
#define AWB_RESERVE_NO        60
#define FIX2INT16(x)          (int)(x * (1 << 16))
#define AE_GRID_STATS         113

struct ae_init_cfg {
    /* 黑电平值，12bit
     * 用于mcu阶段ae统计计算，该值可从json中获取
     */
    uint32_t black_lvl;
    /*
     * 模拟光敏的adc值标定类型
     *    0: 公式法
     *    1: 查表法
     */
    uint32_t adc_calib_type;
    /*
     * 模拟光敏的adc值量程类型
     *    0: 双量程
     *    1: 单量程
     */
    uint32_t adc_range_type;
    /*
```

```
     * 模拟光敏adc公式法标定参数，共4个标定参数，均为10bits数据
     * 对于双量程类型：
     *    adc_calib[0]为低量程环境亮度=0lux时，对应的adc值
     *    adc_calib[1]为低量程环境亮度=100lux时，对应的adc值
     *    adc_calib[2]为高量程环境亮度=0lux时，对应的adc值
     *    adc_calib[3]为高量程环境亮度=1000lux时，对应的adc值
     * 对于单量程类型：
     *    adc_calib[0]为环境亮度=0lux时，对应的adc值
     *    adc_calib[1]为环境亮度=100lux时，对应的adc值
     *    adc_calib[2]为环境亮度=0lux时，对应的adc值，即adc_calib[2]=adc_calib[0]
     *    adc_calib[3]为环境亮度=1000lux时，对应的adc值
     */
    uint32_t adc_calib[ADC_CALIB_NO];
    /*
      * 模拟光敏adc查表法标定参数个数，最大值为15
      */
    uint32_t adc_table_len;
    /*
      * 模拟光敏adc查表法标定参数表，为二维数组
      * 其中adc_table_low[i][0]为各环境亮度值，adc_table_low[i][1]为所对应的adc值
      * 环境亮度要求根据实际标定情况从低至高填写，建议第一行的环境亮度为0lux
      * 注：对于双量程类型，adc_table_low代表低量程查表标定参数
      */
    uint32_t adc_table_low[ADC_CALIB_TBL_LEN][2];
    /*
      * 模拟光敏adc查表法标定参数表，为二维数组
      * 其中adc_table_high[i][0]为各环境亮度值，adc_table_high[i][1]为所对应的adc值
      * 环境亮度要求根据实际标定情况从低至高填写，建议第一行的环境亮度为0lux
      * 注：对于双量程类型，adc_table_high代表高量程查表标定参数，对于单量程类型，adc_table_high无效，
可不用填写
      */
    uint32_t adc_table_high[ADC_CALIB_TBL_LEN][2];
    /*
      * 预设三组环境照度（单位：lux），根据环境照度设置RTT初始曝光值，为软光敏算法使用
      */
    uint32_t start_exp[AE_START_EXP_NO];
};

/*
 * 黑夜切白天的awb标定值，预留参数，暂未使用
 */
struct awb_init_cfg {
    /*
     * 黑夜切白天的rgain基准值，无量纲
     */
    uint32_t rg_gain_base;
    /*
     * 黑夜切白天的bgain基准值，无量纲
     */
    uint32_t bg_gain_base;
    /*
     * 黑夜切白天的awb滤波半径，无量纲
     */
```

```
    uint32_t awb_gain_rad;
    /*
     * 黑夜切白天的awb离散阈值，无量纲
     */
    uint32_t awb_gain_dis;
};
```

11.2.3 模拟光敏标定方法

模拟光敏可分为单量程和双量程。单量程模拟光敏，其量程的起始点可通过选用不同电阻进行调整，线性度一般较差，故常使用查表法进行标定。双量程指可通过切换GPIO的高低位，实现不同精度的测量，一般分别应用于暗环境和亮环境的测试，其线性度较好，可使用公式法进行标定，高低量程需要分别标定。

以下针对模拟光敏的标定方法进行介绍。

- **标定设备**

照度计、均匀面光源（DNP灯箱或者积分球等可提供均匀照明的设备），要求亮度可调，且亮度调节范围可达0~5000lux。

- **标定流程**

样机贴近均匀面光源放置，光敏的感应面需要和镜头保持一致，对准光源。照度计放置在样机旁边，与镜头保持平行，感应面与光敏、镜头的朝向一致，对准光源。调整光源亮度，按照从低至高的顺序，待adc值稳定后，分别记录照度计测量的照度值及adc值。建议亮度档位(单位：lux)可按照以下进行测试：0、5、10、50、100、300、500、1000、3000、5000，具体档位可在实测过程中，自行根据数据的线性度进行调整。如光敏为双量程类型，需要切换量程，分别记录环境亮度及其对应adc值。

依据记录的各亮度档位及对应的adc值，可判断当前使用的光敏adc值随环境亮度的变化趋势，变化趋势可分为：线性及非线性。线性趋势的光敏建议采用公式标定法，非线性趋势的光敏建议采用查表法。

- **公式（线性）标定法**

adc值随环境亮度的变化呈线性趋势的光敏，可采用公式标定法。

公式标定法的标定参数adc_calib，参数共4个。对于双量程类型：adc_calib[0]为低量程环境亮度=0lux时，对应的adc值；adc_calib[1]为低量程环境亮度=100lux时，对应的adc值；adc_calib[2]为高量程环境亮度=0lux时，对应的adc值；adc_calib[3]为高量程环境亮度=1000lux时，对应的adc值。

对于单量程类型：adc_calib[0]为环境亮度=0lux时，对应的adc值；adc_calib[1]为环境亮度=100lux时，对应的adc值；adc_calib[2]=adc_calib[0]，为环境亮度=0lux时，对应的adc值；adc_calib[3]为环境亮度=1000lux时，对应的adc值。

$$\begin{cases} lux1 = a * adc1 + b, \\ lux2 = a * adc2 + b \end{cases}$$

算法内部根据填写的标定参数，计算公式的斜率a及截止b。

- **查表法（非线性）**

adc值随环境亮度的变化呈非线性趋势的光敏，可采用查表法。

查表法标定参数为二维数组，分为两列，第一列填写环境亮度档位（单位：lux），第二列填写环境亮度对应的adc值。目前支持至多填写15行，即15个环境亮度档位，要求环境亮度档位按照从低至高填写，建议首档环境亮度为0lux。查表法的原理此处不再赘述，根据输入的adc值，判断档位区间，具体环境亮度值根据插值法计算。标定示例如下：

```
.adc_table_low = {
            {0,1000},
            {6,458},
            {13,446},
            {55,427},
            {102,419},
            {209,410},
            {597,396},
            {1042,388},
            {3017,343},
            {5033,253},
            {7043,254},
        },
```

11.2.4 软光敏快启流程说明

软光敏快启流程与硬光敏快启流程主要的区别在于：

1. RTT阶段软光敏快启平均收敛时间比硬光敏快启方案慢。
2. 由于没有硬件光敏判断环境照度，当rk_night_mode=2（自动红外补光灯夜视模式）时，在应用中加入smartIr算法进行日夜切换。
3. 为了避免快启出现频繁切换ircut的现象，软光敏快启流程中加入rk_color_mode参数，来记录ircut的实际位置情况。

以下对smartIr算法参数、rk_night_mode参数和rk_color_mode参数进行介绍。

- **smartIr调试参数**

当rk_night_mode=2时，应用中加入smartIr算法进行自动日夜切换。其中rk_smart_ir_autoled参数仅在IR补光灯支持亮度可调（设置PWM占空比）才有效。smartIr算法可以参考SDK里`<SDK>/media/isp/camera_engine_rkaiq/smart_ir/`，具体配置可以参考SDK里`project/app/fastboot_client/fastboot_client.c`。

```
typedef struct rk_smart_ir_params_s {
    /*
     * 白天切黑夜的亮度阈值，需要与RTT阶段的亮度阈值相匹配
     * 该值可从json中的AecEnvLvCalib计算获取
     */
    float d2n_envL_th;
    /*
     * 黑夜切白天的亮度阈值，需要调试
     */
    float n2d_envL_th;
    /*
     * 黑夜切白天的rgain基准值，默认值为1
     */
```

```
    float rggain_base;
    /*
     * 黑夜切白天的bgain基准值，默认值为1
     */
    float bggain_base;
    /*
     * 黑夜切白天的awb滤波半径，默认值为0
     */
    float awbgain_rad;
    /*
     * 黑夜切白天的awb离散度阈值，需要调试，默认值为0.2
     */
    float awbgain_dis;
    /*
     * 切换阈值，保持相同状态次数大于该阈值时才允许状态切换，默认值为50
     */
    uint16_t switch_cnts_th;
} rk_smart_ir_params_t;
```

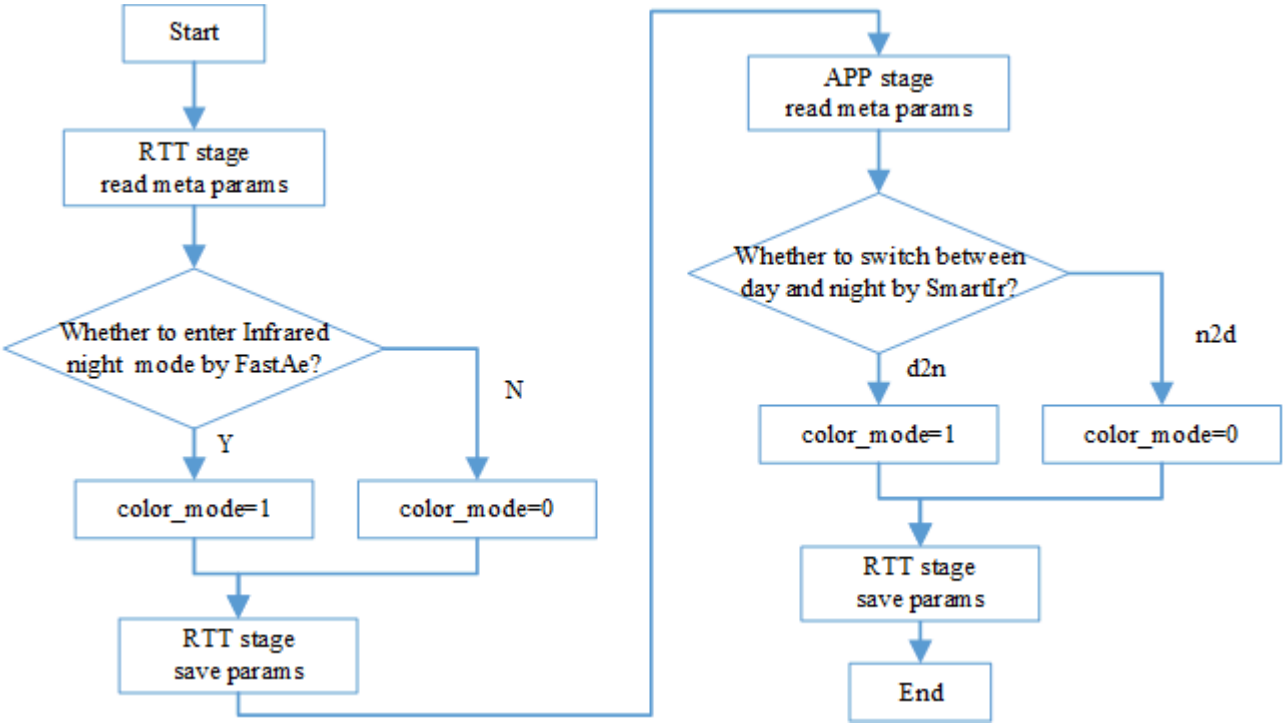
- **rk_night_mode参数**

该参数包含5种夜视模式，可在meta分区中将其设置成特定的夜视模式。

night mode	现象
0：不进行夜视操作	始终工作在白天模式，ircut红外截止，画面为彩色
1：自动进入白光灯夜视	若进入夜视时，开白色补光灯，ircut红外截止，画面为彩色
2：自动进入红外灯夜视	若进入夜视时，开红外补光灯，ircut红外导通，画面为黑白，后续应用smartIr自动日夜切换
3：始终保持白光灯夜视	白色补光灯常亮，ircut红外截止，画面始终为彩色
4：始终保持红外灯夜视	红外补光灯常亮，ircut红外导通，画面始终为黑白

- **rk_color_mode参数**

该参数是在软光敏快启流程中记录当前ircut状态的参数，避免特定环境下ircut频繁切换的问题。该参数不用设置，由软光敏方案自动计算并记录。当rk_color_mode=1时，代表当前（或者上一次断电前）的sensor工作在黑白模式，对应ircut的全透光谱滤光片开始工作。当rk_color_mode=0时，代表当前（或者上一次断电前）的sensor工作在全彩模式，对应ircut的红外截止滤光片开始工作。



11.3 meta分区说明

meta分区目前仅用于RV1126B平台需要快速启动的产品，主要是保存摄像头的初始化参数以及图像效果文件。

meta分区存储布局	说明
32KB	meta数据: META_HEAD PARAM_SHARE2KERNEL_OFFSET SENSOR_INIT_OFFSET CMDLINE_OFFSET AE_TABLE_OFFSET APP_PARAM_OFFSET SECONDARY_SENSOR_INIT_OFFSET RESERVE
32KB backup	meta数据备份: BACKUP
832KB	Sensor IQ文件
832KB	使用双摄方案时，第二份Sensor IQ文件

meta分区详细布局	说明	大小
META_HEAD	meta信息头标记	1KB
PARAM_SHARE2KERNEL_OFFSET	MCU和内核共享内存	1KB
SENSOR_INIT_OFFSET	Sensor初始化参数	1KB
CMDLINE_OFFSET	内核CMDLINE参数	1KB
AE_TABLE_OFFSET	AE/AWB参数	1KB
APP_PARAM_OFFSET	应用参数	1KB
SECONDARY_SENSOR_INIT_OFFSET	使用双摄方案时，第二份Sensor初始化参数	1KB
RESERVE	预留	25KB
BACKUP	备份	32KB
SENSOR_IQ_BIN_OFFSET	Sensor IQ文件	832KB
SECONDARY_SENSOR_IQ_BIN_OFFSET	使用双摄方案时，第二份Sensor IQ文件	832KB

注：meta分区数据结构由`project/make_meta/include/rk_meta.h`定义，各个SDK版本可能存在差异，需要注意版本代码匹配问题。

11.3.1 meta信息头说明

```
struct meta_head {
    uint32_t tag;                               // meta标记
    uint32_t load;                              // meta加载到内存的地址，不可修改
    uint32_t size;                              // meta分区的大小，不可修改
    uint32_t comp_type;                         // Camera Sensor的IQ文件是否压缩（默认gzip）
    uint32_t comp_size;                         // Camera Sensor的IQ文件压缩后的大小
    uint32_t comp_off;                          // Camera Sensor的IQ文件的偏移
    uint32_t iq_item_size;                      // Camera Sensor的IQ文件未压缩后的大小
    uint32_t total_part_num;                    // 分区个数
    uint32_t part_flag;                         // 分区标识
    uint32_t part_reserved_size;                // 分区预留大小
    uint8_t  reserved[META_HEAD_RESERVED_SIZE]; // 预留空间
    uint32_t crc32;                             // meta分区的CRC32
    uint32_t meta_flags;                        // meta分区由SPL读取，设置meta_flag通知MCU开始读
取
};
```

meta_head是在打包meta.img时自动生成，不需要手动修改。系统启动时，SPL会读取meta分区到内存，MCU再去读取。具体代码参考`arch/arm/mach-rockchip/rk_meta.c`

11.3.2 Sensor初始化参数说明

```
struct gpio_info
{
    /*
     * 例如:
     *     GPIO2_D3 (2*32 + 3*8 + 3 = 91)
     *     GPIO3_A2 (3*32 + 0*8 + 2 = 98)
     */
    int gpio_index;
    /*
     * bit25: FLGA_GPIO_INPUT  --> gpio is input
     * bit25: FLGA_GPIO_OUTPUT --> gpio is output
     *
     * bit26: FLAG_GPIO_DEFAULT_LOW  --> gpio default level is low
     * bit26: FLAG_GPIO_DEFAULT_HIGH --> gpio default level is high
     *
     * bit0~bit3: gpio's iomux --> define in sensor_define.h eIOMUX_FUNC
     */
    int gpio_func;

    /*
     * PWM 通道号
     * 例如: PWM11_IR_M1 --> pwm_channel = 11
     */
    int pwm_channel;

    /* PWM 周期, 单位: 纳秒 */
    int pwm_period;
    /* PWM 占空比(Duty Ratio) */
    int pwm_pulse;

    /*
     * ADC 通道号
     * 例如: SARADC_IN1 --> adc_channel = 1
     */
    int adc_channel;

    /*
     * 0: adc值越大环境越暗
     * 1: adc值越大环境越亮
     */
    int adc_direction;
};

struct sensor_init_cfg
{
    uint32_t head;
    int32_t len;
    uint32_t crc32;

    /**
```

```
     * IRCUT
     */
    struct gpio_info ircut_a;
    struct gpio_info ircut_b;

    /**
     * IR_LED
     */
    struct gpio_info led_ir_enable;
    struct gpio_info led_ir;

    /**
     * White_LED
     */
    struct gpio_info led_white_enable;
    struct gpio_info led_white;

    /* ADC */
    struct gpio_info gpio_adc_enable;
    struct gpio_info gpio_adc;

    /* Sensor resolution */
    int32_t cam_w;
    int32_t cam_h;

    /* ambient light sensor: 0: none; 1: analog; 2: digital */
    int32_t als_type;
    /* 白天夜视切换的环境亮度阈值, typical: FIX2INT16(10.0)*/
    int32_t als_value;
};
```

sensor_init_cfg用于配置以下信息：

- 摄像头的分辨率（cam_w/cam_h）
- IRCUT的2个GPIO（ircut_a/ircut_b）
- 红外补光灯（led_ir）
- 红外补光灯使能（led_ir_enable）
- 白光补光灯（led_white）
- 白光补光灯使能（led_white_enable）
- 环境光敏传感器类型（als_type）
- 环境光敏传感器对白天夜视切换的环境亮度阈值（als_value）

sensor_init_cfg具体配置可以参考SDK里`project/make_meta/sensor_init/rv1126b_evb2_gpio.c`

11.3.3 Camera Sensor IQ 参数说明

```
#define SENSOR_IQ_INFO_UPPER_ALIGN(size, align_size)   (((size) + align_size - 1) & (~
(align_size - 1)))
#define SENSOR_IQ_INFO_UPPER_ALIGN_16(size)   (SENSOR_IQ_INFO_UPPER_ALIGN(size, 16))
#define SENSOR_IQ_INFO_UPPER_ALIGN_64(size)   (SENSOR_IQ_INFO_UPPER_ALIGN(size, 64))

struct sensor_iq_info
{
    unsigned int tag;                         // Identify of sensor_iq_info
    unsigned int main_sensor_iq_offset;       // The offset of main camera sensor IQ
    unsigned int main_sensor_iq_size;         // The size of main camera sensor IQ
    unsigned int secondary_sensor_iq_offset;  // The offset of secondary camera sensor IQ
    unsigned int secondary_sensor_iq_size;    // The size of secondary camera sensor IQ
    int len;
    /*
     * Ensure data's offset align with 16 byte
     *
     * 4 = sizeof(unsigned int), sizeof(int)
     * 7 = count of (tag + main_sensor_iq_offset + main_sensor_iq_size
     *      + secondary_sensor_iq_offset + secondary_sensor_iq_size + len + crc32)
     */
    uint8_t  reserved[SENSOR_IQ_INFO_UPPER_ALIGN_16(7*4)-7*4];
    unsigned int crc32;
    uint8_t data[0]; /* Just a way for get data */
};
```

11.3.4 AE/AWB参数说明

```
struct ae_init_cfg {
    uint32_t black_lvl;   // Black level value(12bit)
    uint32_t adc_calib_type; //0: Formula 1: LUT(look up table)
    uint32_t adc_range_type; //0: dual 1: single

    /*
     * adc_calib[0]=lowadc_lux0
     * adc_calib[1]=lowadc_lux100
     * adc_calib[2]=highadc_lux0
     * adc_calib[3]=highadc_lux1000, 10bit
     */
    uint32_t adc_calib[ADC_CALIB_NO];

    uint32_t adc_table_len;
    uint32_t adc_table_low[ADC_CALIB_TBL_LEN][2];
    uint32_t adc_table_high[ADC_CALIB_TBL_LEN][2];
    uint32_t start_exp[AE_START_EXP_NO]; //typical: start_exp[0]=1lux, start_exp[1]=10lux,
start_exp[2]=100lux
};

struct awb_init_cfg {
```

```
    uint32_t rg_gain_base; //used for soft light sensor, need float => FIX2INT16
    uint32_t bg_gain_base;
    uint32_t awb_gain_rad;
    uint32_t awb_gain_dis;
};

struct rtt_2a_res {
    uint32_t day_or_night; // 0: invalid 1:day 2:night
    uint32_t awb_gain; //0~-15bit: rgain 16~31bit: bgain
    uint32_t exp;
    uint32_t isp_dgain;
    uint32_t reg_time;
    uint32_t reg_gain;
    uint32_t dcg_mode;
    uint32_t ae_stats[AE_GRID_STATS]; // only G channel (16bit)
    uint32_t ae_diff_th;//unit:%
    uint32_t nr_diff_th;//unit:%
    uint32_t last_lodif_0;
    uint32_t last_lodif_1;
    uint32_t last_hidif_0;
    uint32_t last_hidif_1;
};

struct ae_awb_init_cfg {
    uint32_t head;
    int32_t len;
    uint32_t crc32;
    struct ae_init_cfg ae;
    struct awb_init_cfg awb;
    struct rtt_2a_res rtt_res;
};
```

ae_awb_init_cfg具体配置可以参考SDK里`project/make_meta/sensor_ae_awb_table/sc450ai_2a.c`

11.3.5 应用参数说明

```
struct app_param_info
{
    uint32_t head; // app参数信息标记
    int32_t len;   // app_param_info大小

    /*
     * 设置主摄像头水平镜像和垂直镜像
     *    0: NONE
     *    1: Mirror Horizontally
     *    2: Mirror Vertically
     *    3: Mirror Horizontally and Vertically
     */
    int32_t cam_mirror_flip;

    int32_t cam_fps; // 设置摄像头帧率
```

```
    int32_t cam2_fps;

    /*
     * 设置补光灯
     *    0: 不使用补光灯
     *    1: 自动白光灯模式
     *    2: 自动红外灯模式
     *    3: 固定白光灯模式
     *    4: 固定红外灯模式
     */
    int32_t night_mode;

    int32_t led_value;    // 设置补光灯亮度
    int32_t venc_w;       // 设置主摄的编码分辨率宽
    int32_t venc_h;       // 设置主摄的编码分辨率高
    int32_t venc_type;    // 设置主摄的编码类型 RK_VENC_TYPE_H264/RK_VENC_TYPE_H265
    int32_t venc_bitrate; // 设置主摄的编码比特率（默认是venc_w x venc_h）

    /*
     * 设置摄像头是否使用HDR功能
     *    0: 不使用HDR
     *    5: 使用2帧HDR
     */
    int32_t cam_hdr;
    int32_t cam2_hdr;

    /*
     * 记录摄像头sensor工作在黑白模式或全彩模式
     *    1: 黑白模式
     *    0: 全彩模式
     * 内核cmdline里会记录color_mode的物理地址rk_color_mode
     * 具体参考代码: project/app/fastboot_client/fastboot_client.c
     */
    int32_t color_mode;

    /* 多摄像头时，指定摄像头启动顺序 */
    int32_t cam_index;

    /* 设置MCU快速AE收敛的最大帧数 */
    int32_t fastae_max_frame;
    /*
     * MCU出大图的条件
     * MCU AE收敛帧数小于rtt_stream_on_threshold时使用MCU出大图
     * MCU AE收敛帧数大于等于rtt_stream_on_threshold时使用ARM出大图
     */
    int32_t rtt_stream_on_threshold;
    /* MCU大图出图帧数 */
    int32_t rtt_stream_buf_cnt;

    /*
     * 定义是双摄或单摄
     * 1: 单摄
     * 2: 双摄
```

```
     */
    int32_t cam_num;

    int32_t venc2_w;       // 设置副摄的编码分辨率宽
    int32_t venc2_h;       // 设置副摄的编码分辨率高
    int32_t venc2_type;    // 设置副摄的编码类型 RK_VENC_TYPE_H264/RK_VENC_TYPE_H265
    int32_t venc2_bitrate; // 设置副摄的编码比特率（默认是venc_w x venc_h）

    /*
     * 设置副摄像头水平镜像和垂直镜像
     *    0: NONE
     *    1: Mirror Horizontally
     *    2: Mirror Vertically
     *    3: Mirror Horizontally and Vertically
     */
    int32_t cam2_mirror_flip;

    /* 主摄内核驱动支持的最大帧率 */
    int32_t cam1_max_fps;
    /* 辅摄内核驱动支持的最大帧率 */
    int32_t cam2_max_fps;

    /* 离线帧大图跳帧间隔 */
    uint32_t cam_skip_frame_interval;

    /* MCU AE收敛容忍度 */
    int32_t fastae_tolerance_coef;

    /* 预录曝光信息 */
    uint32_t pr_exp_gain;
    uint32_t pr_exp_time;

    uint32_t crc32; // app_param_info的CRC32
};
```

11.3.6 内核CMDLINE参数说明

```
struct cmdline_info {
    uint32_t tag;                       // cmdline_info标记
    uint8_t data[MAX_CMDLINE_LENGTH];   // 写内核cmdline的参数
    uint32_t crc32;                     // cmdline_info的CRC32
};
```

目前传递给内核cmdline的变量有如下：

```
meta_load_addr=0x41240000
rk_app_param_offset=0x1400
meta_part_size=0x000e0000
rk_cam_mirror_flip=0
rk_cam2_mirror_flip=0
```

```
rk_cam_fps=15
rk_cam2_fps=15
rk_cam1_max_fps=60
rk_cam2_max_fps=60
rk_cam_skip=0
rk_cam_hdr=0
rk_cam2_hdr=0
rk_cam_index=0
rk_night_mode=2
rk_led_value=20
rk_cam_num=1
rk_color_mode=0x41241434
rk_iqbin_addr=0x41250040
rk_iqbin_size=0x000b3508
rk_cam_h=1080
rk_cam_w=1920
```

rk_color_mode这个地址记录的值为1时，camera sensor工作在黑白模式; 为0时，camera sensor工作在全彩模式。
具体可以参考fastboot_server.c代码。

rk_iqbin_addr/rk_iqbin_size记录camera sensor的IQ效果配置文件的内存地址和大小。
具体可以参考fastboot_server.c代码。

11.3.7 生成meta.img

生成meta.img的工具在project/make_meta目录。具体步骤如下：

- 在BoardConfig.mk里增加meta分区（分区大小最大896KB，不建议修改）
- 编译

`./build.sh meta`

11.3.8 单板端meta分区操作

11.3.8.1 查看meta分区数据

```
make_meta -d  /dev/block/by-name/meta
```

11.3.8.2 更新meta分区数据

更新meta分区，可以使用make_meta命令或者参考代码
`project/make_meta/make_meta/sample_read_write_meta.c`

以下是用make_meta更新的例子（/dev/block/by-name/meta是meta分区设备节点）：

11.3.8.3 更新cmdline，使能内核打印开机log

```
make_meta --update --meta_path=/dev/block/by-name/meta --cmdline="loglevel=8"
```

11.3.8.4 更新cmdline，使能摄像头AIQ的调试log

```
# log在/tmp/rkaiq0.log /tmp/rkaiq1.log
make_meta --update --meta_path /dev/block/by-name/meta --cmdline
"persist_camera_engine_log=0x3ff4"

# 打开smart IR相关log，保存在/tmp/rkaiq1.log
make_meta --update --meta_path /dev/block/by-name/meta --cmdline
"persist_camera_engine_log=0x800000000"
```

11.3.8.5 更新Sensor初始化

- 编译系统会根据BoardConfig.mk里配置的IQ文件选择编译对应的Sensor
- Sensor的初始化bin文件：project/make_meta/sensor_init/init_sensor.bin
- 拷贝init_sensor.bin 到单板（例如/userdata目录）

```
make_meta --update --meta_path=/dev/block/by-name/meta --
sensor_init="/userdata/init_sensor.bin"
```

注：详细修改请参考**project/make_meta/readme.txt**

11.3.8.6 更新Sensor IQ文件

例如，拷贝IQ文件到单板（例如/userdata目录）

```
# 针对单摄
make_meta --update --meta_path=/dev/block/by-name/meta --sensor_iq_bin
/userdata/sc3338_FKO1_30IRC-F16.bin

# 对于双摄，同时更新主摄和副摄
make_meta --update --meta_path=/dev/block/by-name/meta --sensor_iq_bin
/userdata/sc3338_FKO1_30IRC-F16.bin --secondary_sensor_iq_bin
/userdata/sc4336_OT01_40IRC_F16.bin

# 对于双摄，只更新主摄
make_meta --update --meta_path=/dev/block/by-name/meta --sensor_iq_bin
/userdata/sc3338_FKO1_30IRC-F16.bin

# 对于双摄，只更新副摄
make_meta --update --meta_path=/dev/block/by-name/meta --secondary_sensor_iq_bin
/userdata/sc4336_OT01_40IRC_F16.bin
```

11.3.8.7 其他参数说明

参数	说明
--create	创建meta.img镜像
--update	更新--meta_path指定的meta镜像
-d	查看meta分区数据
-r	从备份里恢复数据（IQ文件不支持）
--cmdline	添加参数到内核CMDLINE
--meta_path	指定meta分区路径
--sensor_init	指定init_sensor.bin路径
--ae_awb_tab	指定ae_awb_table.bin路径
--sensor_iq_bin	指定Sensor的IQ文件路径
--meta_load_addr	meta分区加载到内存的地址（默认不改）
--meta_part_size	meta分区大小（最大896KB）
--rk_cam_w	主摄Sensor分辨率的宽
--rk_cam_h	主摄Sensor分辨率的高
--rk_cam2_w	副摄Sensor分辨率的宽
--rk_cam2_h	副摄Sensor分辨率的高
--rk_cam_mirror_flip	主摄Sensor水平镜像和垂直镜像功能 0: 无 1: 水平镜像 2: 垂直镜像 3: 水平镜像和垂直镜像
--rk_cam2_mirror_flip	副摄Sensor水平镜像和垂直镜像功能 0: 无 1: 水平镜像 2: 垂直镜像 3: 水平镜像和垂直镜像
--rk_cam_fps	Sensor帧率
--rk_led_value	补光灯亮度<size: 0~100>

参数	说明
--rk_night_mode	补光灯选择 0: 不开补光灯 1: 使用白光补光灯 2: 使用红外补光灯 3: 白光补光灯常开 4: 红外补光灯常开
--rk_venc_w	主摄的编码器的宽度
--rk_venc_h	主摄的编码器的高度
--rk_venc_type	主摄的编码类型 <1: h264, 2: h265>
--rk_venc_bitrate	主摄的编码码率 KB (默认rk_venc_w x rk_venc_h)
--rk_venc2_w	副摄的编码器的宽度
--rk_venc2_h	副摄的编码器的高度
--rk_venc2_type	副摄的编码类型 <1: h264, 2: h265>
--rk_venc2_bitrate	副摄的编码码率 KB (默认rk_venc_w x rk_venc_h)
--rk_cam_hdr	摄像头Sensor的HDR模式 0: 不开HDR（默认） 5: 2帧HDR
--rk_cam_index	多摄像头时，指定摄像头启动顺序
--rk_color_mode	设置场景是全彩或黑白状态 0：全彩 1：黑白
--rk_fastae_max_frame	设置MCU快速AE收敛的最大帧数

11.4 修改系统内存布局

产品开发时，内核和文件系统可能会变大，超过SDK默认配置的大小。此时就得调整内存布局。
假设ramdisk解压后的大小是10MB，这边以修改ramdisk大小为例。

11.4.1 修改内核DTS的内存布局

SDK默认的内存布局在/sysdrv/source/kernel/arch/arm64/boot/dts/rockchip/rv1126b-thunder-boot.dtsi

注意：tee、rtos、meta节点的offset和size都不能修改，rkisp_thunderboot节点的offset不能修改，大小需要根据分辨率进行修改，rkisp_thunderboot和rkisp_thunderboot1总大小不能超过0x70E0000。

```
&reserved_memory {
```

```
/*
 * reserved memory layout
 *
 *  H        _____________________
 *       |           |
 *       |   ramdisk_c   |
 *       |___________________|
 *       |           |
 *       |   ramdisk_r   |
 * 0x48c40000   |___________________|
 *       |           |
 *       |        mcu_log    |
 * 0x48c3c000   |___________________|
 *       |           |
 *       |         mcu       |
 * 0x48c00000   |___________________|
 *       |           |
 *       |     [TEEOS+TA+SHM]    |
 * 0x48400000   |___________________|
 *       |           |
 *       |    isp[0..1]_tb_rmem  |
 *       |___________________|
 *       |           |
 *       |         meta      |
 * 0x41240000   |___________________|
 *       |           |
 *       |      fdt      |
 * 0x41200000   |___________________|
 *       |           |
 *       |   kernel_r    |
 * 0x40208000   |___________________|
 *       |           |
 *       |      bl31     |
 * L        |___________________|
 *
 */

meta: meta@41240000 {
    /* reg's offset MUST match with RTOS */
    reg = <0x41240000 0x000e0000>;
};

rkisp_thunderboot: rkisp@41320000 {
    /* reg's offset MUST match with RTOS */
    /*
     * vicap, capture raw10, ceil(w*10/8/256)*256*h *4(buf num)
     * e.g. 1920x1080: 0xa8c000
     */
    reg = <0x41320000 0xa8c000>;
};

rkisp1_thunderboot: rkisp1_thunderboot {
    /* vicap capture for a second camera */
```

```
    };

    /* Should enable this node if the security feature is enabled, like TA. */
    tee: tee@48400000 {
        reg = <0x48400000 0x00800000>;
        status = "disabled";
    };

    rtos: rtos@48c00000 {
        reg = <0x48c00000 0x0003a000>;
    };

    mcu_log: mcu_log@48c3c000 {
        reg = <0x48c3c000 0x4000>;
    };

    ramdisk_r: ramdisk_r {
        reg = <0x48c40000 (10 * 0x00100000)>;
    };

    ramdisk_c: ramdisk_c {
        reg = <0x49640000 (5 * 0x00100000)>;
    };
};
```

对应的dts文件(例如：/sysdrv/source/kernel/arch/arm/boot/dts/rv1126b-evb2-v10-tb-400w.dts)末尾增加如下：

```
/* ramdisk_r is the max size of output/out/fastboot/rootfs_romfs.img */
&ramdisk_r {
    reg = <0x48c40000 (30 * 0x00100000)>;
};

/* ramdisk_c is the max size of output/out/fastboot/rootfs_romfs.img.gz */
&ramdisk_c {
    reg = <0x4aa40000 (20 * 0x00100000)>;
};
```

11.4.2 修改FIT打包脚本

修改文件/project/scripts/rv1126b-boot-tb-ramdisk.its

- ramdisk节点里load的值为ramdisk_r
- ramdisk节点里comp的值为ramdisk_c
- kernel节点里comp的值为kernel_c, 编译时自动根据ramdisk_c计算

Configuration in DTS

```
&ramdisk_r {
    reg = <0x48c40000 (30 * 0x00100000)>;
};

&ramdisk_c {
    reg = <0x4aa40000 (20 * 0x00100000)>;
};
```

The output of fdtdump rootfs.img

```
ramdisk {
    data-size = <0x00a4791d>;
    data-position = <0x00000800>;
    compression = "gzip";
    type = "ramdisk";
    arch = "arm";
    os = "linux";
    preload = <0x00000001>;
    comp = <0x4aa40000>;
    load = <0x48c40000>;
    decomp-async;
    hash {
        value = <0x6f47c336 0x85f67cf7 0x395e3492 0x68c2e3db 0x99eb6297 0x2bd43cf3 0xf6cc2ead 0xc69b160b>;
        algo = "sha256";
        uboot-ignore = <0x00000001>;
    };
};
```

11.4.3 重新打包编译

```
./build.sh kernel
./build.sh firmware
```

11.5 MCU快启驱动开发说明

参考Rockchip_Developer_Guide_Camera_Driver_CN.md中的快速启动章节。

12. AVS说明

AVS(Any View Stitching)多目拼接算法将多路输入图像拼接为一路图像，从而实现获取更大视野的需求。

API请参考 Rockchip_Developer_Guide_MPI_C_CN.pdf中AVS部分。

标定说明文档和工具较大，请通过网盘链接下载：https://console.zbox.filez.com/l/g0a8F8

13. AI-ISP说明

AI-ISP是一种通过深度学习的方式提升画质的算法，具体包含噪声、清晰度、对比度、亮度和色彩的调整。

RV1126B集成专用 AI-ISP 硬件，突破传统方案的算力瓶颈，运行 AI-ISP时无需占用通用NPU 资源。

应用上需要使用media/isp/camera_engine_rkaiq/rkaiq/iqfiles/isp35/ainr中的IQ和模型

API和框图请参考Rockchip_Development_Guide_ISP35_CN.pdf中aibnr部分。

参数调试请参考Rockchip_Tuning_Guide_ISP35_CN.pdf中aibnr部分。

14. AI-Remosaic说明

Quad Bayer 和 4CELL 本质上描述的是同一种传感器技术。它们核心的共性是：将四个同色像素（2x2阵列）排列在一起，形成一个“超级像素”，提高暗光下的效果。

Remosaic（像素重排）是将Quad Bayer/4CELL传感器采集的原始数据，转换回传统Bayer阵列排列的过程，以输出全分辨率图像。

RV1126B的AI Remosaic功能是指通过芯片集成的专用AI-ISP硬件，将传感器采集的Quad Bayer阵列数据实时转换为传统的Bayer阵列。在不占用通用NPU算力的前提下，实现日夜双模自适应成像——白天输出超高清画质，夜晚在极低照度下仍能保持清晰图像.

应用上需要使用media/isp/camera_engine_rkaiq/rkaiq/iqfiles/isp35/airms中的IQ和模型.

sensor需要支持输出4CELL格式，可参考sysdrv/source/kernel/drivers/media/i2c/os12d40.c。

15. FEC说明

鱼眼畸变矫正（Fisheye Distortion Correction，简称FEC）实现校正由鱼眼镜头造成的图像畸变，使图像看起来更接近人眼自然视角的过程。

请参考Rockchip_Developer_Guide_MPI_C_CN.pdf里的GDC部分和Rockchip_Development_Guide_FEC_CN.pdf。

16. IPC Linux SDK Q&A

16.1 如何修改分区表和增加自定义分区以及分区可读写说明

16.1.1 存储介质以及文件系统类型说明

存储介质	支持的可读可写文件系统格式	支持的只读文件系统格式
eMMC	ext4	squashfs
spi nand 或slc nand	ubifs	squashfs
spi nor	jffs2	squashfs

文件系统格式	制作烧录镜像文件的脚本
ext4	output/out/sysdrv_out/pc/mkfs_ext4.sh
jffs2	output/out/sysdrv_out/pc/mkfs_jffs2.sh
ubifs	output/out/sysdrv_out/pc/mkfs_ubi.sh
squashfs	output/out/sysdrv_out/pc/mkfs_squashfs.sh

注：Nand Flash硬件有不同的page size和block size，需要烧录对应的镜像文件，所以mkfs_ubi.sh默认会打包出不同page size和block size的烧录镜像。

详细的Nand Flash说明可以参考文档 Rockchip_Developer_Guide_Linux_Nand_Flash_Open_Source_Solution_CN.pdf

16.1.2 分区表说明

SDK使用env分区设置分区表，分区表信息配置在`<SDK>/project/cfg/BoardConfig_IPC/BoardConfig*.mk`里的`RK_PARTITION_CMD_IN_ENV`参数中。

分区表以字符串的形式保存在配置中，以下是各存储介质的分区表例子。

存储介质	分区表
eMMC	RK_PARTITION_CMD_IN_ENV="32K(env),512K@32K(idblock),4M(uboot),32M(boot),2G(rootfs),-(userdata)"
spi nand 或slc nand	RK_PARTITION_CMD_IN_ENV="256K(env),256K@256K(idblock),1M(uboot),8M(boot),32M(rootfs),-(userdata)"
spi nor	RK_PARTITION_CMD_IN_ENV="64K(env),128K@64K(idblock),128K(uboot),3M(boot),6M(rootfs),-(userdata)"

每个分区的格式为：`<size>[@<offset>](part-name)`，其中，分区大小和分区名是必须的，偏移量则因情况而定（见以下注意事项第三点）。

在配置分区表时有以下几点注意事项：

1. 分区之间用英文逗号","隔开。
2. 分区大小的单位有：K/M/G/T/P/E，不区分大小写，无单位则默认为byte；"-"表示该分区大小为剩余容量。
3. 第一个分区若从0x0地址开始时，不加偏移量，反之，则必须添加偏移量。后续分区任意选择是否添加偏移量。
4. idblock分区的偏移固定的，**请勿修改**。
5. 不建议修改env分区名。（如果要修改env地址和大小，需要修改对应U-Boot的defconfig 配置 CONFIG_ENV_OFFSET和CONFIG_ENV_SIZE，重新生成固件，擦除板端0地址的env数据，再烧录新的固件）

16.1.3 增加自定义分区

以下是eMMC增加一个大小为64MB的可读写分区custom_part的例子。

注意：自定义分区的分区名不能命名为update等SDK里已有分区的分区名。

- 使用上述方法（分区表说明）修改对应板级配置中的分区表参数，以此例为：64M(custom_part)

- 修改`<SDK>/project/cfg/BoardConfig*.mk`里配置增加分区`RK_PARTITION_CMD_IN_ENV`和`RK_PARTITION_FS_TYPE_CFG`分区挂载

```
# config partition's filesystem type (squashfs is readonly)
# emmc:    squashfs/ext4
# nand:    squashfs/ubifs
# spi nor: squashfs/jffs2
# RK_PARTITION_FS_TYPE_CFG format:
#     AAAA@/BBBB/CCCC@DDDD
#         AAAA ----------> partition name
#         /BBBB/CCCC ----> partition mount point
#         DDDD ----------> partition filesystem type (squashfs/ext4/ubifs/jffs2)
export RK_PARTITION_FS_TYPE_CFG=rootfs@IGNORE@ext4,custom_part@/opt/custom_part@ext4

# config partition in environment
# RK_PARTITION_CMD_IN_ENV format:
#     <partdef>[,<partdef>]
#       <partdef> := <size>[@<offset>](part-name)
#
export
RK_PARTITION_CMD_IN_ENV="32K(env),512K@32K(idblock),4M(uboot),32M(boot),2G(rootfs),64M(custo
m_part),-(userdata)"
```

- 制作custom_part分区镜像

```
mkdir -p custom_part
./output/out/sysdrv_out/pc/mkfs_ext4.sh custom_part custom_part.img 64*0x100000
# 注意：使用SDK默认的挂载脚本时，分区镜像文件名需要以分区名称命名
#       例如：分区名称是custom_part，分区镜像名称custom_part.img
```

- 在文件系统创建custom_part分区的挂载目录

```
mkdir -p output/out/rootfs_glibc_rv1126b/opt/custom_part
```

- 重新打包根文件系统（rootfs.img）`./build.sh firmware`
- 烧录rootfs.img、env.img以及custom_part.img

16.1.4 oem分区挂载说明

挂载oem分区需要进行如下配置：

- 在分区表中添加oem分区（详细见分区表说明），例如：

RK_PARTITION_CMD_IN_ENV="32K(env),512K@32K(idblock),4M(uboot),32M(boot),2G(rootfs),**64M(oem)**,-(userdata)"

- 在文件系统类型的配置里添加oem相关配置（详细见增加自定义分区），例如：

RK_PARTITION_FS_TYPE_CFG=rootfs@IGNORE@ext4,**oem@/oem@ext4**

- 开启如下配置：

```
# enable oem partition to install app
export RK_BUILD_APP_TO_OEM_PARTITION=y
```

16.2 如何在U-Boot终端下使用tftp进行分区升级

tftp升级文件会随固件一同编译在`<SDK>/output/image/`目录下，文件名为`tftp_update.txt`，使用方法如下：

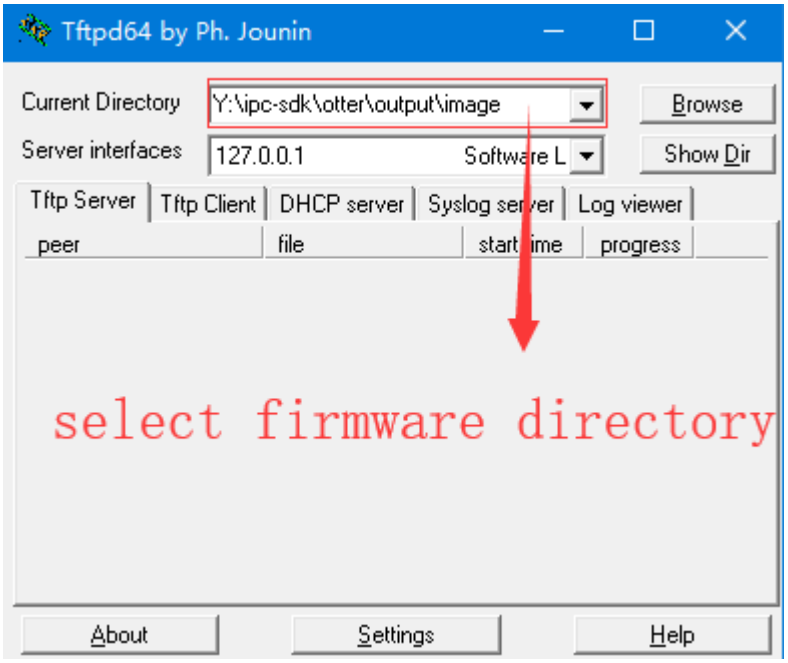
- 配置tftp服务器

Tftpd64下载地址 https://pjo2.github.io/tftpd64

注意：

1、使用Tftpd64软件需要遵守相关的开源协议

2、使用Tftpd64带来的所有的法律风险以及后果全部由客户自己承担



- 将升级文件`tftp_update.txt`和所有后缀名为`.img`的固件放进服务器指定的目录下

（注：slc nand暂时不支持以下载固件的方式升级`idblock`分区。）

- U-Boot终端下设置IP地址

```
=> setenv ipaddr 192.168.1.111
=> setenv serverip 192.168.1.100
=> saveenv
Saving Environment to envf...
=>
```

以上IP地址仅供参考，请根据实际情况自行设置，保证客户端与服务器在同一网段即可。

- U-Boot终端下运行升级指令`tftp_update`

```
=>  tftp_update
ethernet@ffc40000 Waiting for PHY auto negotiation to complete. done
Using ethernet@ffc40000 device
TFTP from server 192.168.1.100; our IP address is 192.168.1.111
Filename 'tftp_update.txt'.
Load address: 0x3be24c00
Loading: *•#
     203.1 KiB/s
done
Bytes transferred = 1250 (4e2 hex)
...
```

16.3 如何在U-Boot终端下使用SD卡进行分区升级

SD卡升级文件会随固件一同编译在`<SDK>/output/image/`目录下，文件名为`sd_update.txt`，使用方法如下：

- 将升级文件`sd_update.txt`和所有后缀名为`.img`的固件放进SD卡根目录下

（注：1. slc nand暂时不支持以下载固件的方式升级`idblock`分区；2. SD卡仅支持FAT格式的文件系统。）

- 设备端插上SD卡
- U-Boot终端下运行升级指令`sd_update`

```
=> sd_update
PartType: ENV
reading sd_update.txt
1511 bytes read in 2 ms (737.3 KiB/s)
...
```

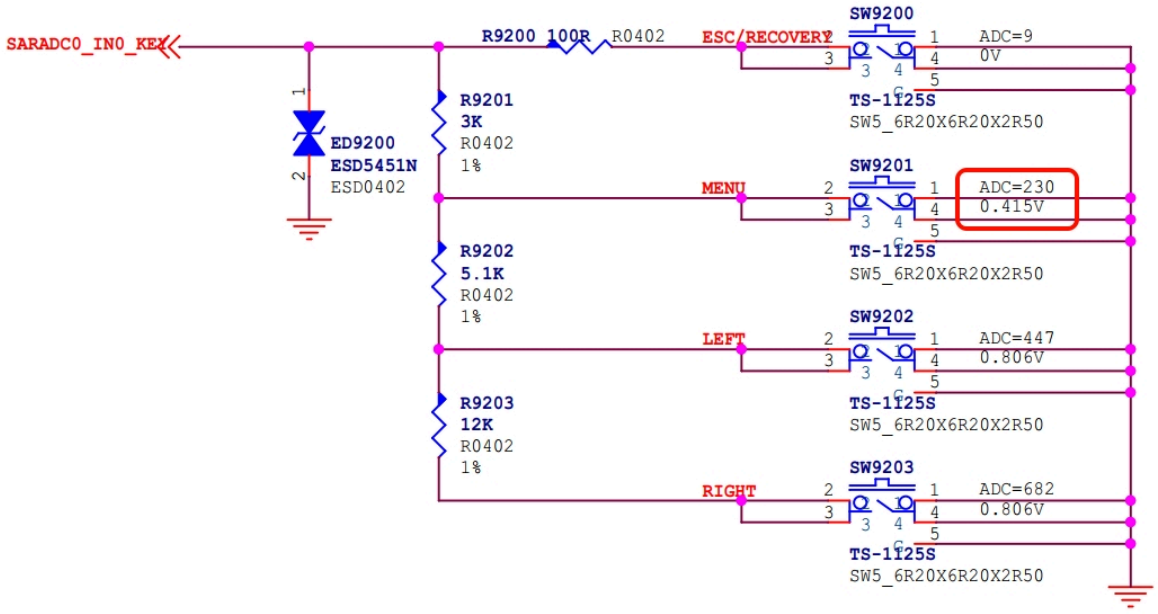
- 升级完成后重启设备

16.4 如何在U-Boot阶段通过按键触发SD卡升级功能

该功能需要ADC功能。U-Boot配置好相应dts与defconfig后，通过按住按键并重启的方式，在U-Boot阶段触发`sd_update`升级设备分区。具体配置流程如下所示：

- 根据硬件电路图，获取按键电压值，用于ADC配置

以RV1126B EVB1 V10的menu按键为例，获取对应ADC电压值为230。



- 配置相应dts的adc-keys节点

以RV1126B EVB1 V10为例，修改rv1126b-evb.dts中的adc-keys。

```
diff --git a/arch/arm/dts/rv1126b-evb.dts b/arch/arm/dts/rv1126b-evb.dts
index 344558f8f9..10caf5b0da 100644
--- a/arch/arm/dts/rv1126b-evb.dts
+++ b/arch/arm/dts/rv1126b-evb.dts
@@ -27,6 +27,13 @@
+     adc-keys {
+         compatible = "adc-keys";
+         io-channels = <&saradc0 0>;
+         io-channel-names = "buttons";
+         keyup-threshold-microvolt = <1800000>;
+         u-boot,dm-pre-reloc;
+         status = "okay";
+     };
 };
```

- 配置相应defconfig的CONFIG_ADC和CONFIG_SARADC_ROCKCHIP_V2
- 添加按键检查代码

rockchip_dnl_key_pressed 函数里val值即为上面原理图ADC值（以menu 按键为例，val=230）。

```
diff --git a/arch/arm/mach-rockchip/boot_rkimg.c b/arch/arm/mach-rockchip/boot_rkimg.c
index 4aed2a36bc..0e2ea4f7f9 100644
--- a/arch/arm/mach-rockchip/boot_rkimg.c
+++ b/arch/arm/mach-rockchip/boot_rkimg.c
@@ -320,6 +320,11 @@ __weak int rockchip_dnl_key_pressed(void)
                return 0;
        }

+       /* val以实际ADC范围为准 */
+       if ((val >= 200) && (val <= 300)) {
```

```
+               run_command("sd_update", 0);
+               return 0;
+       }
+
        return ((val >= KEY_DOWN_MIN_VAL) && (val <= KEY_DOWN_MAX_VAL));
 #endif
```

16.5 插入SD卡检测不到设备

对应内核的defconfig需要打开支持SD卡的配置。

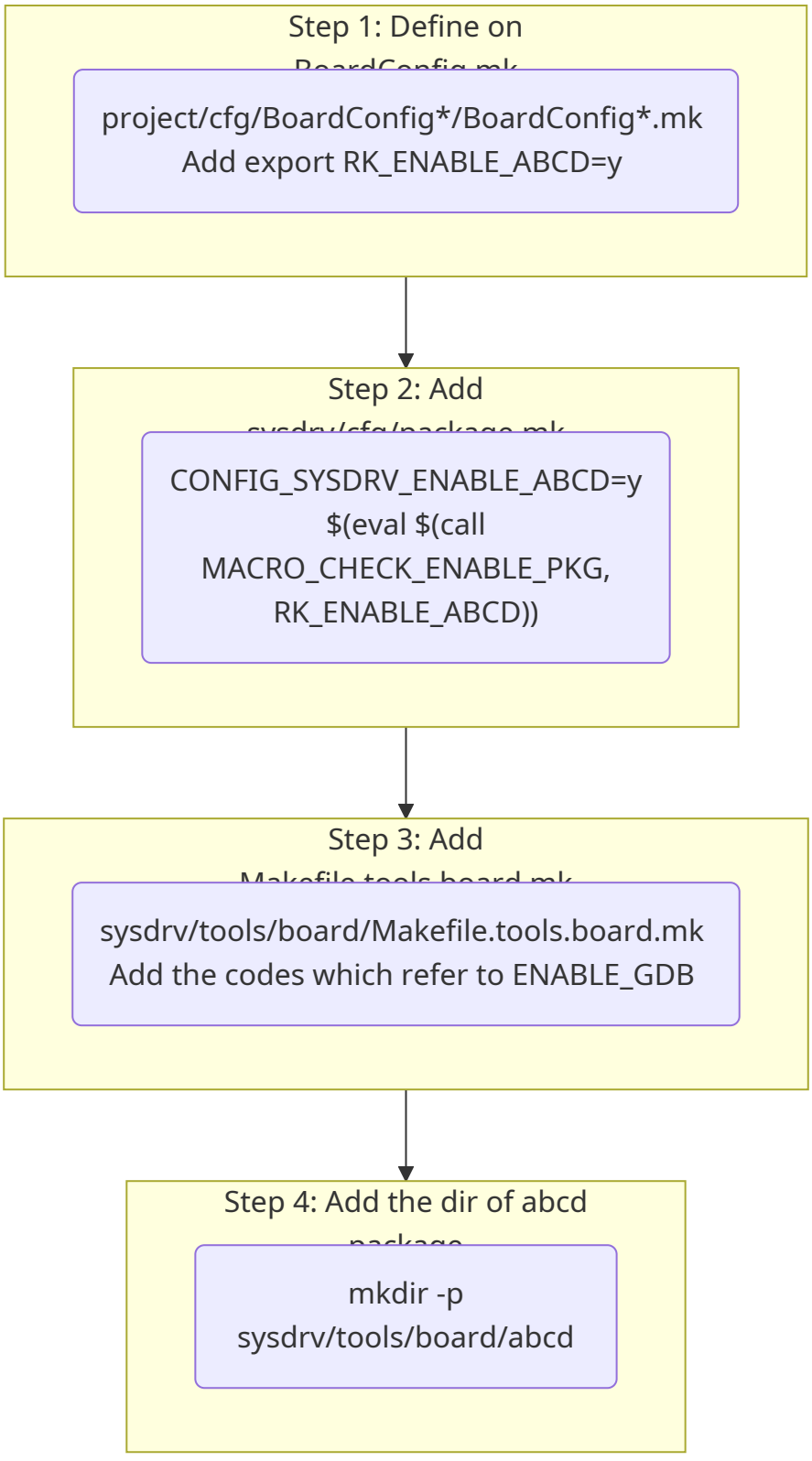
```
CONFIG_MMC_DW=y
CONFIG_MMC_DW_PLTFM=y
CONFIG_MMC_DW_ROCKCHIP=y
```

增加vfat文件系统支持

```
CONFIG_VFAT_FS=y
CONFIG_MSDOS_PARTITION=y
CONFIG_EFI_PARTITION=y
```

16.6 如何添加第三方库到sysdrv目录编译

- sysdrv/tools/board运行在板子上的程序，这里介绍如何添加一个第三方程序（例如第三方程序名为ABCD）。



- `touch sysdrv/tools/board/abcd/Makefile`

```
# sysdrv/tools/board/abcd/Makefile reference code
ifeq ($(SYSDRV_PARAM), )
SYSDRV_PARAM:=../../../Makefile.param
include $(SYSDRV_PARAM)
endif
```

```
export LC_ALL=C
SHELL:=/bin/bash

CURRENT_DIR := $(shell pwd)
PKG_TARBALL := abcd.tar.xz
PKG_NAME := abcd
PKG_BIN := out

all:
    rm -rf $(CURRENT_DIR)/$(PKG_NAME); \
    tar -xf $(PKG_TARBALL); \
    mkdir -p $(CURRENT_DIR)/$(PKG_NAME)/$(PKG_BIN); \
    mkdir -p $(CURRENT_DIR)/$(PKG_BIN); \
    pushd $(CURRENT_DIR)/$(PKG_NAME)/; \
        ./configure --host=$(SYSDRV_CROSS) \
        --target=$(SYSDRV_CROSS) CFLAGS="$(SYSDRV_CROSS_CFLAGS)" \
        LDFLAGS="$(SYSDRV_CROSS_CFLAGS)" \
        --prefix=$(CURRENT_DIR)/$(PKG_NAME)/$(PKG_BIN); \
        make -j$(SYSDRV_JOBS) > /dev/null || exit -1; \
        make install > /dev/null; \
    popd; )
    $(call MAROC_COPY_PKG_TO_SYSDRV_OUTPUT, $(SYSDRV_DIR_OUT_ROOTFS), $(PKG_BIN))

clean: distclean

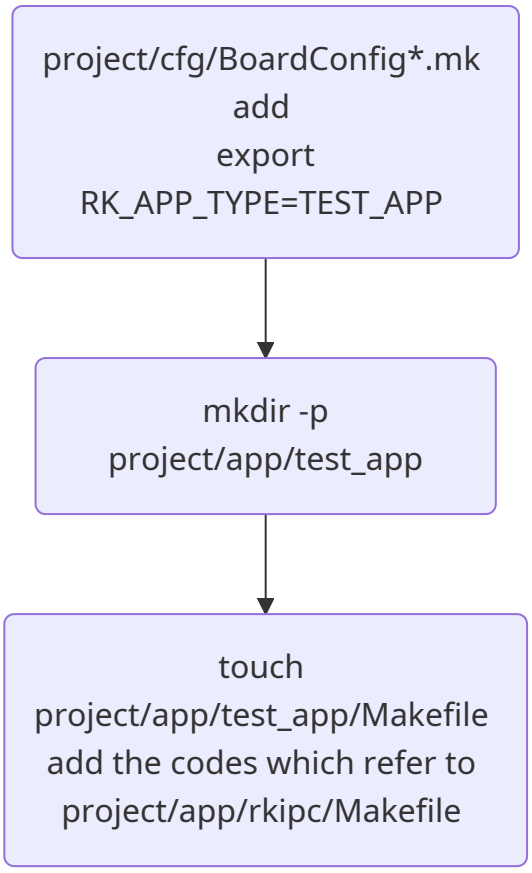
distclean:
    -rm -rf $(PKG_NAME) $(PKG_BIN)
```

- Test and build

```
cd sysdrv/tools/board/abcd/
make
cd ../../..
make
```

16.7 如何在project/app里增加新应用程序

- 以增加应用程序test_app为例




```
# project/app/test_app/Makefile reference code

ifeq ($(APP_PARAM), )
APP_PARAM:=../Makefile.param
include $(APP_PARAM)
endif

export LC_ALL=C
SHELL:=/bin/bash

CURRENT_DIR := $(shell pwd)

PKG_NAME := test_app
PKG_BIN ?= out
PKG_BUILD ?= build

RK_APP_CFLAGS = -I $(RK_APP_MEDIA_INCLUDE_PATH)

RK_APP_LDFLAGS = -L $(RK_APP_MEDIA_LIBS_PATH)

RK_APP_OPTS += -Wl,-rpath-
link,$(RK_APP_MEDIA_LIBS_PATH):$(RK_APP_PATH_LIB_INCLUDE)/root/usr/lib
PKG_CONF_OPTS += -DCMAKE_C_FLAGS="$(RK_APP_CFLAGS) $(RK_APP_LDFLAGS) $(RK_APP_OPTS)" \
                 -DCMAKE_CXX_FLAGS="$(RK_APP_CFLAGS) $(RK_APP_LDFLAGS) $(RK_APP_OPTS)"

# define project/cfg/BoardConfig*.mk
ifneq ($(findstring $(RK_APP_TYPE),TEST_APP),)
```

```
PKG_TARGET := test_app_build
endif

ifeq ($(PKG_BIN),)
$(error ### $(CURRENT_DIR): PKG_BIN is NULL, Please Check !!!)
endif

all: $(PKG_TARGET)
    @echo "build $(PKG_NAME) done"

test_app_build:
    rm -rf $(PKG_BIN) $(PKG_BUILD); \
    mkdir -p $(PKG_BIN);
    mkdir -p $(PKG_BUILD);
    pushd $(PKG_BUILD)/; \
        rm -rf CMakeCache.txt; \
        cmake $(CURRENT_DIR)/$(PKG_NAME)/ \
        -DCMAKE_C_COMPILER=$(RK_APP_CROSS)-gcc \
        -DCMAKE_CXX_COMPILER=$(RK_APP_CROSS)-g++ \
        -DCMAKE_INSTALL_PREFIX="$(CURRENT_DIR)/$(PKG_BIN)" \
        $(PKG_CONF_OPTS) ;\
        make -j$(RK_APP_JOBS) || exit -1; \
        make install; \
        popd;
    $(call MAROC_COPY_PKG_TO_APP_OUTPUT, $(RK_APP_OUTPUT), $(PKG_BIN))

clean:
    @rm -rf $(PKG_BIN) $(PKG_BUILD)

distclean: clean
```

- Test and build

```
./build.sh clean app
./build.sh app
```

16.8 如何查看多媒体驱动部分内存使用情况

```
# cat /proc/rk_dmabuf/dev
        DMABUF NAME             EXPORT                 SIZE:KiB AttachedDevices

b1acae40 (null)           system                    4 KiB 21f40000.rkvenc
b1acac00 (null)           system                   20 KiB 21f40000.rkvenc
b1aca780 (null)           system                   20 KiB 21f40000.rkvenc
b1aca540 (null)           system                  512 KiB 21f40000.rkvenc
b1aca180 (null)           system                    8 KiB
b199cf00 (null)           system                    8 KiB 21f40000.rkvenc
b199cc00 (null)           system                    4 KiB 21f40000.rkvenc
```

统计多媒体驱动总共使用内存大小：`awk 'NR>1 {sum += $4} END {print sum " KiB"}' /proc/rk_dmabuf/dev`

16.9 如何使用coredump功能

- 内核打开对应defconfig

```
CONFIG_ELF_CORE=y
CONFIG_CORE_DUMP_DEFAULT_ELF_HEADERS=y
```

- 板端设置coredump文件大小

```
ulimit -c unlimited
```

- 板端设置coredump文件位置

```
echo "/data/core-%p-%e" > /proc/sys/kernel/core_pattern
```

注：如果要把coredump文件直接写到NFS或VFAT文件系统，开机后需要执行如下代码， 否则在生成的coredump文件大小一直为0

```
# dump core file to /mnt/sdcard (which mount on vfat)
ulimit -c unlimited
echo "| /bin/coredump.sh %p %e" > /proc/sys/kernel/core_pattern
cat > /bin/coredump.sh << EOF
#!/bin/sh
exec cat -  > "/mnt/sdcard/core-\$1-\$2"
EOF
chmod a+x /bin/coredump.sh
```

- 查看coredump里的堆栈

把板端生成的coredump文件（例如：/data/core-279-rkipc_get_nn_up）拷贝到SDK根目录，然后运行如下命令：
例如RV1126B平台工具链是arm-rockchip1240-linux-gnueabihf-gdb

```
arm-rockchip1240-linux-gnueabihf-gdb ./output/out/app_out/bin/rkipc ./core-279-
rkipc_get_nn_up
# ...
(gdb) set solib-search-path output/out/media_out/lib/
Reading symbols from /home/rk/ipc-sdk/output/out/media_out/lib/librkaiq.so...done.
Reading symbols from /home/rk/ipc-sdk/output/out/media_out/lib/librockiva.so...(no debugging
symbols found)...done.
Reading symbols from /home/rk/ipc-sdk/output/out/media_out/lib/librockchip_mpp.so.0...(no
debugging symbols found)...done.
Reading symbols from /home/rk/ipc-sdk/output/out/media_out/lib/libaec_bf_process.so...(no
debugging symbols found)...done.
Reading symbols from /home/rk/ipc-sdk/output/out/media_out/lib/librkaudio_detect.so...(no
debugging symbols found)...done.
Reading symbols from /home/rk/ipc-sdk/output/out/media_out/lib/librga.so...(no debugging
symbols found)...done.
Reading symbols from /home/rk/ipc-sdk/output/out/media_out/lib/librknnmrt.so...(no debugging
symbols found)...done.
```

```
(gdb) bt
#0  0x00042080 in xx_list_pop ()
#1  0x0004229c in rkipc_xx_object_get ()
#2  0x0001a3c4 in rkipc_xx_update_osd ()
#3  0xa6c05390 in start_thread ()
   from /home/rk/ipc-sdk/tools/linux/toolchain/arm-rockchip1240-linux-gnueabihf/arm-
rockchip1240-linux-gnueabihf/sysroot/lib/libc.so.0
#4  0xa6bb8764 in clone ()
   from /home/rk/ipc-sdk/tools/linux/toolchain/arm-rockchip1240-linux-gnueabihf/arm-
rockchip1240-linux-gnueabihf/sysroot/lib/libc.so.0
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
```

16.10 内核驱动insmod说明

本章节适用于RV1126B。
可以参考`sysdrv/drv_ko/insmod_ko.sh`

```
#!/bin/sh
# if not install udevadm, ignore 'udevadm control'
udevadm control --stop-exec-queue

insmod rk_dvbm.ko

insmod video_rkcif.ko
insmod video_rkisp.ko
insmod phy-rockchip-csi2-dphy-hw.ko
insmod phy-rockchip-csi2-dphy.ko

insmod os04a10.ko
insmod sc4336.ko
insmod sc3336.ko
insmod sc530ai.ko

echo 1 > /sys/module/video_rkcif/parameters/clr_unready_dev
echo 1 > /sys/module/video_rkisp/parameters/clr_unready_dev

insmod rga3.ko

insmod kmpp.ko

insmod rknpu.ko

# $sensor_height is the height of the camera sensor (e.g. os04a0/sc4336/sc3336 and so on)
insmod rockit.ko

udevadm control --start-exec-queue
```

16.11 RV1126B平台相关库文件和驱动文件的信息

- 库文件

名称	用途	是否必须
ld-linux-armhf.so.3	toolchain标准库	是
libatomic.so*	toolchain标准库	是
libc.so.6	toolchain标准库	是
libdl.so.2	toolchain标准库	是
libgcc_s.so*	toolchain标准库	是
libm.so.6	toolchain标准库	是
libpthread.so.0	toolchain标准库	是
libstdc++.so*	toolchain标准库	C++程序必需
libBrokenLocale.so.1	修复区域设置（locale）兼容性问题	否
libanl.so.1	异步名称解析库（DNS查询等）	否
libasan.so*	AddressSanitizer（内存错误检测工具）的运行时库	否
libc_malloc_debug.so.0	C库的调试内存分配器	否
libcrypt.so.1	加密功能（如密码哈希）	否
libgomp.so*	GNU OpenMP（并行编程）支持库	否
libitm.so*	GNU事务内存（Transactional Memory）库	否
libmemusage.so	内存使用统计工具库	否
libnsl.so.1	网络服务库（NIS/NIS+等）	否
libnss_*	名称服务切换库（如DNS、用户/组数据库查询）	否
libpcprofile.so	性能分析工具库	否
libresolv.so.2	DNS解析库	否
librt.so.1	实时扩展库（如时钟、异步I/O）	否
libssp.so*	栈溢出保护（Stack Smashing Protection）库	否
libthread_db.so*	多线程调试支持库	否
libubsan.so*	Undefined Behavior Sanitizer（未定义行为检测）库	否
libutil.so.1	实用函数库（如终端操作）	否
librga.so	2D图形加速库	是
librkaiq.so	瑞芯微自动图像算法库	是
librockchip_mpp.so	通用媒体处理软件平台	是

名称	用途	是否必须
librockit.so	通用多媒体接口	是
librkaudio_common.so	音频算法基础库	否
librkaudio.so	音频算法基础库	否
librkaudio_detect.so	音频检测库	否
librkaudio_vqe.so	音频3A算法库	否
librockiva.so	NPU分析算法库	否
librknnmrt.so	NPU分析算法依赖库	否
librkfsmk.so	存储优化相关	否
librkmuxer.so	多媒体文件封装库	否
libdrm_rockchip.so	显示驱动框架（Rockchip）	否
libdrm.so	显示驱动框架	否

- 驱动文件

部分内核驱动文件	用途	是否必须
kmpp.ko	视频编码器驱动	是
phy-rockchip-csi2-dphy-hw.ko	mipi dphy rx物理驱动	是
phy-rockchip-csi2-dphy.ko	mipi dphy rx逻辑驱动	是
video_rkcif.ko	CIF驱动	是
video_rkisp.ko	图像信号处理驱动	是
video_rkvpss.ko	图像信号VPSS驱动	是
video_rkaiisp.ko	图像信号AI-ISP驱动	是
rockit.ko	多媒体框架驱动	是
rga3.ko	2D图像处理模块驱动	是
os04a10.ko	os04a10 sensor 驱动	否
sc3336.ko	sc3336 sensor 驱动	否
sc4336.ko	sc4336 sensor 驱动	否
sc530ai.ko	sc530ai sensor 驱动	否
rknpu.ko	NPU驱动	否

16.12 如何使用NFS文件系统

- 内核打开NFS配置

```
CONFIG_EXPORTFS_BLOCK_OPS=y
CONFIG_FILE_LOCKING=y
CONFIG_KEYS=y
CONFIG_NETWORK_FILESYSTEMS=y
CONFIG_ASSOCIATIVE_ARRAY=y
CONFIG_DNS_RESOLVER=y
# CONFIG_ECRYPT_FS is not set
# CONFIG_ENCRYPTED_KEYS is not set
CONFIG_FS_POSIX_ACL=y
CONFIG_GRACE_PERIOD=y
CONFIG_LOCKD=y
CONFIG_LOCKD_V4=y
CONFIG_MANDATORY_FILE_LOCKING=y
CONFIG_NFS_ACL_SUPPORT=y
CONFIG_NFS_COMMON=y
CONFIG_NFS_DISABLE_UDP_SUPPORT=y
CONFIG_NFS_FS=y
CONFIG_NFS_USE_KERNEL_DNS=y
# CONFIG_NFS_USE_LEGACY_DNS is not set
CONFIG_NFS_V2=y
CONFIG_NFS_V3=y
CONFIG_NFS_V3_ACL=y
CONFIG_NFS_V4=y
CONFIG_OID_REGISTRY=y
# CONFIG_PERSISTENT_KEYRINGS is not set
CONFIG_SUNRPC=y
CONFIG_SUNRPC_GSS=y
```

- 配置PC端NFS服务器

```
# Ubuntu 16.04 install NFS server
sudo apt-get install nfs-kernel-server
# Create /opt/rootfs
mkdir /opt/rootfs
# Enable nobody mount /opt/rootfs
chmod 0+w -R /opt/rootfs
# Add the directory to exports
sudo echo "/opt/rootfs *(rw,sync,root_squash)" >> /etc/exports
# Update NFS configure
sudo exportfs -r
# Test NFS server
sudo mount -t nfs localhost:/opt/rootfs /mnt
# Ubuntu 16.04 disable firewall
sudo ufw disable
```

- 单板端NFS挂载命令

```
# Get the IP address 192.168.1.123 of the PC
mount -t nfs -o nolock 192.168.1.123:/opt/rootfs /opt
```

16.13 如何增加新用户并设置登陆密码

以创建新用户（testNewUser）为例。

- 查看当前用户列表，获取新用户的UID和GID

文件格式：
Username:Password:User ID(UID):Group ID(GID):User ID Info (GECOS):Home directory:Login shell

修改单板上/etc/passwd，增加如下

```
testNewUser:x:1000:1000:testNewUser:/home:/bin/sh
```

- 在PC端用mkpasswd命令生成密码

```
sudo apt install whois
mkpasswd -m "md5" "test123" # --> $1$kprQOoLU$kOU2H.ecXkAwlZJ0oplu/.
```

- 修改单板上/etc/shadow，增加如下

```
testNewUser:$1$kprQOoLU$kOU2H.ecXkAwlZJ0oplu/.:0:0:99999:7:::
```

- 修改单板上/etc/group，增加如下

```
testNewUser:x:1000:
```

- 修改单板上/etc/inittab，修改如下

```
-::respawn:-/bin/sh
+#::respawn:-/bin/sh
::sysinit:/etc/init.d/rcS

# Put a getty on the serial port
-#ttyFIQ0::respawn:/sbin/getty -L ttyFIQ0 0 vt100 # GENERIC_SERIAL
+ttyFIQ0::respawn:/sbin/getty -L ttyFIQ0 0 vt100 # GENERIC_SERIAL
```

16.14 如何使用OTA升级

IPC SDK支持两种OTA升级方案，分别是Recovery和A/B系统。

16.14.1 如何使用Recovery

Recovery模式是在设备上多一个Recovery分区，该分区由kernel+resource+ramdisk组成，主要用于升级操作。u-boot会根据misc分区存放的字段来判断将要引导的系统是Normal系统还是Recovery系统。由于系统的独立性，所以Recovery模式能保证升级的完整性，即升级过程被中断，如异常掉电，升级仍然能继续执行。

本章主要介绍了通过userdata分区或SD卡执行Recovery本地升级程序的流程及技术细节，下面是使用Recovery的方法。

- 在`<SDK>/project/cfg/BoardConfig*.mk`中添加如下配置：

```
export RK_RECOVERY_KERNEL_DEFCONFIG_FRAGMENT="rv1126b-recovery.config"

#misc image
export RK_MISC=recovery-misc.img

# enable build recovery
export RK_ENABLE_RECOVERY=y

# select image to update
# export RK_OTA_RESOURCE="uboot.img boot.img rootfs.img userdata.img"

# Configure the compression format for ramdisk, default is xz, optional: gzip, xz
# Note: if xz, enable CONFIG_RD_XZ for kernel's defconfig
#       if gzip, enable CONFIG_RD_GZIP for kernel's defconfig
RK_RECOVERY_RAMDISK_COMPRESS_TYPE=xz
```

注：不开启RK_OTA_RESOURCE则默认打包uboot.img、boot.img、rootfs.img。

- 修改分区表

在BoardConfig.mk的RK_PARTITION_CMD_IN_ENV分区表中添加`misc`和`recovery`两个分区，分区大小和顺序可根据实际需求在合理范围内进行调整。

存储介质	misc分区大小
eMMC	64KB
spi nand 或slc nand	建议512KB
spi nor	64KB

- 编译Recovery

```
./build.sh recovery
```

- 打包Recovery镜像

```
./build.sh firmware
```

- 编译需要升级的固件

手动编译RK_OTA_RESOURCE包含的固件，若未开启则编译默认分区固件（固件编译方式见前文）。

- 打包OTA升级包

```
./build.sh ota
```

- 拷贝OTA升级包

将生成的OTA升级包（`<SDK>/output/image/update_ota.tar`）拷贝至SD卡根目录下。

注：若使用程序下载OTA升级包至板端userdata分区，则不需要这一步。

- 设备端插上SD卡

注：若使用程序下载OTA升级包至板端userdata分区，则不需要这一步。

- 设备端进入Recovery系统开始升级

```
reboot recovery
# 注：该命令需要使用SDK里的busybox或打上对应补丁
```

设备端（板端）输入上述代码进入Recovery系统，即可开始升级。

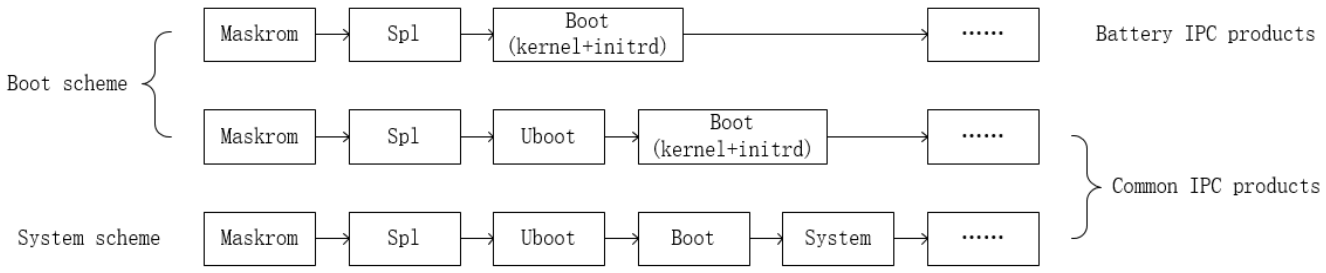
注：升级完成后设备将会重启进入Normal系统，失败则停留在Recovery系统并打印log。未检索到SD卡或升级包也将会重启进入Normal系统。

16.15 A/B系统使用方法

16.15.1 启动方案介绍

A/B系统支持两种启动方案：boot方案——boot分区兼根文件系统（kernel+initrd）、system方案——独立根文件系统（boot+system）。

目前，这两种启动方案，通用IPC类产品均支持；电池IPC类产品仅支持boot方案。



16.15.2 配置

- Uboot

通用IPC类产品和电池IPC类产品需要开启的uboot配置不同，对应defconfig开启配置如下所示：

```
# 通用IPC类产品
CONFIG_ANDROID_AB=y
CONFIG_SPL_MTD_WRITE=y
CONFIG_SPL_AB=y
CONFIG_EFI_PARTITION=y
CONFIG_SPL_EFI_PARTITION=y
CONFIG_AVB_LIBAVB=y
CONFIG_AVB_LIBAVB_AB=y
CONFIG_AVB_LIBAVB_USER=y
CONFIG_RK_AVB_LIBAVB_USER=y
```

```
# 电池IPC类产品
CONFIG_SPL_MTD_WRITE=y
CONFIG_SPL_AB=y
CONFIG_EFI_PARTITION=y
CONFIG_SPL_EFI_PARTITION=y
```

- Kernel

仅在**boot方案使用initramfs文件系统**时，才需要开启以下配置，其余情况不需要kernel开启额外配置。

```
# initramfs文件系统kernel配置
CONFIG_BLK_DEV_INITRD=y
```

- Root File System

当**使用SPI Nand存储介质**时，需要保证文件系统里有以下shell工具：

工具名称	**工具功能**
flash_erase	擦除指定mtd设备的块
nandwrite	写入指定的mtd设备
nanddump	转储nand mtd分区的内容
md5sum	计算和检查md5消息摘要
dd	转换和复制一个文件

- BoardConfig

对应BoardConfig板级配置中使能OTA，以编译OTA升级工具和打包相应镜像，用于A/B系统切换和升级。

```
# Enable OTA tool
export RK_ENABLE_OTA=y
# OTA package
export RK_OTA_RESOURCE="uboot.img boot.img system.img"
```

修改**分区表**`RK_PARTITION_CMD_IN_ENV`、**文件系统类型**`RK_PARTITION_FS_TYPE_CFG`这两个参数。

以下提供了boot、system两种方案的例子，仅供参考，实际参数可根据需求自行修改。

boot方案：

```
# 分区表（添加misc分区，去掉rootfs分区，boot分区改为boot_a、boot_b分区）
export
RK_PARTITION_CMD_IN_ENV="64K(env),256K@64K(idblock),256K(uboot),64K(misc),384K(meta),6M(boot_a),6M(boot_b),1M(userdata)"

# 文件系统类型（电池IPC类产品默认使用erofs，通用IPC类产品默认使用initramfs）
export RK_PARTITION_FS_TYPE_CFG=boot_a@IGNORE@erofs,userdata@/userdata@jffs2
```

system方案：

```
# 分区表（添加misc分区，去掉rootfs分区，uboot分区改为uboot_a、uboot_b分区，boot分区改为boot_a、boot_b分区，添加system_a、system_b分区）
export
RK_PARTITION_CMD_IN_ENV="256K(env),256K@256K(idblock),256K(uboot_a),256K(uboot_b),256K(misc),4M(boot_a),4M(boot_b),16M(system_a),16M(system_b),32M(oem),32M(userdata),-(media)"

# 文件系统类型
export
RK_PARTITION_FS_TYPE_CFG=system_a@IGNORE@ubifs,oem@/oem@ubifs,userdata@/userdata@ubifs
```

16.15.3 OTA升级工具

AB系统默认使用RETRY_BOOT模式启动。若需要使用SUCCESSFUL_BOOT模式，则修改配置如下所示：

注：两种模式的区别，可见U-Boot文档第6.3.2章节：Rockchip_Developer_Guide_UBoot_Nextdev_CN.pdf

```
diff --git a/tools/board/rk_ota/Makefile b/tools/board/rk_ota/Makefile
index 8bf67f5..21cf5e0 100644
--- a/tools/board/rk_ota/Makefile
+++ b/tools/board/rk_ota/Makefile
@@ -10,7 +10,7 @@ CURRENT_DIR := $(shell pwd)
 PKG_BIN := out
 TARGET_BIN := rk_ota
 PKG_SRCS := src/mtdutils.c src/md5sum.cpp src/bootloader.cpp src/main.cpp
-CONFIG_IS_SUCCESSFUL_BOOT := n
+CONFIG_IS_SUCCESSFUL_BOOT := y

 PKG_DEP_OPENSSL := $(CURRENT_DIR)/../toolkits/openssl
 PKG_DEP_ZLIB := $(CURRENT_DIR)/../toolkits/zlib
```

BoardConfig中添加配置`RK_ENABLE_OTA`后，SDK会将OTA升级工具编译到固件文件系统的`/usr/bin`下，指令名为`rk_ota`，同时打包相应的镜像至`<SDK>/output/image/update_ota.tar`中。

```
# rk_ota --help
[I/]RECOVERY *** rk_ota: Version V1.0.0 ***.
[I/]RECOVERY --misc=now                          Linux A/B mode: Setting the current
partition to bootable.
[I/]RECOVERY --misc=other                        Linux A/B mode: Setting another partition
to bootable.
[I/]RECOVERY --misc=update                       Linux A/B mode: Setting the partition to
be upgraded.
[I/]RECOVERY --misc=display                      Display misc info.
[I/]RECOVERY --tar_path=<path>                   Set upgrade firmware path.
[I/]RECOVERY --save_dir=<path>                   Set the path for saving the image.
[I/]RECOVERY --partition=<uboot/boot/system/all>  Set the partition to be upgraded.('all'
means 'uboot', 'boot' and 'system' are included.)
[I/]RECOVERY --extra_part=<name>                 Set the extra partition to be upgraded.
[I/]RECOVERY --reboot                            Restart the machine at the end of the
program.
```

16.15.4 A/B系统切换

切换A/B系统的指令为`rk_ota --misc=other`，运行情况如下：

```
# rk_ota --misc=other --reboot
[I/]RECOVERY *** rk_ota: Version V1.0.0 ***.
[I/]RECOVERY Now is MTD.
A/B-slot: B, successful: 0, tries-remain: 6
[I/]RECOVERY Now is MTD.
```

`--reboot`：重启设备。

进入另一个系统，若该系统成功启动，需要运行指令`rk_ota --misc=now`，将该系统设置为“最后启动的系统”。

```
# rk_ota --misc=now
[I/]RECOVERY *** rk_ota: Version V1.0.0 ***.
[I/]RECOVERY Now is MTD.
A/B-slot: A, successful: 0, tries-remain: 6
info.mafic is 0
info.mafic is 41
info.mafic is 42
info.mafic is 30
[I/]RECOVERY Now is MTD.
```

16.15.5 A/B系统升级

升级A/B系统的相关选项有：

`--misc=update`：选择“升级”模式；

`--tar_path=<path>`：设置OTA升级包的路径（存放镜像的tar包）；

`--save_dir=<path>`：（可选）设置解包文件夹，未设置则默认为`/mnt/sdcard/rk_update/`；

`--partition=<uboot/boot/system/all>`：（可选）设置需要升级的分区，“all”表示“uboot”、“boot”和“system”都升级，未设置则默认为“all”。（当源文件不存在时，会跳过相应分区的升级。）

`--extra_part=<name>`：（可选）设置一个需要升级的自定义分区，未设置则默认忽略该选项。

注：目前只能升级uboot、boot、system分区和一个自定义分区；

示例：

```
# rk_ota --misc=update --tar_path=/mnt/sdcard/update_ota.tar --save_dir=/mnt/sdcard/ --
partition=all --reboot
# 写boot
[I/]RECOVERY *** rk_ota: Version V1.0.0 ***.
[I/]RECOVERY tar path = /mnt/sdcard/update_ota.tar
[I/]RECOVERY save path = /mnt/sdcard/
[I/]RECOVERY Now is MTD.
A/B-slot: A, successful: 0, tries-remain: 5
[I/]RECOVERY Now is MTD.
[I/]RECOVERY mtd_write src=/mnt/sdcard//boot.img dest=/dev/block/by-name/boot_b.
Erasing 128 Kibyte @ 3e0000 -- 100 % complete
Writing data to block 0 at offset 0x0
Writing data to block 1 at offset 0x20000
...
2846720+0 records in
2846720+0 records out

[I/]RECOVERY Now is MTD.
[I/]RECOVERY [checkdata_mtd:30] offset [0] checksize [2846720]
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 131072, page size 2048, OOB size 128
Dumping data starting at 0x00000000 and ending at 0x002b7000...

read new md5: [6c60afda3ab31a49acd6d5d65e86a2e6]
new md5:6c60afda3ab31a49acd6d5d65e86a2e6
[I/]RECOVERY MD5Check is ok of /dev/block/by-name/boot_b
new md5:6c60afda3ab31a49acd6d5d65e86a2e6
[I/]RECOVERY MD5Check is ok for /mnt/sdcard//boot.img
[I/]RECOVERY check /dev/block/by-name/boot_b ok.
[I/]RECOVERY Write /mnt/sdcard//boot.img into /dev/block/by-name/boot_b successfully.

# 写system
[I/]RECOVERY Now is MTD.
[I/]RECOVERY mtd_write src=/mnt/sdcard//system.img dest=/dev/block/by-name/system_b.
Erasing 128 Kibyte @ fe0000 -- 100 % complete
Writing data to block 0 at offset 0x0
Writing data to block 1 at offset 0x20000
...
8126464+0 records in
8126464+0 records out
```

```
Writing data to block 62 at offset 0x7c0000
[I/]RECOVERY Now is MTD.
[I/]RECOVERY [checkdata_mtd:30] offset [0] checksize [8126464]
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 131072, page size 2048, OOB size 128
Dumping data starting at 0x00000000 and ending at 0x007c0000...
ECC: 1 corrected bitflip(s) at offset 0x004be000

read new md5: [9f9fad6f08cbdd210488ff544e14af25]
new md5:9f9fad6f08cbdd210488ff544e14af25
[I/]RECOVERY MD5Check is ok of /dev/block/by-name/system_b
new md5:9f9fad6f08cbdd210488ff544e14af25
[I/]RECOVERY MD5Check is ok for /mnt/sdcard//system.img
[I/]RECOVERY check /dev/block/by-name/system_b ok.
[I/]RECOVERY Write /mnt/sdcard//system.img into /dev/block/by-name/system_b successfully.

[I/]RECOVERY Now is MTD.
[I/]RECOVERY Now is MTD.
mtd: successfully wrote block at 395a800000000
mtd: successfully wrote block at 395a800020000
reboot
```

16.16 获取摄像头支持列表

可以在Redmine上获取https://redmine.rock-chips.com/documents/53

16.17 获取Flash支持列表

可以在Redmine上获取https://redmine.rock-chips.com/documents/46

16.18 获取eMMC支持列表

可以在Redmine上获取https://redmine.rock-chips.com/documents/50

16.19 获取DDR支持列表

可以在Redmine上获取https://redmine.rock-chips.com/documents/49

16.20 压力测试使用方法

压力测试需要开启如下配置：

```
# enable rockchip test
export RK_ENABLE_ROCKCHIP_TEST=y
```

目前已支持的压力测试列表：

```
# ./rockchip_test/rockchip_test.sh
*****************************************************
***                                               ***
***        ********************                   ***
***       *ROCKCHIPS TEST TOOLS*                  ***
***        *                   *                  ***
***        ********************                   ***
***                                               ***
*****************************************************
*****************************************************
ddr test :            1 (memtester & stressapptest)
cpufreq test:         2 (cpufreq stresstest)
flash stress test:    3
auto reboot test:     4
*****************************************************
please input your test moudle:
```

16.20.1 memtester test

- 打开压力测试列表

```
sh rockchip_test/rockchip_test.sh
```

- 开始测试（压力测试列表内选择测试项对应序号1）
- 再选择`memtester test`对应序号（默认使用空闲内存的一半容量进行测试）

16.20.2 stressapptest

- 打开压力测试列表

```
sh rockchip_test/rockchip_test.sh
```

- 开始测试（压力测试列表内选择测试项对应序号1）
- 再选择`stressapptest`对应序号（默认使用空闲内存的一半容量测试48小时）

16.20.3 cpufreq test

- 打开压力测试列表

```
sh rockchip_test/rockchip_test.sh
```

- 开始测试（压力测试列表内选择测试项对应序号2）
- 再选择`cpu freq stress test`或`cpu freq test:(with out stress test)`对应序号（前者默认使用空闲内存的一半容量测试24小时；后者默认一秒变频一次）

16.20.4 flash stress test

- 打开压力测试列表

```
sh rockchip_test/rockchip_test.sh
```

- 开始测试（压力测试列表内选择测试项对应序号3）

16.20.5 reboot test

- 打开压力测试列表

```
sh rockchip_test/rockchip_test.sh
```

- 开始测试（压力测试列表内选择测试项对应序号4）
- 退出测试（默认重启10000次后停止测试；若需要提前退出，可输入以下指令）

```
echo off > /data/cfg/rockchip_test/reboot_cnt
```

16.21 如何使用rndis功能

确认内核dts配置

```
&usb2phy {
    status = "okay";
};

&usb2phy_otg {
    status = "okay";
};

&usb3phy {
    status = "okay";
};
```

```
&usb_drd_dwc3 {
    status = "okay";
};
```

确认内核defconfig配置`rv1126b-rndis.config`。

确认usb的vbus 有供电3.3V。

修改对应的BoardConfig.mk，增加`export RK_ENABLE_RNDIS=y`，然后按如下命令编译：

```
./build.sh sysdrv
./build.sh firmware
```

烧录固件后，开机运行命令：`rndis.sh`
详细log如下：

```
# rndis.sh
serialnumber is  f5bc7ed083b85dcf
config usb0 IP...
# ifconfig
usb0      Link encap:Ethernet  HWaddr C2:44:18:6D:9A:05
          inet addr:192.168.1.100  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:133 errors:0 dropped:69 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:11360 (11.0 KiB)  TX bytes:0 (0.0 B)
```

注：如果需要修改默认IP地址，可以修改SDK文件"sysdrv/tools/board/rndis/rndis.sh"，然后重新编译烧录固件。

16.22 如何增加非root用户登陆

这里介绍使用busybox时，增加普通用户登陆终端的方法。

1. 修改/etc/passwd

```
test123:x:1010:1011:test123:/opt:/bin/sh
```

x表示用户设有密码
1010表示用户id
1011表示用户组id
/opt表示test123用户的HOME目录
注：/etc/passwd详细说明参考https://www.man7.org/linux/man-pages/man5/passwd.5.html

2. 修改/etc/shadow

```
test123:$1$x.YlInZQ$N.kbNTIEOkBjmlfftSVFs0:10933:0:99999:7:::
```

`$1$x.YlInZQ$N.kbNTIEOkBjmlfftSVFs0:10933`是用命令`mkpasswd -m "md5" "rockchip123"`生成。
注：/etc/shadow详细说明参考https://www.man7.org/linux/man-pages/man5/shadow.5.html

3. 修改/etc/group

```
test123:x:1011:test123
```

注：/etc/group详细说明参考https://www.man7.org/linux/man-pages/man5/group.5.html

4. 修改/etc/inittab

```
#::respawn:-/bin/sh
ttyFIQ0::respawn:/sbin/getty -L ttyFIQ0 0 vt100 # GENERIC_SERIAL
```

16.23 如何添加新的Camera sensor配置

详细调试请查阅文档：

```
<SDK>/docs/zh/media/Rockchip_Trouble_Shooting_Linux5.10_Camera_CN.pdf
```

16.24 如何添加reboot命令进U-Boot终端

以RV1126B平台为例。

目录：sysdrv/source/kernel

```
diff --git a/arch/arm64/boot/dts/rv1126b.dtsi b/arch/arm64/boot/dts/rv1126b.dtsi
index 4564909db8b2..99228b4b80cf 100644
--- a/arch/arm64/boot/dts/rv1126b.dtsi
+++ b/arch/arm64/boot/dts/rv1126b.dtsi
@@ -359,6 +359,7 @@ reboot_mode: reboot-mode {
                        mode-ums = <BOOT_UMS>;
                        mode-panic = <BOOT_PANIC>;
                        mode-watchdog = <BOOT_WATCHDOG>;
+                       mode-uboot = <BOOT_TO_UBOOT>;
                };

                rgb: rgb {
diff --git a/include/dt-bindings/soc/rockchip,boot-mode.h b/include/dt-
bindings/soc/rockchip,boot-mode.h
index 1436e1d32619..cc5a421aef26 100644
--- a/include/dt-bindings/soc/rockchip,boot-mode.h
+++ b/include/dt-bindings/soc/rockchip,boot-mode.h
@@ -21,4 +21,6 @@
 /* enter usb mass storage mode */
 #define BOOT_UMS                (REBOOT_FLAG + 12)

+#define BOOT_TO_UBOOT           (REBOOT_FLAG + 14)
```

```
+
 #endif
```

目录：sysdrv/source/uboot/u-boot

```
diff --git a/arch/arm/include/asm/arch-rockchip/boot_mode.h b/arch/arm/include/asm/arch-
rockchip/boot_mode.h
index bc1395ee2c..d68099a94e 100644
--- a/arch/arm/include/asm/arch-rockchip/boot_mode.h
+++ b/arch/arm/include/asm/arch-rockchip/boot_mode.h
@@ -26,6 +26,8 @@
 /* enter bootrom download mode */
 #define BOOT_BROM_DOWNLOAD     0xEF08A53C

+#define BOOT_TO_UBOOT          (REBOOT_FLAG + 14)
+
 #ifndef __ASSEMBLY__
 int setup_boot_mode(void);
 #endif
diff --git a/arch/arm/mach-rockchip/boot_mode.c b/arch/arm/mach-rockchip/boot_mode.c
index 61f0e85c1c..0d555314e2 100644
--- a/arch/arm/mach-rockchip/boot_mode.c
+++ b/arch/arm/mach-rockchip/boot_mode.c
@@ -189,6 +189,11 @@ int rockchip_get_boot_mode(void)
                        boot_mode[PL] = BOOT_MODE_UMS;
                        clear_boot_reg = 1;
                        break;
+               case BOOT_TO_UBOOT:
+                       printf("boot mode: uboot\n");
+                       boot_mode[PL] = BOOT_MODE_UBOOT_TERMINAL;
+                       clear_boot_reg = 1;
+                       break;
                case BOOT_CHARGING:
                        printf("boot mode: charging\n");
                        boot_mode[PL] = BOOT_MODE_CHARGING;
@@ -227,6 +232,8 @@ int setup_boot_mode(void)
 {
        char env_preboot[256] = {0};

+       env_set("cli", NULL); /* removed by default */
+
        switch (rockchip_get_boot_mode()) {
        case BOOT_MODE_BOOTLOADER:
                printf("enter fastboot!\n");
@@ -259,6 +266,10 @@ int setup_boot_mode(void)
                printf("enter charging!\n");
                env_set("preboot", "setenv preboot; charge");
                break;
+       case BOOT_MODE_UBOOT_TERMINAL:
+               printf("enter uboot!\n");
+               env_set("cli", "yes");
+               break;
        }
```

```
        return 0;
diff --git a/common/autoboot.c b/common/autoboot.c
index c64d566d1c..9a6679aca9 100644
--- a/common/autoboot.c
+++ b/common/autoboot.c
@@ -220,7 +220,7 @@ static int __abortboot(int bootdelay)
 #endif

 #ifdef CONFIG_ARCH_ROCKCHIP
-       if (!IS_ENABLED(CONFIG_CONSOLE_DISABLE_CLI) && ctrlc()) {       /* we press ctrl+c ?
*/
+       if ((!IS_ENABLED(CONFIG_CONSOLE_DISABLE_CLI) && ctrlc()) || env_get("cli")) {   /*
we press ctrl+c ? */
 #else
        /*
         * Check if key already pressed
diff --git a/include/boot_rkimg.h b/include/boot_rkimg.h
index cb5781850e..d8ef3e6127 100644
--- a/include/boot_rkimg.h
+++ b/include/boot_rkimg.h
@@ -19,6 +19,7 @@ enum _boot_mode {
        BOOT_MODE_PANIC,
        BOOT_MODE_WATCHDOG,
        BOOT_MODE_DFU,
+       BOOT_MODE_UBOOT_TERMINAL,
        BOOT_MODE_UNDEFINE,
 };
```

注：BOOT_TO_UBOOT的值在内核和U-Boot要一样。

重新编译内核和U-Boot，`./build.sh kernel; ./build.sh uboot`，然后更新boot.img、uboot.img。

开机后输入`reboot uboot`，单板重启后会进U-Boot的终端。

16.25 如何通过指令修改GPIO寄存器配置

GPIO全部寄存器与配置指令可见文档`<SDK>/docs/zh/bsp/Rockchip_XXXX_User_Manual_GPIO.pdf`，指令在U-Boot终端和根文件系统中有所不同，下面将分别阐述。

注：使用引脚复用功能时，需提前确认该引脚是否已被某项功能占用。

Name	Config	Description	Register	Position	Value	IO Command
GPIO0A0	IOmux	RTC_32K_OUT	0x201B0000	[0,3]	1	io -4 0x201B0000 0x000F0001
		CLK_32K	0x201B0000	[0,3]	2	io -4 0x201B0000 0x000F0002
		CLK_24M_OUT	0x201B0000	[0,3]	3	io -4 0x201B0000 0x000F0003
		FSPI1_CLK	0x201B0000	[0,3]	4	io -4 0x201B0000 0x000F0004
		GPIO0_A0	0x201B0000	[0,3]	0	io -4 0x201B0000 0x000F0000
	Pull	z	0x201B0200	[0,1]	0	io -4 0x201B0200 0x00030000
		up	0x201B0200	[0,1]	1	io -4 0x201B0200 0x00030001
		down	0x201B0200	[0,1]	2	io -4 0x201B0200 0x00030002
	I/O Mode	input	0x20520008	[0,0]	0	io -4 0x20520008 0x00010000
		output	0x20520008	[0,0]	1	io -4 0x20520008 0x00010001
	Output	low	0x20520000	[0,0]	0	io -4 0x20520000 0x00010000
		high	0x20520000	[0,0]	1	io -4 0x20520000 0x00010001
	Input Ctrl	disable	0x201B0300	[0,0]	0	io -4 0x201B0300 0x00010000
		enable	0x201B0300	[0,0]	1	io -4 0x201B0300 0x00010001
	Schmitt	disable	0x201B0400	[0,0]	0	io -4 0x201B0400 0x00010000
		enable	0x201B0400	[0,0]	1	io -4 0x201B0400 0x00010001
	Drive Strength	level0	0x201B0100	[0,5]	1	io -4 0x201B0100 0x003F0001
		level1	0x201B0100	[0,5]	3	io -4 0x201B0100 0x003F0003
		level2	0x201B0100	[0,5]	7	io -4 0x201B0100 0x003F0007
		level3	0x201B0100	[0,5]	15	io -4 0x201B0100 0x003F000F
		level4	0x201B0100	[0,5]	31	io -4 0x201B0100 0x003F001F
		level5	0x201B0100	[0,5]	63	io -4 0x201B0100 0x003F003F
	ExtPort	read gpio data in input mode	0x20520070	[0,0]		io -4 0x20520070

16.25.1 U-Boot终端

U-Boot终端使用`md`指令显示寄存器值，使用`mw`指令写入寄存器值。

- 以设置GPIO0A4输出高电平为例：

```
# 复用模式：GPIO（IOmux->GPIO0_A4）
=> mw.l 0x201A0004 0x000F0000
# I/O模式：输出（GPIO0A4->I/O Mode->output）
=> mw.l 0x20600008 0x00100010
# 输出电平：高电平（GPIO0A4->Output->high）
=> mw.l 0x20600000 0x00100010
```

- 以读取GPIO0A4输入数据为例：

```
# 复用模式：GPIO（IOmux->GPIO0_A4）
=> mw.l 0x201A0004 0x000F0000
# I/O模式：输入（GPIO0A4->I/O Mode->input）
=> mw.l 0x20600008 0x00100000
# 读取数据：16字节（GPIO0A4->ExtPort）（4个long，即4*4=16字节）
=> md.l 0x20600070
```

16.25.2 根文件系统

根文件系统使用`io`指令读写寄存器值。

- 以设置GPIO0A4输出高电平为例：

```
# 复用模式：GPIO（IOmux->GPIO0_A4）
io -4 0x201A0004 0x000F0000
# I/O模式：输出（GPIO0A4->I/O Mode->output）
io -4 0x20600008 0x00100010
# 输出电平：高电平（GPIO0A4->Output->high）
io -4 0x20600000 0x00100010
```

- 以读取GPIO0A4输入数据为例：

```
# 复用模式：GPIO（GPIO0A4->IOmux->GPIO0_A4）
io -4 0x201B0000 0x000F0000
# I/O模式：输入（GPIO0A4->I/O Mode->input）
io -4 0x20600008 0x00100000
# 读取数据
io -4 0x20600070
```

16.26 如何修改串口波特率

串口波特率需要同时修改ddr和内核dts两个地方。

16.26.1 ddr bin文件

1. 获取loader ini文件

```
./build.sh uboot 2>&1 |grep "pack loader.*Input"
pack loader(SPL) okay! Input:
/opt/rv1126b/sysdrv/source/uboot/rkbin/rkboot/RV1126BMINIALL.ini
```

2. 获取ddr bin文件的路径

从上述获取到的loader ini文件中，可以得到ddr bin文件的路径。

这里以`RV1126BMINIALL.ini`为例：

```
# cat /opt/rv1126b/sysdrv/source/uboot/rkbin/rkboot/RV1126BMINIALL.ini
[CHIP_NAME]
NAME=RV110F
[VERSION]
MAJOR=1
MINOR=1
[CODE471_OPTION]
NUM=1
```

```
Path1=bin/rv11/rv1126b_ddr_v0.03.bin
Sleep=1
[CODE472_OPTION]
NUM=1
Path1=bin/rv11/rv1126b_usbplug_v0.02.bin
[LOADER_OPTION]
NUM=2
LOADER1=FlashData
LOADER2=FlashBoot
FlashData=bin/rv11/rv1126b_ddr_v0.03.bin
FlashBoot=bin/rv11/rv1126b_spl_v0.01.bin
[LOADER2_PARAM]
LOAD_ADDR=0x43f00000
FLAG=0x0
[OUTPUT]
PATH=rv1126b_download_v0.03.001.bin
IDB_PATH=rv1126b_idblock_v0.03.001.img
[SYSTEM]
NEWIDB=true
[FLAG]
471_RC4_OFF=true
RC4_OFF=true
CREATE_IDB=true
```

“FlashData”即为ddr bin文件的配置，这里得ddr bin文件为`bin/rv11/rv1126b_ddr_v0.03.bin`，具体路径为`<SDK>/sysdrv/source/uboot/rkbin/bin/rv11/rv1126b_ddr_v0.03.bin`。

3. 修改ddr bin文件的串口波特率

使用ddr bin工具修改串口波特率，工具获取方式如下：

Windows工具：Redmine下载，链接为https://redmine.rock-chips.com/documents/49

Linux工具：`<SDK>/sysdrv/source/uboot/rkbin/tools/ddrbin_tool`

下面是Windows工具的操作步骤：



修改完成后重编uboot即可。

16.26.2 内核dts

示例板级配置使用的dts为`rv1126b-evb1-v10.dts`，以该dts为例：

```
diff --git a/arch/arm64/boot/dts/rv1126b-evb1-v10.dts b/arch/arm64/boot/dts/rv1126b-evb1-
v10.dts
index dc6152543b20..bc8f062edc99 100644
--- a/arch/arm64/boot/dts/rv1126b-evb1-v10.dts
+++ b/arch/arm64/boot/dts/rv1126b-evb1-v10.dts
@@ -14,6 +14,10 @@ / {
+&fiq_debugger {
+       rockchip,baudrate = <1500000>;
+};
```

修改后重编内核即可。

16.27 启用 sysstat 测试套件

sysstat是一个Linux开源工具集，包括sar、pidstat、iostat、mpstat、sadf、sadc。其中sar是目前Linux上最为全面的系统性能分析工具之一。sar工具将对系统当前的状态进行取样，然后通过计算数据和比例来表达系统的当前运行状态。它的特点是可以连续对系统取样，获得大量的取样数据；取样数据和分析的结果都可以存入文件，所需的负载很小。
可以从14个大方面对系统的活动进行报告，包括文件的读写情况、系统调用的使用情况、串口、CPU效率、内存使用状况、进程活动及IPC有关的活动等。

建议在调试时将此套件打开，利用sar工具可以查看设备数据情况：

- 查看CPU使用率
- 查看平均负载
- 查看内存使用状况
- 查看I/O设备情况
- 查看网络情况
- 查看页面交换发生状况
- ...

如需打开sysstat工具，内核defcofnig配置如下：

```
CONFIG_TASKSTATS=y
CONFIG_SCHED_INFO=y
CONFIG_TASK_DELAY_ACCT=y
CONFIG_TASK_IO_ACCOUNTING=y
CONFIG_TASK_XACCT=y
CONFIG_DEBUG_USER=y
```

- BoardConfig 中添加：

```
export RK_ENABLE_SYSSTAT=y
```

以上即可启用 sysstat 工具，其中 sar 工具用于统计系统性能，

常用指令：

```
CPU使用情况：可用 sar -u 1来查看
内存使用情况：可用 sar -r 1来查看
I/O使用情况：可用 sar -d 1来查看
```

其他详细命令请自行通过浏览器查询。

16.28 如何使用DM-Verity和DM-Crypt功能

SDK目前只在常电IPC的eMMC上验证过。

在BoardConfig.mk里配置如下：

内核defconfig配置加上： `rv1126b-device-mapper.config`
单独使能DM-Verity功能： `RK_ENABLE_DM_V`
单独使能DM-Crypt功能： `RK_ENABLE_DM_CRYPT`，配置`RK_MISC`，增加`misc`分区，使能`CONFIG_SYSDRV_ENABLE_RK_TEE_USER`，如果是spi nand或者spi nor需要增加大小为1M的`security`分区
同时使能DM-Verity和DM-Crypt功能： `RK_ENABLE_DM_V_DM_CRYPT`

以下以同时使能DM-Verity和DM-Crypt功能为例：

SDK默认以rootfs为DM-Crypt分区和DM-Verity分区。

```
diff --git a/BoardConfig-SPI_NAND-RK801-RV1126B_EVB1_V10-IPC.mk b/BoardConfig-SPI_NAND-
RK801-RV1126B_EVB1_V10-IPC.mk
index ac51b19..4c622a1 100644
--- a/BoardConfig-SPI_NAND-RK801-RV1126B_EVB1_V10-IPC.mk
+++ b/BoardConfig-SPI_NAND-RK801-RV1126B_EVB1_V10-IPC.mk
@@ -48,7 +48,11 @@ export RK_BOOTARGS_CMA_SIZE="48M"
 #       <partdef> := <size>[@<offset>](part-name)
 # Note:
 #   If the first partition offset is not 0x0, it must be added. Otherwise, it needn't
adding.
-export
RK_PARTITION_CMD_IN_ENV="256K(env),1M@256K(idblock),1M(uboot),8M(boot),24M(rootfs),48M(oem),
40M(userdata)"
+export
RK_PARTITION_CMD_IN_ENV="256K(env),1M@256K(idblock),256K(misc),1M(security),1M(uboot),8M(boo
t),24M(rootfs),48M(oem),40M(userdata)"
+
+export RK_MISC=blank-misc.img
+
+export RK_ENABLE_DM_V_DM_CRYPT=y
+
+export CONFIG_SYSDRV_ENABLE_RK_TEE_USER=y

 # Kernel defconfig fragment
-export RK_KERNEL_DEFCONFIG_FRAGMENT=rv1126b-evb.config
+export RK_KERNEL_DEFCONFIG_FRAGMENT="rv1126b-evb.config rv1126b-device-mapper.config"

 # config partition's filesystem type (squashfs is readonly)
 # emmc:    squashfs/ext4
 # nand:    squashfs/ubifs
 # spi nor: squashfs/jffs2
 # RK_PARTITION_FS_TYPE_CFG format:
 #     AAAA:/BBBB/CCCC@ext4
 #         AAAA ----------> partition name
 #         /BBBB/CCCC ----> partition mount point
 #         ext4 ----------> partition filesystem type
-export RK_PARTITION_FS_TYPE_CFG=rootfs@IGNORE@ext4,userdata@/userdata@ext4,oem@/oem@ext4
+export
RK_PARTITION_FS_TYPE_CFG=rootfs@IGNORE@squashfs,userdata@/userdata@ext4,oem@/oem@ext4
```

注：DM-Verity的镜像只支持只读文件系统格式（例如：squashfs）。

```
./build.sh clean      # 清除编译缓存

./build.sh all        # 编译

./build.sh dm-key-gen # 如果使能DM-Crypt，需要生成dm-crypt的key，运行一次即可
                      # key保存的目录在SDK根目录下dm-key
                      # 编译DM-Crypt需要sudo权限，`echo "XXX" > dm-key/root_passwd`，XXX为sudo
密码

./build.sh firmware   # 生成原始固件

./build.sh dm         # 编译rootfs 支持DM功能
```

DM-Verity和DM-Crypt的镜像目录在：/output/image-dm

DM-Crypt的key保存在misc分区，实际应用时，需要将key写到eMMC的RPMB分区（写RPMB参考/project/scripts/init.in）。

16.29 如何使用UVC预览

SDK提供UVC功能，可使用UVC功能对摄像头进行预览调焦。

- 打开UVC编译

以project/cfg/BoardConfig_IPC/BoardConfig-SPI_NAND-RK801-RV1126B_EVB1_V10-IPC.mk为例：

1. RK_APP_TYPE增加UVC_TINY重新编译
2. 使能`RK_ENABLE_RNDIS`，打开内核驱动

```
 # app config
+export RK_APP_TYPE="UVC_TINY"

+ # enable rndis
+ export RK_ENABLE_RNDIS=y
```

编译命令

```
./build.sh all
```

- PC端预览

OTG连接开发板与PC，PC端使用UVC预览工具开USB设备进行预览。

16.30 安全启动相关代码以及文档说明

相关加解密代码路径：`sysdrv/tools/board/security`

相关安全文档路径：`docs/zh/security`

U-Boot签名文档路径（FIT章节）：`docs/zh/bsp/Rockchip_Developer_Guide_UBoot_Nextdev_CN.pdf`

16.30.1 U-Boot配置

U-Boot的defconfig需要打开如下配置：

```
# 必选项
CONFIG_FIT_SIGNATURE=y
CONFIG_SPL_FIT_SIGNATURE=y
CONFIG_SPL_KEYLAD=y
CONFIG_SPL_KEYLAD_SUPPORT=y
CONFIG_ROCKCHIP_CIPHER=y
CONFIG_SPL_ROCKCHIP_CIPHER=y
CONFIG_SPL_ROCKCHIP_SECURE_OTP=y
CONFIG_RSA_N_SIZE=0x200
CONFIG_RSA_E_SIZE=0x10
CONFIG_RSA_C_SIZE=0x20
# 修改项
CONFIG_SPL_FIT_IMAGE_KB=1024     # 开启安全启动之后spl固件会超大小

# 可选项（U-Boot读写otp）
CONFIG_ROCKCHIP_OTP=y    # 使能U-Boot阶段读写非安全otp
CONFIG_MISC=y            # 相关读写函数的编译配置
# 可选项（防回滚）
CONFIG_OPTEE_CLIENT=y                # 使能U-Boot阶段读写安全otp
CONFIG_OPTEE_V2=y                    # 使能U-Boot阶段读写安全otp
CONFIG_FIT_ROLLBACK_PROTECT=y        # boot.img防回滚
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y    # uboot.img防回滚
```

16.30.2 rkbin配置

默认RKTRUST.ini中需要开启BL32_OPTION

```
[BL32_OPTION]
SEC=1
```

注意：U-Boot defconfig中如果没有配置RKTRUST.ini，ini一般默认为<芯片型号>TRUST.ini，也可以在U-Boot编译日志中确认

16.30.3 BoardConfig配置

```
# 使能安全启动配置
+export RK_ENABLE_SECURE_BOOT=y

# 配置uboot分区rollback index为2
# 配置boot分区rollback index为3
# 配置rootfs分区rollback index为2
+export RK_ENABLE_SECURE_BOOT_ROLLBACK_CFG=uboot@2,boot@3,recovery@2
```

注意：

1. boot和recovery需要开启TEE支持如何开启OP-TEE功能
2. boot,recovery,system,rootfs分区的rollback index是共用的

16.30.4 生成Key

SDK工程下执行如下命令可以生成签名用的RSA密钥对。通常情况下只需要生成一次，此后都用**这对密钥对（dev.key、dev.pubkey）和自签名证书（dev.crt）**来签名和验证固件，请妥善保管。

```
# 先生成key
./build.sh secure-boot-key-gen
```

`ls secure-boot-key/`查看结果：

```
dev.crt dev.key dev.pubkey
```

```
# 编译完整sdk
./build.sh lunch    # 选择上面配置好的BoardConfig.mk
./build.sh all
```

注：上述的"keys"、"dev.key"、"dev.crt" 、"dev.pubkey"名字都不可变。因为这些名字已经在its文件中静态定义，如果改变则会打包失败。

16.30.5 固件签名

`./build.sh firmware`签名固件，启动关键日志：

```
## Verified-boot: 1
sha256,rsa2048:dev## Verified-boot: 1
```

注意：SDK的`./build.sh`命令编译并签名固件，实际生产时用`./build.sh firmware burn-key-hash`生成的固件，才会把key的hash写到otp以实际使能安全启动。

16.30.6 应用场景

通常同一个公司开发整套固件并生产，这种场景按上面步骤即可。

如果是公司A负责固件开发和测试，公司B负责固件签名，那公司A用`./build.sh`编译好固件，把output/image打包给公司B签名。

公司B签名步骤：

```
# 在固件目录下
# boot.img env.img idblock.img oem.img rootfs.img uboot.img update.img userdata.img
download.bin fit_signcfg secure-boot-sign-tools
# 1. 生成key（执行一次，并保存key）
mkdir -p secure-boot-key-dir
./secure-boot-sign-tools/rk_sign_tool kk --bits 2048 --out ./secure-boot-key-dir
mv ./secure-boot-key-dir/private_key.pem ./secure-boot-key-dir/dev.key
mv ./secure-boot-key-dir/public_key.pem ./secure-boot-key-dir/dev.pubkey
openssl req -batch -new -x509 -key ./secure-boot-key-dir/dev.key -out ./secure-boot-key-
dir/dev.crt

# 2. 签名固件
./secure-boot-sign-tools/fit-sign.sh --key-dir ./secure-boot-key-dir \
    --src-dir ./ --out-dir ./  --burn-key-hash

# 支持rollback，格式镜像+rollback-index，例如设置uboot.img的rollback-index为2，设置boot.img的
rollback-index为3
./secure-boot-sign-tools/fit-sign.sh --key-dir ./secure-boot-key-dir \
    --src-dir ./ --out-dir ./  --burn-key-hash \
    --rollback-index uboot.img 2 boot.img 3
```

16.31 如何下载NPU模型转换工具以及runtime库

SDK没有带NPU模型转换工具以及runtime，需要从以下github地址下载。

NPU模型转换工具下载地址：

```
https://github.com/airockchip/rknn-toolkit2
```

NPU runtime库下载地址：

```
https://github.com/airockchip/rknn-toolkit2/tree/master/rknpu2
```

16.32 如何测试大语言模型（LLM）和视觉语言模型（VLM）

按以下命令编译，然后烧录固件。

```
./build.sh clean
./build.sh lunch BoardConfig-EMMC-RK801-RV1126B_EVB1_V10_V11-LLM_VLM_64BIT.mk
./build.sh all
```

固件启动脚本RkLunch.sh, 默认跑LLM Demo。
详细开发参考《Rockchip_Developer_Guide_Linux_RKAI_CN.pdf》。

16.33 如何读取CPU序列号

CPU序列号（即CPU ID）。

```
cat /sys/module/rockchip_cpuinfo/parameters/id
```

16.34 如何开启 DDR 变频

EVB 默认未开启 DDR 变频功能。如有开启需求，以 EVB1 为例，可通过以下配置进行开启：

```
--- a/arch/arm64/boot/dts/rockchip/rv1126b-evb.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rv1126b-evb.dtsi
@@ -120,6 +120,15 @@ vcc12v_dcin: vcc12v-dcin {
        };
 };

+ &dfi {
+        status = "okay";
+ };
+
+ &dmc {
+        center-supply = <&vdd_logic>;
+        status = "okay";
+ };
```

defconfig配置：

```
+ CONFIG_ARM_ROCKCHIP_DMC_DEVFREQ=y
+ CONFIG_PM_DEVFREQ_EVENT=y
```

```
# 查看DDR频率的可设置范围
cat /sys/class/devfreq/dmc/available_frequencies
# 设置DDR governors为userspace模式
echo userspace > /sys/class/devfreq/dmc/governor
# 设置DDR频率
echo 780000000 > /sys/class/devfreq/dmc/userspace/set_freq
# 查看当前DDR频率
cat /sys/class/devfreq/dmc/cur_freq
```

17. 注意事项

在windows下复制源码包时，linux下的可执行文件可能变为非可执行文件，或者软连接失效导致无法编译使用。
因此使用时请注意不要在windows下复制源代码包。